RSDK

for S32R41

Rev.0.8.10/16.12.2024

Chapter 1

RADAR SDK Overview

RADAR SDK Overview

1.1 Introduction

This manual provides information about the interface, components, operating modes, folder structure, build and execution of the NXP Radar SDK software product.

1.2 Product Description

The Radar SDK provides basic radar processing algorithms and device drivers for S32R hardware devices. Its purpose is to facilitate radar algorithm development (using SPT kernels, Matlab models, DSP functions creation of higher level algorithms (starting from the basic blocks supplied with RSDK) , enablement of radar front-end devices and easy application development by integrating driver and platform support.

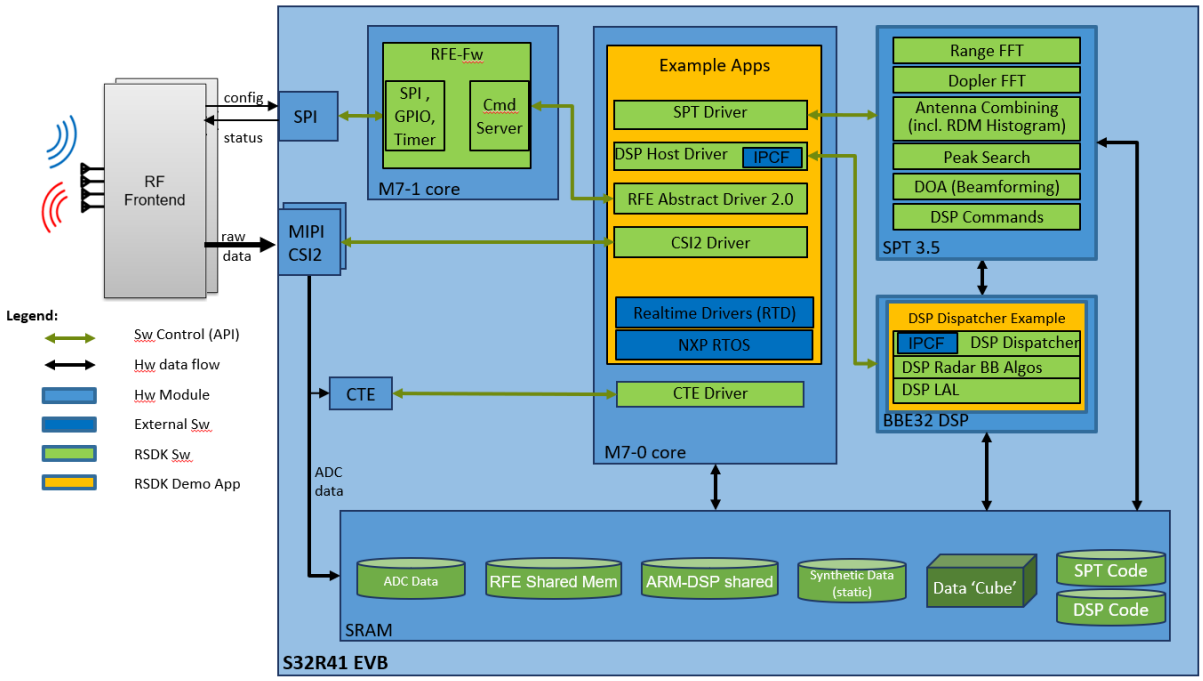


Figure 1.1 Radar SDK Overview on the S32R41

1.2.1 RSDK Modules

The Radar SDK provides the following software modules :

RF Abstraction Layer 2.0 for NXP Radar Front Ends: provides a common software interface to configure and run different hardware front-ends.

CSI2 Driver: this module provides an interface between the user application running on the Application CPU cores and the PacketProcessor and only for external data flows, the MIPI-CSI2 interface. Details provided in CSI2 User Guide and CSI2 Driver.

CTE Driver: this module provides an interface between the user application running on the CPU cores and the CTE unit. Details provided in CTE User Guide and cte.

SPT Driver: this module provides an interface between the user application running on the CPU cores and the SPT hardware accelerator. Details provided in SPT User Guide and spt_driver_api.

SPT Kernels: precompiled routines of SPT code packed into a library, implementing parts of the radar processing flow from Range and Doppler analysis up to Peak Detection and DoA. Details provided in spt_kernels_api.

DSP Host Driver: Enables scheduling of processing jobs on the BBE32EP directly from the host CPU (ARM) , without going through SPT . Details provided in DSP Host Driver. and a few dummy functions which demonstrate the integrated control and data flow with the ARM cores SPT and the DSP Dispatcher. Details provided in DSP Radar Baseband Algo Library.

DSP Linear Algebra Library: BBE32EP function library for accelerated vectorial linear algebra operations. Details provided in DSP Linear Algebra Library User Guide and DSP LAL API.

Application-level support tools: Source code and debug scripts to help develop and debug working applications on the S32R platform. Described in app_tools1. These tools are provided as helper code for applications, it is not within the scope of this release to perform full testing on them.

Matlab bit-exact model for SPT kernels: SPT functionality exposed as Matlab “mex” functions with mathematical sense. Can be used by designers to evaluate the numerical performance of their SPT algorithm in Matlab environment without considering SPT hardware constrains (most notably memory layout). `Kernels info and usage` document describes the functionality and usage of these models.

1.2.2 RSDK Example Apps

The Radar SDK includes executable applications showing how to enable and integrate the Radar Front-end API, CSI2 Driver, SPT Driver, SPT Kernels, DSP Host Driver, DSP Dispatcher and algorithms on the S32R41 platform. These apps currently run on the S32R41 EVB board, with or without a connected radar front-end.

See the RSDK Example Applications for details.

Note

These applications are not tested exhaustively and should be considered as demonstrative code only. They exercise only certain configurations of the RSDK modules' APIs (e.g. a single combination of chirps per frame, samples per chirps, operating mode etc).

The applications which use the SPT for radar data processing configure the size of the input data acquisition buffer to 2 chirps' worth of data, since only this size is compatible with the RSDK SPT Range FFT kernels.

1.3 Deployment environment and dependencies

The following tools and dependencies are used for building and executing the RSDK modules and sample applications:

- Software:
  - NXP S32 Design Studio for S32 Platform, including: S32R41 development package, Radar Extension package for S32R41 and BBE32 DSP Add-on package for S32R41
  - NXP RTOS and Real-time Drivers for S32R41
  - NXP Inter Platform Communication Framework for S32R41
  - Lauterbach Trace32 Debugger
  - Matlab, Matlab Signal Processing Toolbox
- Hardware:
  - Host PC running Windows 10 with optional GNU environment (e.g. mingw or Ubuntu console)
  - S32R41 processor (S32R41AA, or S32R418AB, or S32R416LB) mounted on S32R41 EVB board.
  - S32R41 TEF82XX adapter + TEF82XX CAB
  - Lauterbach Power Debug Interface and cable (with license for A53 and BBE32 debugging)

The particular tool versions are specified in the Release Notes for each individual release.

1.3.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors: S32R418AA

The complete device revision (cut, configuration, trim etc) is specified in the Release Notes.

1.3.2 Runtime environment

The RSDK software modules can be integrated in user applications running on NXP RTOS running on M7-0 core

1.3.3 RSDK module dependencies

The RSDK software apps and modules have the following internal and external compile-time dependencies:

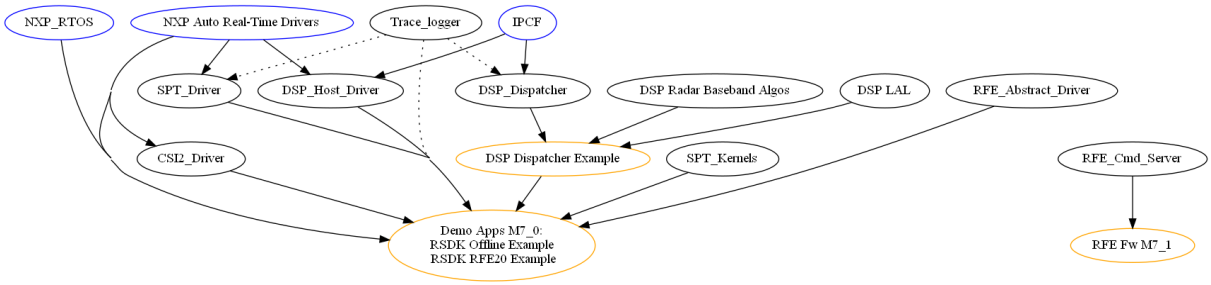


Figure 1.2 Radar SDK Module Dependencies

1.4 Package Folder Structure

The structure of the software deliverable package is the following:

```
rsdk_drivers/
    api/                                    Common header files used by all RSDK modules and applications.
        rsdk_status.h                       Defines the return datatype for RSDK API calls, including SPT and RFE error codes.
        rsdk_status_helper.h                Helper functions to aid in error reporting.
        rsdk_toolchain_helper.h             Helpers for accessing linker-command-file-defined symbols.
        typedefs.h                          Type definitions for all compilers.
        esdk_version.h                      Defines for RSDK AUTOSAR and Plugin versions of the current release.
        rsdk_osenv.h                        Defines the targeted OS environment.
    Apps/
        common/                             Common application header/source files
        DSP_Dispatcher_example/
            src/
                IPCF/
                    SAF85XX/                Location for IPCF sources required by DSP Dispatcher
                        ipcf_Ip_Cfg.h
                        ipcf_Ip_Cfg.c
            Makefile
        RSDK_offline_example/
            bin/                            Location of ELF files
            data/                           Location of in/out/ref data
            include/                        Application specific header files
            matlab/                         MATLAB visualizer for rhe data
            project/                        Design Studio project files
            src/                            Application specific source files
            T32/                            T32 scripts for Lauterbach
    autosar_plugins/
        SAF85XX/
            EBT/eclipse/plugins/
                Csi2_TS_T40D47M9I5R0        EB Tresos plugin for CSI2 Driver
                Cte_TS_T40D47M9I5R0         EB Tresos plugin for CSI2 Driver
                Dsphd_TS_T40D47M9I5R0       EB Tresos plugin for DSP Host Driver
                Spt_TS_T40D47M9I5R0         EB Tresos plugin for SPT Driver
            S32DS/
                Csi2                        S32DS plugin for CSI2 Driver
                Cte                         S32DS plugin for CTE Driver
                Dsphd                       S32DS plugin for DSP Host Driver
                Spt                         S32DS plugin for SPT Driver
                rsdk_SAF85XX_itm_manifest.xml   File that describes the link between the S32DS and Tresos files for each plugin
                sdk_components_rsdk.xml     File to help integrate S32DS plugins in DS
                RadarSDK_install_to_DS.py   Script to install all RSDK plugins and helper files under DS
    Docs/
        RSDK_User_Manual/                   The HTML version of the RSDK User manual.
        RSDK_User_Manual.pdf                PDF version of the RSDK User manual - User guide and general info and guidelines
        CSI2_Interface.pdf                  PDF version of the RSDK User manual - CSI2 integration details
        CTE_Interface.pdf                   PDF version of the RSDK User manual - CTE integration details
        DSP_Interface.pdf                   PDF version of the RSDK User manual - DSP integration details
        SPT_Interface.pdf                   PDF version of the RSDK User manual - SPT integration details
    CSI2/
        CSI2_driver/                        CSI2 driver which is used as data acquisition interface
            include/                        Header files
                low_level/CDD_Csi2.h        Defines the CSI2 Driver API (Autosar-CDD compliant)
            src/                            Source files
    CTE/
        CTE_driver/                         CTE driver which is used as signal triggering between different components
            include/                        Header files
                low_level/CDD_Cte.h         Defines the CTE Driver API (Autosar-CDD compliant)
            src/                            Source files
    DSP/
        DSP_common/                         Common source and header files used by multiple DSP modules
            include/
            src/
        DSP_host_driver/
            include/                        Header files
                CDD_Dsphd.h                 Defines the DSP Host Driver API (Autosar-CDD compliant)
            src/                            Source files
        DSP_dispatcher/
            api/                            Dispatcher API header file
            bin/                            location of the binary library file
            project/                        Design Studio project files
            src/                            Source and header files
            Makefile                        For building the sw module outside S32 Design Studio GUI.
        DSP_radar_bb_algos/
            api/                            library API header file
            bin/                            location of binary library file
            project/                        Design Studio project files
            src/                            Source files
            inc/                            Header files
            Makefile                        For building the sw module outside S32 Design Studio GUI.
        DSP_lal                             Linear Algebra Library
            api/                            library API header file
            bin/                            location of the binary library files
            build/                          build configuration, Makefile
            DSP_lal_demos/                  Function demo for RSDK application
            include/                        header files
            project/                        Design Studio project files
            source/                         Source files
    platform_setup/
        config/                             IPCF and RTD config for offline example
        include/                            Platform setup header files
        src/                                Platform setup source files
    SPT/
        SPT_driver/
            include/                        Header files
                common/CDD_Spt.h            Defines the SPT Driver API (Autosar-CDD compliant)
            src/                            Source files for SPT Driver
        SPT_kernels/
            api/
                rsdk_spt_kernels.h          API header file for SPT Kernels lib.
                rsdk_spt_kernels_< hw_platform >.h  Platform-specific API definitions for the SPT Kernels lib.
            bin/                            SPT Kernels libraries
            include/                        Header files
            project/< hw_platform >/        Design Studio project files
            src/< SPT_hw_platform >/        Source files for SPT kernels
            Makefile                        For building the sw module outside S32 Design Studio GUI.
    Tools/
        Build/
            dsp_hex_file_gen.sh             Script used in the BBE32 DSP build process to convert the *elf into a ascii hex image
            dsp_ABL_gen.py                  Script used in the BBE32 DSP build process to convert the *elf into a binary image
            SAF85XX_blob_create.py          Script used to create the bootloader image of the "Radar Control App" demo.
        C_model/                            C Model for DSP Linear Algebra Library
            api/                            Main header files
            bin/                            C model library
        SPT_bitexact_model/                 Matlab source code and examples for SPT algorithm
        Debug_tools/                        Source code and scripts used for file IO and application debugging
        Trace/                              Sample code and lib used for producing a light-weight application trace log
        Trace_parser/                       Files for data conversion from binary trace recordings to text
    Makefile, compilerdefs.mak              Used for batch building of non-Autosar RSDK components (e.g. SPT and DSP code)
```

1.5 Performance Metrics and Memory Footprint

Measurement reports for code size, SRAM usage, stack usage and profiling are not available for this release.

1.6 Rebuilding the RSDK libraries

The software components for the M7 core (SPT Driver, CTE Driver, CSI2 Driver, DSP Host Driver) are Autosar-CDD compliant and are delivered only as source code. They are meant to be integrated and built directly as part of the executable application. See RSDK RFE processing Example for details.

The SPT and BBE32 libraries come as precompiled binaries in the RSDK release. They can be rebuilt either from their corresponding S32 Design Studio projects, or using the < RSDK >/Makefile (see "make" command below).

When using make, the typical command line would be:

```
< rsdk_path >$ make all PLATFORM=S32R41 OSENV=sa
```

See also: "make help" and /compilerdefs.mak for a list of build options

Detailed build instructions are provided for each software component:

- How to build the RFE abstraction drivers
- How to build the SPT driver and SPT Kernels
- How to build the DSP Dispatcher, DSP Host Driver, DSP LAL and DSP Radar Algos library

1.6.1 Build configuration

Target Core	Sw modules	Toolchain
ARM M7	SPT Driver, CSI2 Driver, CTE Driver, DSP Host Driver, RFE Driver, Example apps	NXP GCC (included in NXP S32 Design Studio)
BBE32	DSP Dispatcher, DSP Radar BB Algos, DSP LAL, DSP Dispatcher Example	Cadence Xtensa Tools: xt-clang compiler, xt-ld linker (included in NXP S32 Design Studio)
ARM M7	SPT Kernels	SPT 3.5 Assembler (included in NXP S32 Design Studio)

The build options (compiler, assembler, linker) can be found in the S32 Design Studio projects for RSDK libraries and applications , or in the makefiles which are automatically generated when building the binaries

1.7 Integrating the RSDK software components

The software components for the M7 core (SPT Driver, CTE driver, CSI2 Driver, DSP Host Driver) are integrated as source code and built directly as part of the executable application. See RSDK RFE processing Example for details.

Each software component deployed on the M7 core has an Autosar-compliant:

- Tresos configuration plugin, located in /autosar_plugins/< hw_platform >/EBT/eclipse/plugins folder.
- S32DS configuration plugin, located in /autosar_plugins/< hw_platform >/S32DS folder.

Attention

In order to make the RSDK plugins available to **EB-Tresos** application, a .link file is required in the < local↩ _path >/EB/tresos/links/ folder, pointing to the **rsdk/autosar_plugins/**< **hw_platform** >**/EBT/** folder.
In order to see RSDK plugins into **Design Studio**, the .py script located in the **rsdk/autosar_plugins/**< **hw_platform** >**/S32DS/** folder needs to be run.

Full description and usage instructions for each plugin are provided in its component corresponding "User Guide" section.

The SPT and BBE32 libraries are pre-built and are linked directly to the executable applications.

All RSDK libraries are statically linked and no dynamic memory allocation is done by the library functions.

A naming convention applies to all symbols externally-visible from RSDK libraries

- prefix (starting with rsdk<component>) is defined for every module. This prefix is applied to the names of all externally visible functions, variables, types and C macro definitions
- prefixes include identification of variable scope/storage class or pointer types
- macros in uppercase, variables and functions in camelcase, with variables starting lowercase and functions starting uppercase
- filenames do not include space character.

The user must place all RSDK sections and symbols in memory explicitly by use of a linker file.

Special attention is required when integrating the BBE32 DSP image into the M7 executable, since the DSP sections must be placed in memory at precise locations specified in the linker file of the DSP executable. See RSDK Offline Example and the DSP Dispatcher Example project in /Apps/DSP_Dispatcher_example for reference.

Apart from the RSDK component libraries, there exist a number of application-level support tools. These should be attached to the application itself as sources (ie. no library). See also debug tools. See the RSDK Example Applications provided with this release for integration examples. They are the perfect starting point for a new application.

1.8 Release version

This document pertains to the **NXP Radar SDK release 0.8.10** for the S32R41 platform. For details concerning this product please refer to the Release Notes.

1.8.1 Doxygen documentation versioning

Project Name	RSDK
Document ID	RSDK_UM_S32R41
Version	4.1
Date	December 18 2024
Status	Release

Chapter 2

RSDK Example Applications

2.1 Introduction

The Radar SDK includes executable applications which show how to integrate the functionality of the SPT Driver, SPT Kernels, RFE Abstract API, CSI2 Driver, DSP Dispatcher and DSP Host Driver into a standard radar processing flow on the S32R platform.

The applications can be found in ***/Apps*** folder.

2.2 Running RSDK applications on S32R41

This section describes the common hardware and software configuration used for running RSDK applications on the S32R41 EVB.

2.2.1 Hardware setup

- Check the switches configuration on your EVB board. Testing of RSDK executables were done with the following configuration:
  - BOOTCFG[0] must be **ON** (as illustrated in the image below)
  - BOOTCFG[11] must be **ON**

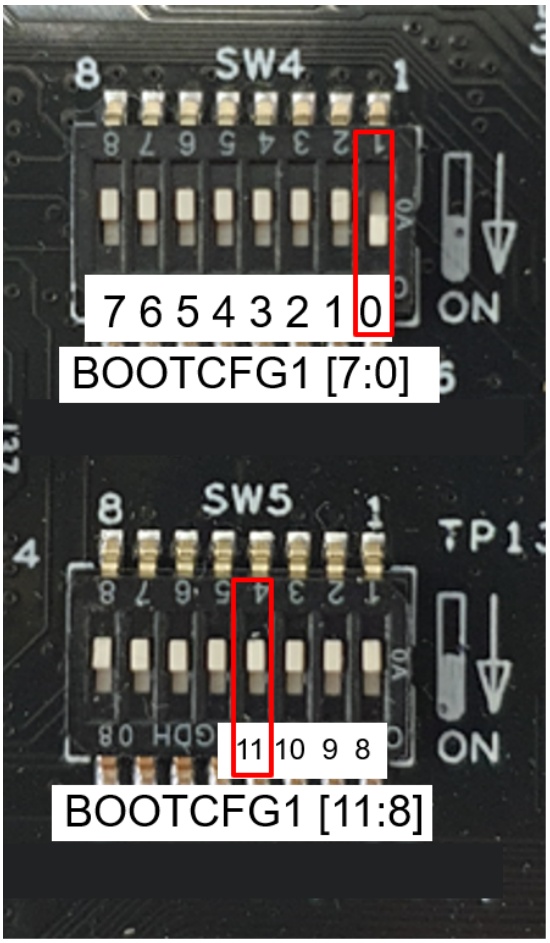


Figure 2.1 S32R41 EVB configuration

- Steps required for the initial board setup:
  - BOOT_MODE jumper J29 should be in 6-4; 5-3 position (serial boot)
  - Refer to "Writing Blob File to QSPI Flash Memory" in the **S32R41 Diagnostic Tool RM** to flash the QSPI image.
  - BOOT_MODE jumper J29 should be in 6-4; 3-1 position (QSPI flash boot).

2.3 RSDK Offline Example

This application, located in /**Apps**/**RSDK_offline_example** demonstrates the integration of the SPT Driver, SPT Kernels and DSP Dispatcher.

No radar front-end is required, data acquisition is emulated by the CPU core using data files. Since there is no realtime synchronization signal from the front-end to the SPT, this application must be linked against the 'offline' version of the spt_kernels library (librsdk_SPT_kernels_offline_gcc_< hw_platform >.a).

It can be run on the S32R41 EVB board, on one A53 core.

2.3.1 Building the application

Before building the application, make sure the SPT Driver library, SPT Kernels library, the DSP Host Driver and the DSP Dispatcher Example boot image are already built.

Note

Trace logger functionality is controlled by the following preprocessor macro in the C cource code:
`#TRACE_ENABLE`

The application is, by default, build with this macro defined, so Trace Logger Library must be already built.

2.3.1.1 Building the DSP image

This image is created when building the /Apps/DSP_Dispatcher_example/ application, which links the DSP Dispatcher , DSP Radar Algos and DSP LAL algos libraries. The linker configuration for this application is located in /Apps/DSP_Dispatcher_example/src/linker/min-rt-local-rsdk.

The DSP executable image can be built in S32R Design Studio: import the .project file located located in /Apps/↩ DSP_Dispatcher_example/project/, then press the 'Build' button.

The elf image is produced in /Apps/DSP_Dispatcher_example/bin/, but it cannot be integrated directly in the RSDK_offline_example. It is translated automatically to a set of global data arrays in /Apps/DSP_Dispatcher↩ _example/src/host_integration/dsp_image_< hw_platform >_debug/release.c, by the /Tools/Build/dsp_hex_file_↩ gen.sh script, which is invoked at the end of the build process.

Note

The DSP image option "debug/release" can be selected using the following preprocesor macro:
`#BBE32_DEBUG`

The application can be built from Design Studio - import the .project file located at < rsdk_path >/Apps/RSDK_↩ offline_example/project/ S32R41/ into Design Studio, then press the 'Build' button.

2.3.2 Running the application

To launch the executable :

- connect the Lauterbach Power Debug Interface to PC.
- open the Lauterbach "Trace32 for ARM" debugger
- after the initialization script in the T32 window has finished, change the working directory:
  `cd < rsdk_path >/Apps/RSDK_offline_example/T32/S32R41`
- run the cmm script with the following command:
  `do S32R41_RSDK_offline_example.cmm "debug"`

The application is expected to print progress messages on the Trace32 console, then indicate end of execution and no error messages. At this point the SPT processing results should be created in the < rsdk_path >/Apps/RSDK↩ _offline_example/data/out/< chip > folder.

Finally you can compare the results (data/out/< hw_platform >/) with the reference data in /Apps/RSDK_offline_↩ example/data/ref/< hw_platform > for bit-exactness.

Note

The < path_to_rfe_bins > represents the absolute path towards the bin folder containing the "CODE" and "DATA" files of the RFE firmware (e.g. "< rsdk_path >rfe/rfe/OC-ES2/rfeFw/bin")

2.3.3 Running the application with S32DS Debugger

The first step in running the application with S32DS Debugger is recompiling the application for S32DS Debugger. Add the preprocessor macro "STDIO" (Project Properties -> C/C++ Build -> Settings -> Standard S32DS C Compiler -> Preprocessor). Change library support to "newlib Semihosting" (Project Properties -> C/C++ Build -> Settings -> Target Processor -> Libraries support).

A Design Studio project is provided in /Apps/RSDK_offline_example/project/S32R41/S32Probe/ to facilitate debugging with the S32DS Debugger. The "RSDK_offline_example_S32R41 Debug_M7.launch" configuration is provided for starting a debug session on the M7 core in AUTOSAR environment.

Open the launch config from the S32 Design Studio menu "Run->Debug Configurations.." and customize the debugger connection settings, as shown in the figure below:

- Select Interface S32 Debug Probe - Ethernet (default)/Interface S32 Debug Probe - USB (if your probe is connected via USB)
- If Ethernet is used enter Hostname or IP of the probe
- Apply settings

Finally, click on "Debug" to start the session.

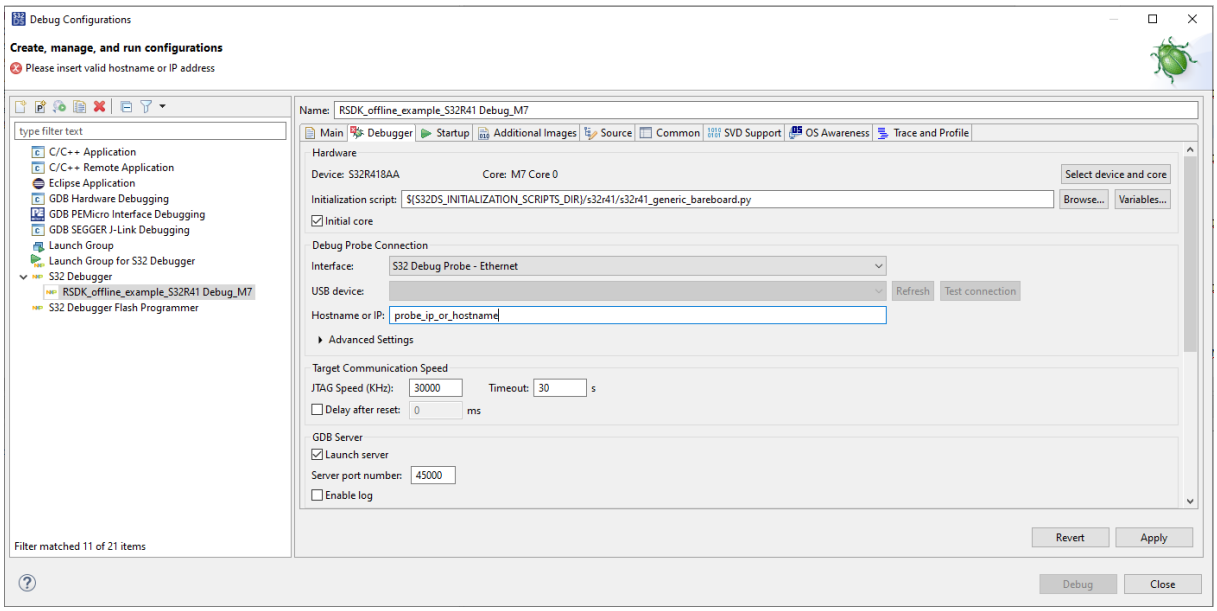


Figure 2.2 RSDK_offline_example_S32R41 Debug Configuration

2.3.4 Verifying the output data

The demo includes a matlab script (in /Apps/RSDK_offline_example/matlab/RSDK_offline_example.m), which can be used to visualise and interpret the data produced by the embedded application.

It can be called from the Matlab command line as follows:

- **RSDK_offline_example('S32R41)** . It computes and plots a Range-doppler spectrum of the input data and also plots the results of the SPT doppler, SPT peak search and BBE32 CA-CFAR algorithms.

2.4 RSDK RFE processing Example

This example, located in < **rsdk_path** >/**Apps**/**RFE_processing_example** demonstrates the integration of the RFE abstract 2.0 with CSI2 Driver, SPT Driver, SPT Kernels, DSP Host driver and DSP algo libs.
The example runs on two cores:

- M7_1 core - runs RFE firmware and controls the frontend (see RFE abstract 2.0 architecture)
- M7_0 core - main application core; ensures synchronization of RFE with signal processing part.

  The configuration for NXP RTOS, RTD drivers used for platform setup and RSDK drivers is done prior to build using Tresos project for M7_0 (*/Apps/RFE_processing_example/RFE_processing_example_M7_0/autosar_cfg/S32R41/EBT/*). For M7_1 core, the Design Studio config tool is used (via mex in the DS project). Configuration files for both cores are already built in release package.
  The application follows a simple processing flow (see figure) for a cascaded front-end configuration: dual frontend(8 Rx), 512 samples 128 chirps.

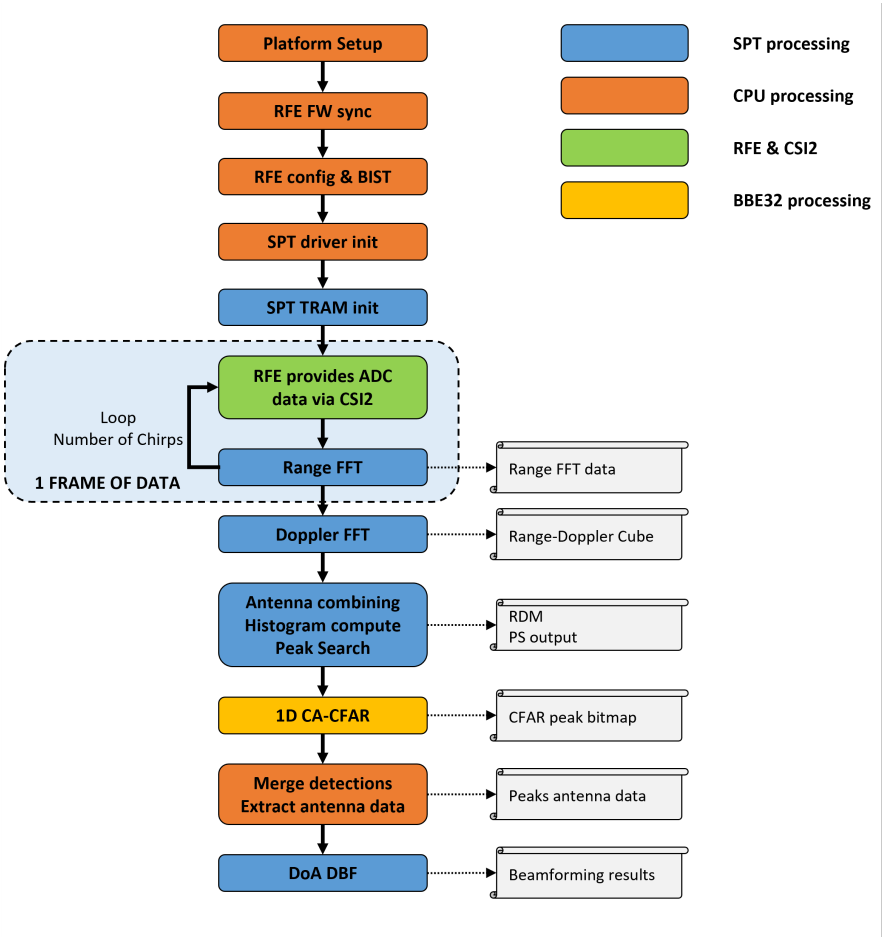


Figure 2.3 RSDK Example flow with RFE data acquisition

2.4.1 Building the application

There are a couple of dependencies for building the example (pre-built in RSDK package):

- **RFE2 firmware driver M7** - used by M7_1; can be rebuilt using DS project found in *./RFE_abstract/RFE_driver2/RF_FW/project/S32R41/RFE_abstraction_2_FW_M7/*

- **SPT kernels** - used by M7_0; can be rebuilt using DS project in *./SPT/SPT_kernels/*
- **DSP Dispatcher example** image running on DSP (loaded by M7_0 core at runtime). Can be rebuilt with DS from *./Apps/DSP_Dispatcher_example/* This project has the following dependencies:
  - **DSP Dispatcher** found in *./DSP/DSP_dispatcher/*
  - **DSP Algos library** found in *./DSP/DSP_radar_bb_algos/*
  - **DSP LAL library** found in *./DSP/DSP_lal/*

For building the application use Design Studio 3.5 and projects located in *./Apps/RFE_procesing_example* :

- build RFE_processing_example_M7_0
- build RFE_processing_example_M7_1

Note

If there are issues with building check the following variables te be defined and corresponding to the install paths on user PC:
GCC toolchain in DS - **S32DS_ARM32_GNU_9_2_NEWLIB_DIR** - example: *${S32DS_ARM32_GNU_9_2_TOOLCHAIN_DIR}/arm-none-eabi/newlib*
Root of Platform SDK - **PLATFORMSDK_S32R_S32R418AA_M7_0_2.0.0_PATH** - example:*${eclipse_home}../S32DS/software/PlatformSDK_S32R41/*

2.4.2 Running the application

Note

The example has only been tested on **S32R41 AA** phantom.

For loading, running and debugging the application on the board, load and run the **RFE_processing_example.cmm** script in Trace 32 (*./Apps/RFE_processing_example/RFE_processing_example_M7_0/T32/*)
Data will be output in the *./data/out/S32R41/* folder
For data visualization go to *./Tools/DataVisualizer* and use Matlab to execute

```
RunRadarApp('S32R41',"../../Apps/RFE_processing_example/RFE_processing_example_M7_0/data/')
```

Note

The RFE configuration is hardcoded in **rfeConfig.c** which is generated with the RFE config tool (see RFE Abstract chapter in User guide) using **rfeConfig0.xml** and **rfeConfig1.xml** (*./Apps/RFE_processing_example/RFE_processing_example_M7_0/data/in/S32R41/*). The config needs to match application settings defined in **ExampleGetConfig(void)** function in the source code. Also, for proper data visualiation, these also need to match **s32r41_tef82xx_config_0.ini** file in *./data/in/S32R41* path.

2.4.3 Running the application without processing

Rebuild the M7_0 executable with **APP_RUNTIME_CONFIG** preprocessor macro defined. Example will load the RFE configuration blobs at runtime from *./data/in/S32R41/config_0/* and run ADC data acquisiton only. Current example shows a dual sequence acquisition.

Note

This build configuration will allow some flexibility in changing RFE configuration without the need to rebuild the example, however, there are some limitations:

- maximum number of sequences is 3
- one CSI2 virtual channel per sequence meaning
  - all chirp profiles within a sequence use the same CSI2 virtual channel
  - different sequences use different virtual channels
- CSI2 is configured to store all chirps in memory; all sequences need to fit in *heapBuffer*

The same T32 is used to run the example. For visualizing data use:
`RadarCycleDataViewer('S32R41',"../../Apps/RFE_processing_example/RFE_processing_example_M7_0/data/')`

from the same *./Tools/DataVisualizer* folder.

2.4.4 Running the application with S32DS Debugger

The first step in running the application with S32DS Debugger is recompiling the M7_0 application for S32DS Debugger:

- Add the preprocessor macro **"STDIO"** (Project Properties -> C/C++ Build -> Settings -> Standard S32DS C Compiler -> Preprocessor).
- Change library support to **"newlib Semihosting"** (Project Properties -> C/C++ Build -> Settings -> Target Processor -> Libraries support).

A Design Studio project is provided in *./Apps/RFE_processing_example/RFE_processing_example_M7_0/Debugger/* to facilitate debugging with the S32DS Debugger. The "RFE_processing_example_M7_0_Debug_RAM.launch" configuration is provided for starting a debug session on the M7-0 core in AUTOSAR environment and "RFE_processing_example_M7_1_Debug_RAM.launch" configuration is provided for starting a debug session M7-1 core. Use "RFE_processing_example_group.launch" configuration to run the application.

Open the launch config from the S32 Design Studio menu "Run->Debug Configurations.." and customize the debugger connection settings, as shown in the figure below (only modify the M7_1 configuration):

- Select Interface S32 Debug Probe - Ethernet (default)/Interface S32 Debug Probe - USB (if your probe is connected via USB)
- If Ethernet is used enter Hostname or IP of the probe
- Apply settings
- Open Launch Group -> RFE_processing_example_group

Finally, click on "Debug" to start the session.

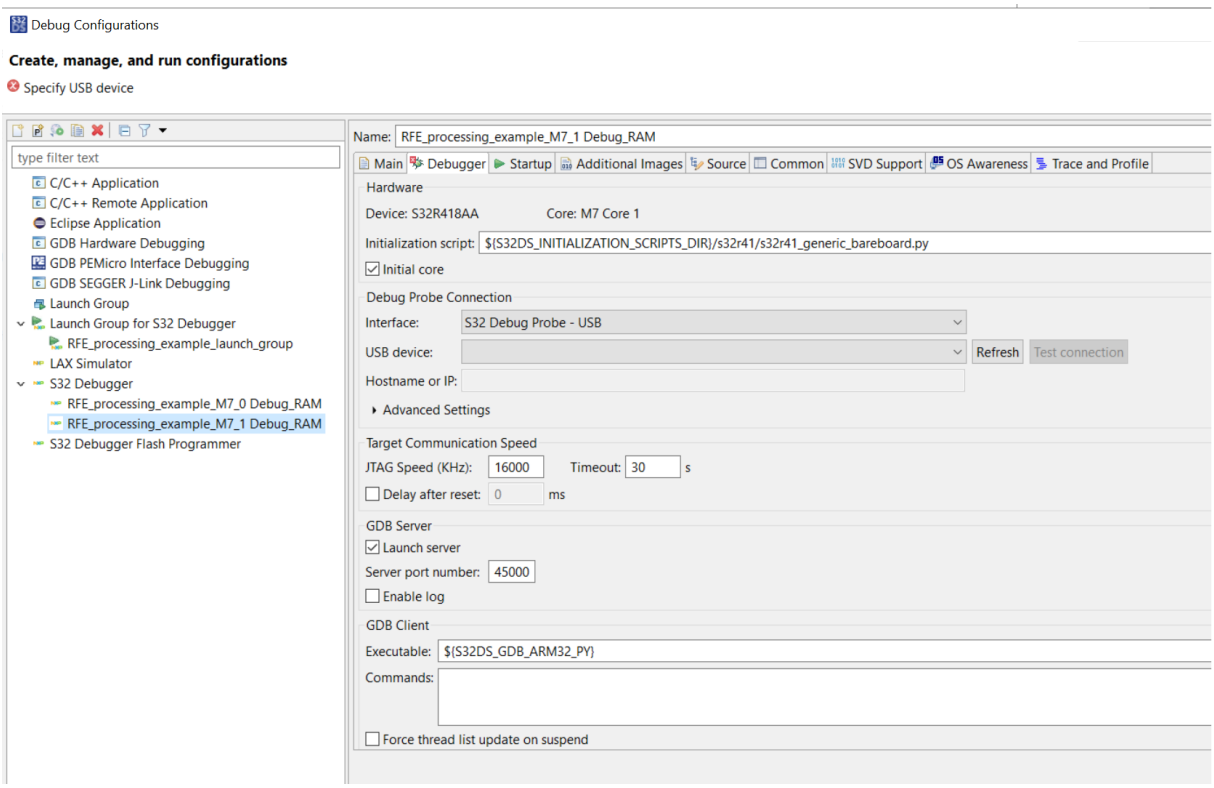


Figure 2.4 RSDK_RFE20_example Debug Configuration

Chapter 3

CSI2 User Guide

3.1 Introduction

This section presents the **Radar SDK** software components provided for enablement of the **CSI2** data interface .
It provides details about their functionality and programming modes. It contains all guidelines for an easy integration.

3.2 Module description

The RSDK CSI2 module consists of the csi2 , software component.

The CSI2 Driver can be used as independent driver.

The CSI2 Driver serves as data input interface between the Radar Front End and the user application. Its purpose is to enable the data input flow from the Radar Front End to the local SRAM. The driver has the same API for all platforms and the same Operating System, but some data structures are different, adjusted to the working platform.
For a detailed description of the driver elements, see csi2.
For platform implementation see the appropriate platform **Reference Manual**.
For MIPI-CSI2 interface see the specific `MIPI-CSI2℠ interface` on MIPI Alliance pages.
The rsdk_sampleapps show how to integrate these software components into the user code.

3.3 CSI2 Module Usage

3.3.1 Environment

The tools and dependencies used for building and executing the RSDK modules are listed in Deployment environment and dependenci section

3.3.2 CSI2 driver usage

The CSI2 Driver is not built into a separate library. As usually AUTOSAR usage, the driver files must be integrated directly as source code in top-level applications deployed on the M7 core. Before building an application which include the CSI2 driver, there are 4 conditions for ensuring the proper functionality of the driver:

- All source files must be added and built along with the application
- The driver's interrupt handlers must be registered accordingly
- The interrupts callbacks must be defined at application level
- The necessary configuration build parameters must be provided, to respect the necessary functionalities and code footprint

It is recommended to use the plugin configurator to provide the CSI2 driver setup parameters, see details in CSI2 plugin usage. The setup parameters provided as example are generated using the plugin. But even not using the plugins to configure the driver, the files CDD_Csi2_PCCfg.h and CDD_Csi2_PCCbk.h must be provided if at compile time **RSDK_AUTOSAR** is defined. The driver states diagram is presented in the figure 1.

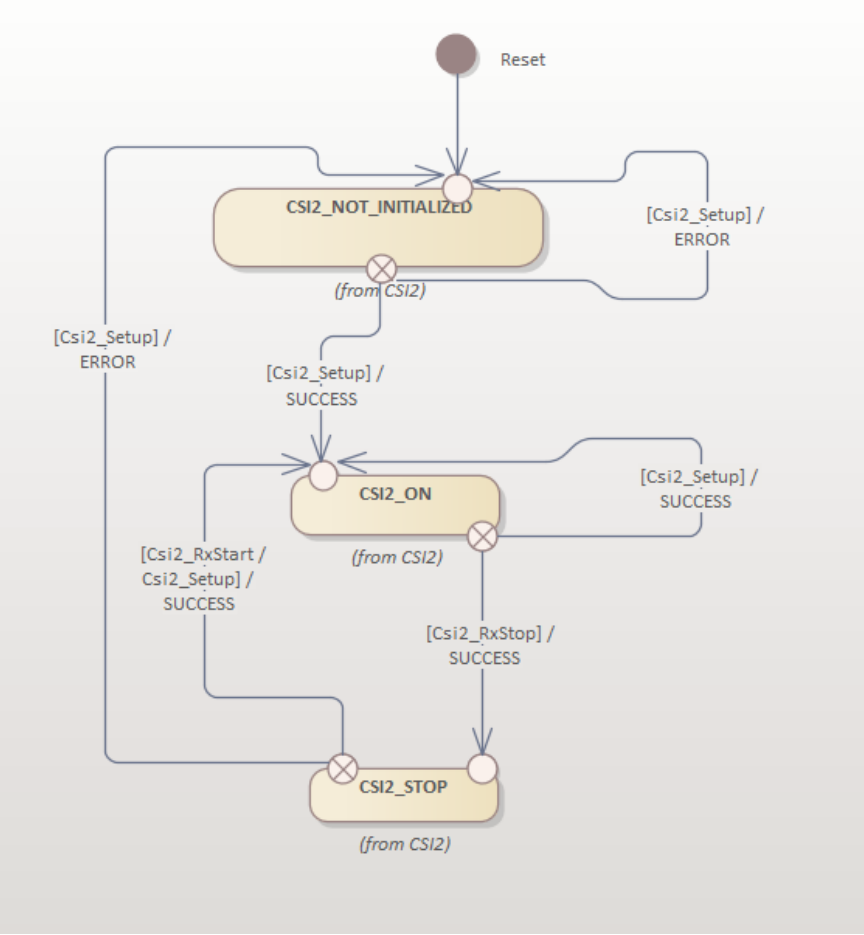


Figure 3.1 : CSI2 States Diagram

3.3.3 Initialization

Before any other usage, the CSI2 driver must be initialized. As described in CSI2 driver usage, an uninitialized driver interface will not accept any other procedure call. An uninitialized interface will not receive any data coming from the Front End.

CSI2 Driver initialization is done by calling Csi2_Setup. The initialization stage sets the internal state of the CSI2 Driver based on Csi2_SetupParamsType structure. Please read carefully the structure specs.
Csi2_Setup must be called first after reset.
The procedure can be called at any time no matter the current state of the driver.
If the procedure is successful, the driver state is **CSI2_ON** and the interface is prepared according the requested parameters. If the procedure is not successful, the state of the driver is **CSI2_NOT_INITIALIZED**, no matter which was the driver state before the call and no matter which was the encountered error. The hardware interface will be in an undefined state.

The Virtual Channel concept is a general concept, a physical channel can be used to transfer many logical data flows, applied to the **MIPI-CSI2^{SM} interface**.

For this interface there are available up to 4 Virtual Channels, numbered from CSI2_VC_0 to CSI2_VC_3, similar as the Virtual Channel Indicator in the data flow.
It is possible to use auxiliary data on CSI2 interface, if the front-end has this feature and the application need this, using pAuxConfig. The Virtual Channel and auxiliary data must use the same id and to use "connected" data types. If no need for auxiliary data, the pAuxConfig pointers must be set to NULL. More details in chapter Auxiliary data usage

Note

If using for Front-End management **RFE Abstract 2.0**, please be aware of the different approach of the CSI2 usage :

- the Rx CSI2 interface is setup on M7_0 core using this CSI2 driver
- the CSI2 interface/transmission is setup on M7_1 core
  So, it is mandatory for the application to understand the CSI2 transmission parameters of the Front-End at a specific time and to use the appropriate setup structure for Csi2_Setup.

Note

If using for Front-End management **RFE Abstract 2.0**, do not use Virtual Channel 3 (CSI2_VC_3) for receiving radar data, as this is used for Front-End BIST procedure. But the CSI2_VC_3 must be set according to the RFE Abstract 2.0 BIST requirements (4.8 RX BIST and 4.1 RF_Abstract 2.0 Communication).

Note

The NXP Radar Front Ends MR3003 and TEF810X are sending data only on Virtual Channel CSI2_VC_0, so only Virtual Channel 0 parameters must be filled. For the new TEF82XX all Virtual Channels can be used.

Note

For interface initialization please use the Tresos or S32DS application and the delivered plugins.

For vcParams.streamDataType use CSI2_DATA_TYPE_RAW12 for NXP Radar Front Ends TEF82XX.

To set the VC parameters, use pVCconfig pointers according to the VC number to receive: **CSI2_VC_0** for VC 0, **CSI2_VC_1** for VC 1, etc..

It is not possible to have many settings for the same VC. If the chirp data coming for the same VC is different on any of chirp length or number of chirps, use please the biggest numbers. In these cases the data will be correctly received, but some errors will be raised. These errors must be ignored.
If the data to be received for the same VC is different for other parameters, please use different setups and call Csi2_Setup each time is necessary before receiving data.

The necessary steps to use the CSI2 driver are :

- use the Tresos application or S32DS peripheral and the CSI2 plugin to generate the necessary setup parameters :
  - configuration parameters, defined in CDD_Csi2_PCCfg.h
  - callback functions, as the customer can use other names than the fixed ones, defined in CDD_Csi2_↩ PCCbk.h
  - setup parameters, defined in Csi2_PBCfg.c
- include into the application build files Csi2_PBCfg.c and the files from < rsdk >/CSI2/CSI2_driver/src/low↩ _level
- adjust the include buld paths to include < rsdk >/CSI2/CSI2_driver/include/low_level and the plugins generated files
- use at least Csi2_Setup function in the application code to start usage of the CSI2 interface

The generated CDD_Csi2_PCCfg.h parameters looks like :

```
/*==================================================================================================
*                                        DEFINES AND MACROS
==================================================================================================*/
/* Pre-processor switch to enable/disable development error detection for Csi2 API */
#define CSI2_DEV_ERROR_DETECT                   STD_ON

/* Pre-processor switch to enable/disable stop execution after error detection for Csi2 API */
#define CSI2_DEV_HALT_ON_ERROR                  STD_ON

/* Pre-processor switch to define single Csi2 management thread or multiple Csi2 management threads.
 * If single thread (which is the normal approach) - there are no necessary exclusive areas for the driver
     */
#define CSI2_SINGLE_MANAGEMENT_THREADS          STD_ON

/* Pre-processor switch to enable/disable version info report for Csi2 API                      */
#define CSI2_VERSION_INFO_API                   STD_ON

/* Pre-processor switch to enable/disable statistics usage for received data for Csi2 API       */
#define CSI2_STATISTIC_DATA_USAGE               STD_OFF

/* Pre-processor switch to enable/disable DC auto compensation for received data on CSI2 API    */

/* Pre-processor switch to enable/disable auxiliary data usage for received data on CSI2 API    */
#define CSI2_AUXILIARY_DATA_USAGE               STD_OFF

/* Pre-processor switch to enable/disable metadata usage for received data on CSI2 API    */
#define CSI2_METADATA_DATA_USAGE                STD_OFF

/* Pre-processor switch to enable/disable single ISR callback for CSI2 API                      */
#define CSI2_SINGLE_CALLBACK_USAGE              STD_ON

/* Pre-processor switch to enable/disable Rx start/stop usage in CSI2 API                       */
#define CSI2_RX_START_STOP_USAGE                STD_ON

/* Pre-processor switch to enable/disable power on/off usage in CSI2 API                        */
#define CSI2_POWER_ON_OFF_USAGE                 STD_ON

/* Pre-processor switch to enable/disable secondary functions usage in CSI2 API                 */
#define CSI2_SECONDARY_FUNCTIONS_USAGE          STD_OFF

/* Pre-processor switch to enable/disable internal frames counter usage in CSI2 API             */
#define CSI2_FRAMES_COUNTER_USED                STD_ON

/* Pre-processor switch to enable/disable usage of GPIO in CSI2 API                             */
#define CSI2_GPIO_USED                          STD_OFF

/* Pre-processor switch to enable/disable usage of SDMA in CSI2 API                             */
#define CSI2_SDMA_USED                          STD_OFF

/* Formal instance id for CSI2 driver, to be used at development time                           */
```

```
#define CSI2_INSTANCE_ID                        0u

/* The type of timer to be used for necessary execution delays                                  */
#define CSI2_TIMER_TYPE                         OSIF_COUNTER_SYSTEM_DUMMY
```

If using **OSIF_COUNTER_SYSTEM_DUMMY**, the time counting is done using local loops which are measured to be correct for a M7 core running at 400/320MHz. If the core is not running at this frequency, it will be necessary to adjust the value of the constant **CSI2_NS_DIVIDER**.

The generated code for Csi2_PBCfg.c file looks like :

```
/* ==================================================================
 *  Start of a params set for CSI2 unit
 * ==================================================================*/

/* ==================================================================
 *  Start of Virtual Channel parameters for this CSI2 unit
 * ==================================================================*/
extern uint8 *pBufDataVc0[];

Csi2_VCParamsType SetupParam_0_Params_VC_0 = {
    .streamDataType     = CSI2_DATA_TYPE_RAW12,
    .channelsNum        = 4u,
    .expectedNumSamples = 512u,
    .expectedNumLines   = 128u,
    .bufNumLines        = 128u,
    .bufLineLen         = 4096u,
    .bufDataPtr         = pBufDataVc0,
    .vcEventsReq        = 0u | CSI2_EVT_FRAME_END,
    .outputDataMode     = CSI2_VC_BUF_OUT_TILE8 | CSI2_VC_BUF_REAL_DATA | CSI2_VC_BUF_NO_FLIP_SIGN,
    .offsetCompReal     = {0,0,0,0},
    .offsetCompImg      = {0,0,0,0},
    .bufNumLinesTrigger = 1
};

extern uint8 *pBufDataVc1[];

Csi2_VCParamsType SetupParam_0_Params_VC_1 = {
    .streamDataType     = CSI2_DATA_TYPE_RAW12,
    .channelsNum        = 4u,
    .expectedNumSamples = 256u,
    .expectedNumLines   = 256u,
    .bufNumLines        = 256u,
    .bufLineLen         = 2048u,
    .bufDataPtr         = pBufDataVc1,
    .vcEventsReq        = 0u | CSI2_EVT_FRAME_END,
    .outputDataMode     = CSI2_VC_BUF_OUT_TILE8 | CSI2_VC_BUF_REAL_DATA | CSI2_VC_BUF_NO_FLIP_SIGN,
    .offsetCompReal     = {0,0,0,0},
    .offsetCompImg      = {0,0,0,0},
    .bufNumLinesTrigger = 1
};

extern uint8 *pBufDataVc2[];

Csi2_VCParamsType SetupParam_0_Params_VC_2 = {
    .streamDataType     = CSI2_DATA_TYPE_RAW12,
    .channelsNum        = 4u,
    .expectedNumSamples = 1024u,
    .expectedNumLines   = 64u,
    .bufNumLines        = 64u,
    .bufLineLen         = 8192u,
    .bufDataPtr         = pBufDataVc2,
    .vcEventsReq        = 0u | CSI2_EVT_FRAME_END,
    .outputDataMode     = CSI2_VC_BUF_OUT_TILE8 | CSI2_VC_BUF_REAL_DATA | CSI2_VC_BUF_NO_FLIP_SIGN,
    .offsetCompReal     = {0,0,0,0},
    .offsetCompImg      = {0,0,0,0},
    .bufNumLinesTrigger = 1
};

/* ==================================================================
 *  End of Virtual Channels parameters for this CSI2 unit
 * ==================================================================*/
/* Setup structure for CSI2_UNIT_LEVEL */
Csi2_SetupParamsType    Csi2SetupParamsList_0 = {
    .numLanesRx = CSI2_LANE_3,
    .lanesMapRx = {CSI2_LANE_0, CSI2_LANE_1, CSI2_LANE_2, CSI2_LANE_3},
    .initOptions = CSI2_DPHY_INIT_SHORT_CALIB,
    .rxClkFreq = 480,
    .vcConfigPtr = {&SetupParam_0_Params_VC_0, &SetupParam_0_Params_VC_1, &SetupParam_0_Params_VC_2},
};

/* ========================================
```

```
*  End of the params set for CSI2 unit
* ========================================*/
```

The simplest way of driver usage is :

```
status = Csi2_Setup(CSI2_UNIT_0, &Csi2SetupParamsList_0);
```

3.3.4 Callback usage

The most important interface interactions are the events. The driver is using the specified callbacks to announce the application about the errors or events occurred on the interface. According to the defined value of the CSI2_↩ SINGLE_CALLBACK_USAGE, the application must provide :

- for STD_ON = single callback, **Csi2_SingleIsrCallback**, for all raised interrupts, for errors or events
- for STD_OFF = different callback for each interrupt :
  - **Csi2_PhyIsrCallbackUnit** = reports errors at protocol level
  - **Csi2_PcktIsrCallbackUnit** = reports errors at packet level
  - **Csi2_EvtIsrCallbackUnit** = reports the required events
    The default names for the callbacks are the ones specified above, but configuring using the plugin is possible to provide any desired, C acceptable name.

The events reported by interrupt/callback are:

Interrupt ID / description	Error field	Error bit	Meaning
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_PHY_SYNC	A recoverable error: the high-speed leader sequence is corrupted, but a proper synchronization can still be achieved. Considered to be a “soft error”.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_PHY_NO_↩ SYNC	A proper synchronization cannot be achieved. This is considered a "hard error" and a new interface initialization could be necessary.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_PHY_ESC	DPHY has detected an illegal command sequence when entering escape mode.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_PHY_SESC	The number of bits received during a Low-↩ Power data transmission is not a multiple of eight when the transmission ends.

Interrupt ID / description	Error field	Error bit	Meaning
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_PHY_CTRL	DPHY has detected an illegal control sequence.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_↩ SPURIOUS_PHY	Improper IRQ call, with no one reporting error bit set.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::errMaskVC	CSI2_ERR_PACK_↩ ECC1	ECC code was computed and a single bit-error in the header was detected and corrected. Csi2_ErrorReport↩ Type::eccOneBitErrPos reports the error position. Csi2_ErrorReportType↩ ::errMaskVC field reports the Virtual Channel with this error.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::errMaskVC	CSI2_ERR_PACK_↩ ECC2	ECC code was computed and at least two bit-errors are detected in the received header. The line data is considered to be wrong and the data will be overwritten by the next line data. Csi2_Error↩ ReportType::errMaskVC field reports the Virtual Channel with this error.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::errMaskVC	CSI2_ERR_PACK_↩ SYNC	Several errors possible on the same Virtual Channel : • two consecutive Frame Start • the frame corresponding to the Frame Start is considered an error • two consecutive Frame End • when a packet level double bit error (ECC2) was signaled, the whole transmission till Stop state is considered error. Csi2_↩ ErrorReportType↩ ::errMaskVC field reports the Virtual Channel with this error.

Interrupt ID / description	Error field	Error bit	Meaning
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::errMaskVC	CSI2_ERR_PACK_↩ DATA	Packet payload error. Csi2_ErrorReportType↩ ::errMaskVC field reports the Virtual Channel with this error.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::errMaskVC	CSI2_ERR_PACK_↩ CRC	CRC code of the message payload is different than the received CRC code. Csi2↩ _ErrorReportType↩ ::receivedCRC reports the received CRC and Csi2_ErrorReportType↩ ::expectedCRC reports the expected CRC for the received data. Csi2_ErrorReportType↩ ::errMaskVC field reports the Virtual Channel with this error.
Csi2_PhyIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::errMaskVC	CSI2_ERR_PACK_↩ ID	Unimplemented or unrecognized ID in the header. Csi2_ErrorReportType↩ ::errMaskVC field reports the Virtual Channel with this error.
Csi2_PcktIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::errMaskVC	CSI2_ERR_LINE_↩ LEN	The expected length of at least one line is different from the one received by interface. Csi2_ErrorReportType↩ ::lineLengthErr reports the line position and the received length. Csi2_ErrorReportType↩ ::errMaskVC field reports the Virtual Channel with this error.
Csi2_PcktIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::errMaskVC	CSI2_ERR_LINE_↩ CNT	The number of lines expected are different from the one received by interface. Csi2_↩ ErrorReportType::line↩ LengthErr reports the number of received lines in the frame. Csi2_↩ ErrorReportType::err↩ MaskVC field reports the Virtual Channel with this error.
Csi2_PcktIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_↩ SPURIOUS_PKT	Improper IRQ call, with no one reporting error bit set.
Csi2_PcktIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_FIFO	The internal FIFO of the controller overflows.

Interrupt ID / description	Error field	Error bit	Meaning
Csi2_PcktIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_BUF_↩ OVERFLOW	The internal buffer is full such that any more data reception will cause high speed data to be dropped on the receive path.
Csi2_PcktIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_AXI_↩ OVERFLOW	The internal buffer is full due a latency on the AXI write datapath such that any more data reception will cause high speed data to be dropped on the receive path.
Csi2_PcktIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_AXI_↩ RESPONSE	The write response channel of AXI observes an error response.
Csi2_PcktIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_HS_EXIT	The number of data bytes received in an HS packet is less than the expected.
Csi2_EvtIsrCallbackUnit	Csi2_ErrorReportType↩ ::errMaskU	CSI2_ERR_↩ SPURIOUS_EVT	Improper IRQ call, with no one reporting bit set.
Csi2_EvtIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::evtMaskVC	CSI2_EVT_FRAME_↩ START	A Frame Start indication received by interface. Csi2_ErrorReportType↩ ::evtMaskVC field reports the Virtual Channel with this event.
Csi2_EvtIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::evtMaskVC	CSI2_EVT_FRAME_↩ END	A Frame End indication received by interface. The frame counter for the Virtual Channel is increased before the callback. Csi2_Error↩ ReportType::evtMaskVC field reports the Virtual Channel with this event.
Csi2_EvtIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::evtMaskVC	CSI2_EVT_LINE_↩ END	An indication when the line is written into the system memory and the number of written lines is multiple of the value specified in rsdkCsi2VCParams_t↩ ::bufNumLinesTrigger.
Csi2_EvtIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::evtMaskVC	CSI2_EVT_SHORT_↩ PACKET	Generic short packet received by the module. Csi2_ErrorReportType↩ ::shortPackets reports the packet: the ID and the data. Csi2_Error↩ ReportType::evtMaskVC field reports the Virtual Channel with this event.

Interrupt ID / description	Error field	Error bit	Meaning
Csi2_EvtIsrCallbackUnit	Csi2_ErrorReport↩ Type::errMaskU and Csi2_ErrorReportType↩ ::evtMaskVC	CSI2_EVT_BIT_↩ NOT_TOGGLE	Bits not toggled on one or many channels during the frame reception. The event is a statistical one and reported at the end of each frame the bits are not toggled. The mask for bits not toggled is stored in Csi2_ErrorReport↩ Type::notToggledBits. For the moment no indication about channel are offered. Csi2_Error↩ ReportType::evtMaskVC field reports the Virtual Channel with this event.

There are some politics to approach for callback management :

- use only one callback routine for all three usages : **Csi2_SingleIsrCallback**
- different callback routine for each defined callback :
  - **Csi2_PhyIsrCallbackUnit** - for PHY level errors
  - **Csi2_PcktIsrCallbackUnit** - for packet level errors
  - **Csi2_EvtIsrCallbackUnit** - for events

3.3.5 Auxiliary data usage

It is possible to receive through the interface not only the radar data, but some auxiliary data too. For the possible auxiliary data modes/usage please check the Reference Manual, chapter **"ADC channel data processing for RAW data"** and mainly the table **"Incoming data rates and formats for RAW data"**.
If auxiliary data not used, it is recommended to set CSI2_AUXILIARY_DATA_USAGE to STD_OFF; If auxiliary data used, the easiest way to set the parameters if the plugin is not used for configuring :

- set the CSI2_VC_BUF_FIFTH_CH_ON bit in the pVCconfig::outputDataMode parameter
- copy the pVCconfig parameters to the pAuxConfig
- set the auxiliary type to use on pAuxConfig::streamDataType (use only CSI2_DATA_TYPE_R12_... types, like CSI2_DATA_TYPE_R12_A0_NO_DROP)
- set the correct output mode on pAuxConfig::outputDataMode, one of CSI2_VC_BUF_5TH_CH_M..., like CSI2_VC_BUF_5TH_CH_M0_NODROP
- set the correct pointer to data buffer, on pAuxConfig::pBufData If necessary, test the necessary line buffer length for the auxiliary data, using the RsdkCsi2GetBufferRealLineLen function, with the data on pAuxConfig. Use the same index for pAuxConfig array as used for pVCconfig array, which is the VC identifier of the data to be received.

3.3.6 Metadata usage

It is possible to receive on CSI2 interface along with radar data some metadata, at the same time with the radar data. The metadata "chirp" must be included into the same frame with the radar data. The data type for the metadata must be CSI2_DATA_TYPE_EMBD or CSI2_DATA_TYPE_USR0 ... CSI2_DATA_TYPE_USR7. The memory buffer for this flow shall be different from the radar data buffer. The length of the metadata chirp must be 16 bytes aligned.

3.4 CSI2 plugin usage

This section explain how to use the CSI2 provided plugins, for ElectroBit Tresos and for S32 Design Studio.

3.4.1 Tresos plugin

CSI2 plugin for Tresos is offering the possibility to setup the CSI2 driver at two levels :

- PreCompile (**PC**) - only **CDD_Csi2_PCCfg.h** and **CDD_Csi2_PCCbk.h** files are produced
- PostBuild (**PB**) - there are produced the two files specified above and the necessary **Csi2_PBCfg.c** file, which contains all required configuration parameters; at least one complete parameters structure must be defined, but is possible to define as many structures as necessary

3.4.1.1 Tresos plugin - PreCompile parameters

In the figure 2 is pictured the plugin interface for the PreCompile setup. All necessary **PC** parameters are there.

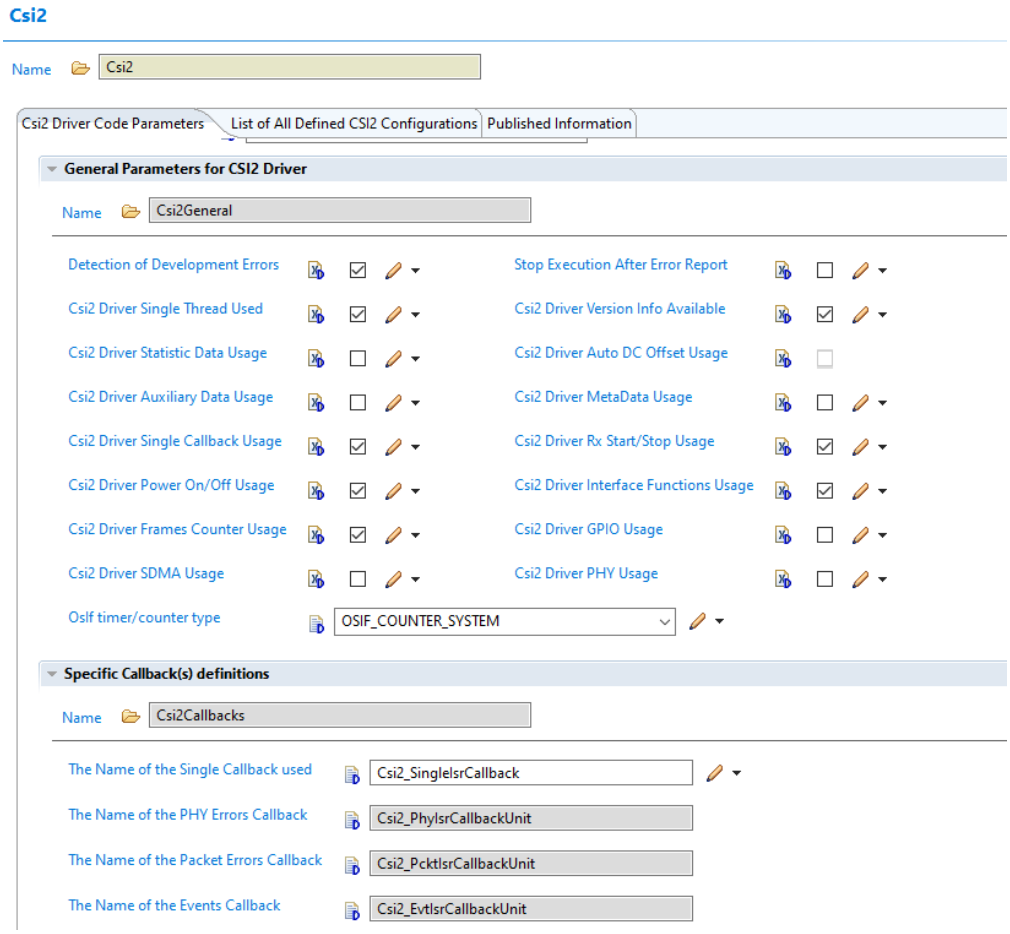


Figure 3.2 : CSI2 Tresos plugin - PreCompile page

The image display the default setup parameters, at the first usage of the plugin. All can be changed, but it could be some links between some parameters:

- **Detection of Development Errors**
  - **ON** = the input parameters will be checked before used
  - **OFF** = the input parameters will not be checked, the necessary code for this is removed before compile time
- **Stop Execution After Error Report**
  - **ON** = the execution of the code stops immediately after the error was reported to **Det**
  - **OFF** = the execution doesn't stop after error is reported to Det
- **Csi2 Driver Single Thread Used**
  - **ON** = the CSI2 driver is used in only one thread, the normal and the recommended usage
  - **OFF** = the CSI2 driver is used in more threads
- **Csi2 Driver Version Info Available**, the AUTOSAR specific parameter
  - **ON** = the driver reports the version info
  - **OFF** = the driver doesn't report the version info, the necessary code is removed at compile time
- **Csi2 Driver Statistic Data Usage**
  - **ON** = the statistic data is reported at the end of each chirp:
    - for this reason the buffer line length must be at least 80 bytes more than as the radar data requires
    - all data buffers must align to this requirement
    - it is possible to request automatic DC offset update on the received data
  - **OFF** = the statistical data is not reported
    - the buffer data lines can have only the length necessary to receive the radar data
    - the specific code is removed at compile time
- **Csi2 Driver Auto DC Offset Usage**, can be set only if **Csi2 Driver Statistic Data Usage** is **ON**
  - **ON** = automatic DC offset update for the requested ADC channels, is possible to update the DC offset automatically only for some of the channels
  - **OFF** = no automatic DC offset, this means the DC offset is not changed after setup
- **Csi2 Driver Auxiliary Data Usage**
  - **ON** = auxiliary data will be used
  - **OFF** = auxiliary data will not be used, the code will be removed at compile time
- **Csi2 Driver MetaData Usage**
  - **ON** = metadata will be used
  - **OFF** = metadata will not be used, the code will be removed at compile time
- **Csi2 Driver Single Callback Usage**
  - **ON** = a single callback used for all three interrupts to report the events/errors
  - **OFF** = three different callbacks used, one for each interrupt, see Callback usage
- **Csi2 Driver Rx Start/Stop Usage**
  - **ON** = The calls for Stop/Start reception will be used
  - **OFF** = The calls for Stop/Start reception will not be used, the code will be removed at compile time
- **Csi2 Driver Power On/Off Usage**,

  - **ON** = The calls for PowerOff/PowerOn the PHY interface will be used
  - **OFF** = The calls for PowerOff/PowerOn the PHY interface will not be used, the code will be removed at compile time

- **Csi2 Driver Interface Functions Usage**
  - **ON** = The calls for checking the PHY interface status will be used
  - **OFF** = The calls for checking the PHY interface status will not be used, the code will be removed at compile time

- **Csi2 Driver Frames Counter Usage**
  - **ON** = the driver will maintain the counters for the number of frames received; it is counted the number of Frame End packets received for each channel, the counter is reset at each Setup
  - **OFF** = the driver will not maintain the counters for the number of frames received

- **Csi2 Driver GPIO Usage**
  - **ON** = the hardware feature of triggering the GPIO signal will be used
  - **OFF** = the hardware feature of triggering the GPIO signal will not be used

- **Csi2 Driver SDMA Usage**
  - **ON** = the hardware feature of triggering the DMA transfer will be used
  - **OFF** = the hardware feature of triggering the DMA transfer will not be used

- **Csi2 Driver PHY Usage**
  - **ON** = the external MIPICSI2-PHY path will be used for receiving data
  - **ON** = the external MIPICSI2-PHY path will not be used, only the internal path will be used

- **OsIf timer/counter type**
  - **OSIF_COUNTER_DUMMY** = the time counting will be done using driver internal loops
  - **OSIF_COUNTER_SYSTEM** = the system counter will be used, it must be implemented by the application
  - **OSIF_COUNTER_CUSTOM** = a custom counter will be used, it must be implemented by the application

- **The Name of the Single Callback used**
  - **Csi2_SingleIsrCallback** is the default name of the call back, but it can be changed to any valid C name

- **The Name of the PHY Errors Callback**
  - **Csi2_PhyIsrCallbackUnit** is the default name of the call back, but it can be changed to any valid C name

- **The Name of the Packet Errors Callback**
  - **Csi2_PcktIsrCallbackUnit** is the default name of the call back, but it can be changed to any valid C name

- **The Name of the Events Callback**
  - **Csi2_EvtIsrCallbackUnit** is the default name of the call back, but it can be changed to any valid C name

3.4.1.2 Tresos plugin - PostBuild parameters

To use the CSI2/PPE interface, is necessary to define at least one setup structure, to be used in the Csi2_Setup call. But is possible to use as many as necessary different structures if the interface must work in many contexts to receive data. Each defined line in the list on the figure 3 represent a setup structure.

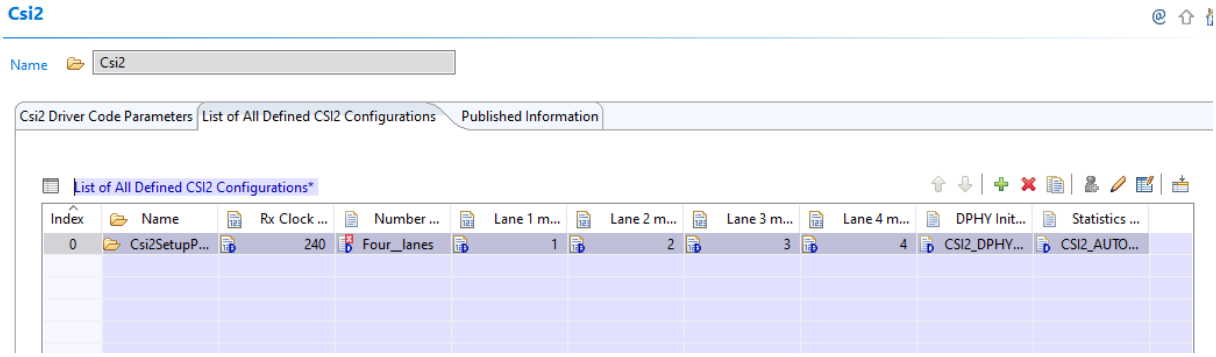


Figure 3.3 : CSI2 Tresos plugin - PostBuild setup parameters list

To define the PostBuild parameters, means to define all necessary functional parameters for the interface to receive correctly the data. For this, is necessary to define two levels of parameters :

- Unit level parameters, to be used at the unit level
- Virtual Channel level parameters, parameters to be used to receive a specific Virtual Channel data

3.4.1.3 Tresos plugin - PostBuild parameters - Unit level

For each parameters line in the figure 3, it is necessary to define the general unit parameters, pictured in figure 4.

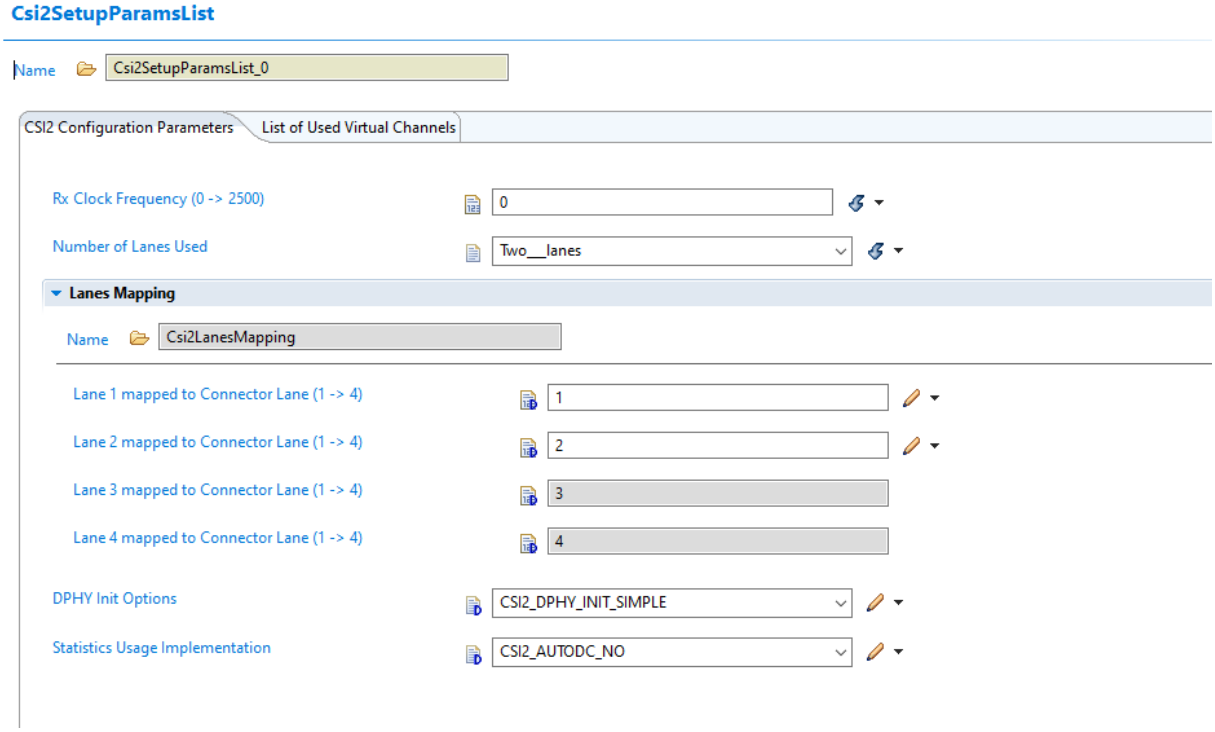


Figure 3.4 : CSI2 Tresos plugin - PostBuild Unit level parameters

The meanings/usage of the parameters :

- **Rx Clock Frequency** = the frequency of the MIPI-CSI2 clock of the transmitter, the unit used is MHz, 2500 in the box represents a 2.5GHz clock. If using an external Radar Front End, 0 must not be used, but in the specific case of SAF85XX 0 means the ADC data must be received over the internal path.
- **Number of Lanes Used** = the number of lines used for receiving data from an external Front-End, possible values :
  - **One___lane**
  - **Two___lanes**
  - **Three_lanes**
  - **Four__lanes**
- **Lanes Mapping** = the way the external input is mapped
- **DPHY Init Options** = the initialization type of CSI2-DHY :
  - **CSI2_DPHY_INIT_SIMPLE** = simple initialization and calibration, all PHY internal values reset to 0
  - **CSI2_DPHY_INIT_SHORT_CALIB** = the quickest initialization and calibration, all PHY internal values updated to the latest used before
  - **CSI2_DPHY_INIT_W_STOP_STATE** = simple initialization and calibration followed by a wait for STOP STATE on the used lanes
  - **CSI2_DPHY_INIT_SHORT_AND_STOP** = the quickest initialization and calibration followed by a wait for STOP STATE on the used lanes
    The time spent to initialize depends of the platform/hardware/initialization type and can be up to 1.2 ms
- **Statistics Usage Implementation** = the necessary selection for the statistical data usage/calculation, please remember that the computation is done in the interrupt handler :
  - **CSI2_AUTODC_NO** = no statistical computation, no automated DC update
  - **CSI2_AUTODC_EVERY_LINE** = the statistical computation will be done using all the statistical data received, for all chirps; this require a long processor time to process data
  - **CSI2_AUTODC_AT_FE** = the computation will be done only at the end of the frame, when the Frame End event is received, using all the chirps/lines in the memory buffer
  - **CSI2_AUTODC_LAST_LINE** = the computation will be done only at the end of the frame, when the Frame End event is received, using only the last chirp/line in the memory buffer

Note

- If an error is reported related to the **Number of Lanes Used**, please check the **Resource** module for the current selected platform and use the appropriate S32R41 phantom.

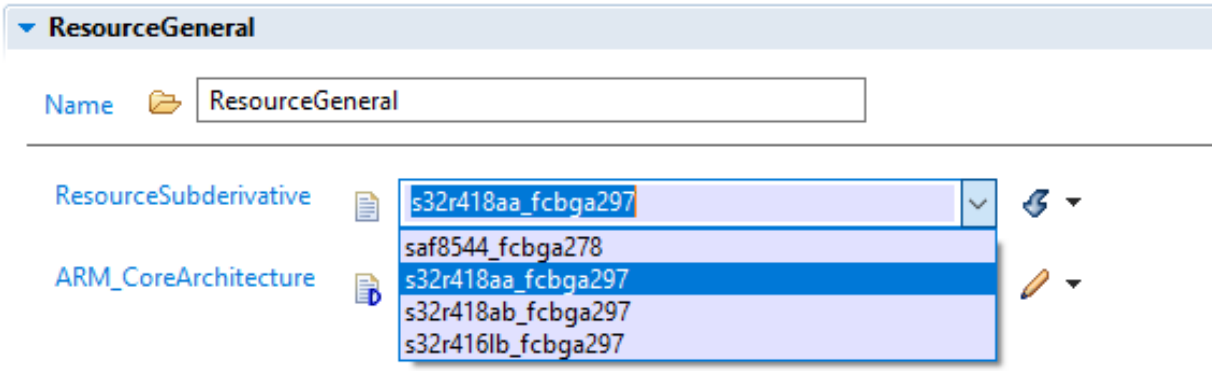


Figure 3.5 : Resource plugin - Correct S32R41 selection

- The generated configuration is not connected to a physical unit, is the customer application where this connection is done, when calling Csi2_Setup using the correct unit and setup parameters.

3.4.1.4 Tresos plugin - PostBuild parameters - Virtual Channel level

For each parameters line in the figure 3, it is necessary to define the Virtual Channel parameters. For each general parameters structure, it is necessary to define from one to four Virtual Channel setup. It is possible to define/use only distinct Virtual Channels in a single setup, numbered from 0 to 3. If this requirement is not respected, the plugin will report an error. A possible list of Virtual Channels setup is pictured in figure 5.

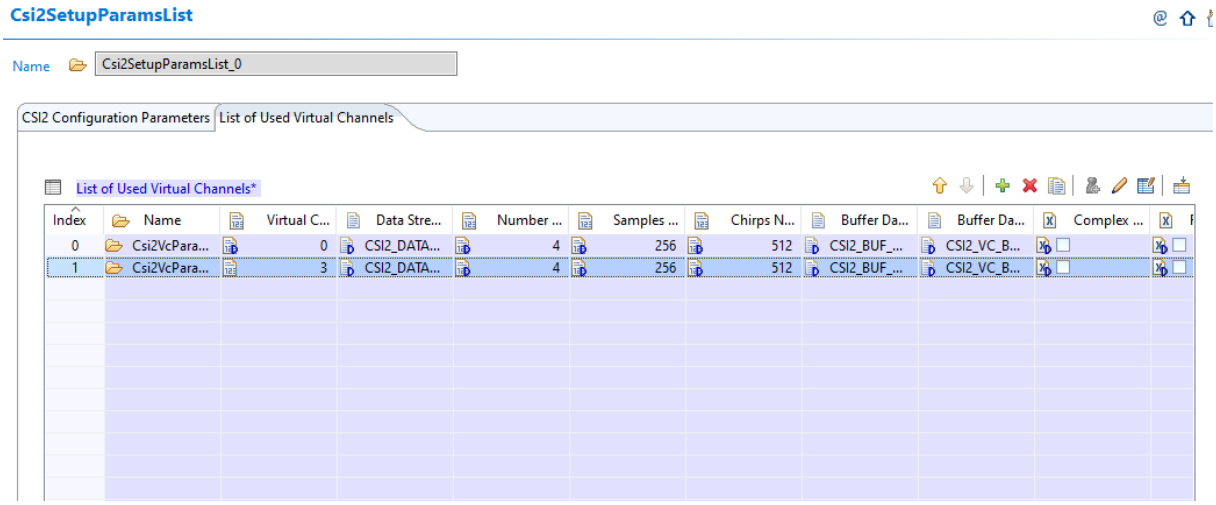


Figure 3.6 : CSI2 Tresos plugin - PostBuild defined Virtual Channel list

For each Virtual Channel used to receive data must be provided the necessary setup data, as above:

General parameters for Virtual Channel

The general **VC** parameters must be provided even all the setup options for PreCompile were set to OFF. The general parameters are pictured below.

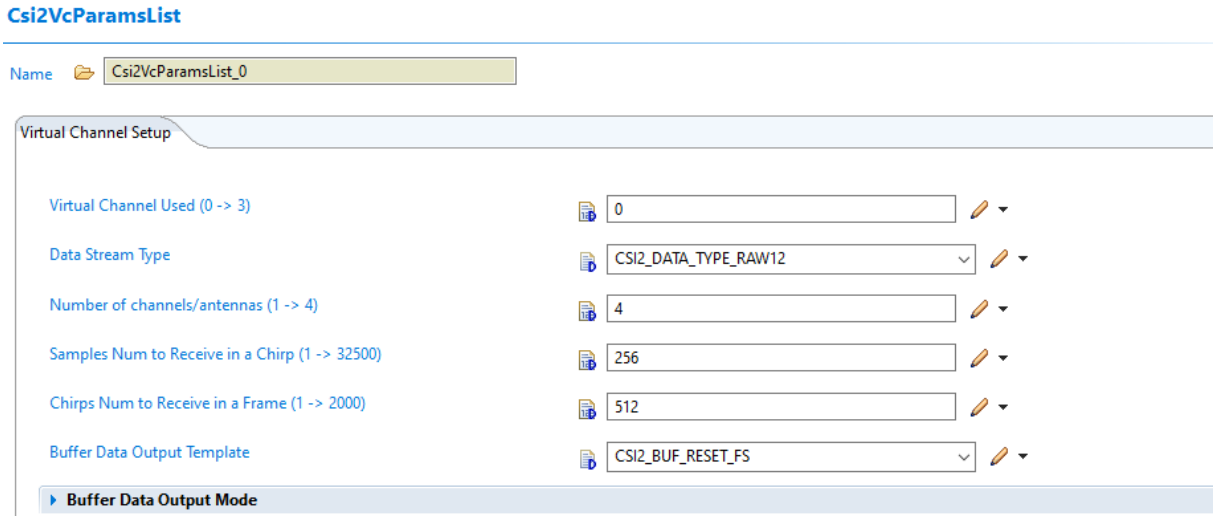


Figure 3.7 : CSI2 Tresos plugin - PostBuild Virtual Channel general parameters

- **Virtual Channel Used** = defines the ID of the virtual channel to which this setup is referring. It could be only in range [0...3] and a different value must be used for each **VC** parameters set.

- **Data Stream Type** = the stream type to be received using this **VC**. The possible types are aligned with the **RM** defined types and **MIPI-CSI2** documentation; it could be one of :
  - **CSI2_DATA_TYPE_EMBD** = embedded data type (0x12)
  - **CSI2_DATA_TYPE_YUV422_8** = YUV422 video data type, 8 bits version (0x1e)
  - **CSI2_DATA_TYPE_YUV422_10** = YUV422 video data type, 10 bits version (0x1f)
  - **CSI2_DATA_TYPE_RGB565** = RGB565 video data type (0x22)
  - **CSI2_DATA_TYPE_RGB888** = RGB888 video data type (0x24)
  - **CSI2_DATA_TYPE_RAW8** = RAW8 data type (0x2a)
  - **CSI2_DATA_TYPE_RAW10** = RAW10 data type (0x2b)
  - **CSI2_DATA_TYPE_RAW12** = RAW12 data type (0x2c)
  - **CSI2_DATA_TYPE_RAW14** = RAW14 data type (0x2d)
  - **CSI2_DATA_TYPE_RAW16** = RAW16 data type (0x2e)
  - **CSI2_DATA_TYPE_USR0** = user data type (0x30)
  - **CSI2_DATA_TYPE_USR1** = user data type (0x31)
  - **CSI2_DATA_TYPE_USR2** = user data type (0x32)
  - **CSI2_DATA_TYPE_USR3** = user data type (0x33)
  - **CSI2_DATA_TYPE_USR4** = user data type (0x34)
  - **CSI2_DATA_TYPE_USR5** = user data type (0x35)
  - **CSI2_DATA_TYPE_16_FROM_8** = = user data type (0x36), used to convert a 8 bits received data to 16 bits data LSB format
  - **CSI2_DATA_TYPE_USR7** = user data type (0x37)
    For **RAW** data type, at processing time, it is necessary to check the format conversion done in the hardware interface, data alignment and sign extension, please check **RM**, chapter **ADC channel data processing for RAW data**
- **Number of channels/antennas** = the number of antennas/channels/ADC used to get data; the interface is expecting to receive data samples in this sequence for 4 antennas and real data : ADC0_0, ADC1_0, ADC2_0, ADC3_0, ADC0_1, ADC1_1, ADC2_1, ADC3_1, ADC0_2, ADC1_2, ADC2_2, ADC3_2, ...
- **Samples Num to Receive in a Chirp** = the number of samples to receive in a chirp for a single channel/antenna
- **Chirps Num to Receive in a Frame** = the number of the chirps in a frame for this **VC**
- **Buffer Data Output Template** = for some platforms is possible to set the data output at chirp level:
  - **CSI2_BUF_RESET_NO** = after Frame End the buffer line pointer is continuing with the first buffer line after the last written; i.e., if the buffer has N lines and the las one written was line M, the next line written will be M+1. This means, if (M == N), the next line will be the first buffer line, else it will be a line in the buffer
  - **CSI2_BUF_RESET_FS** = after Frame End the buffer line pointer is reset always to the first buffer line

Buffer Data setup for Virtual Channel

For each **VC** data to be received is necessary to specify the buffer management :

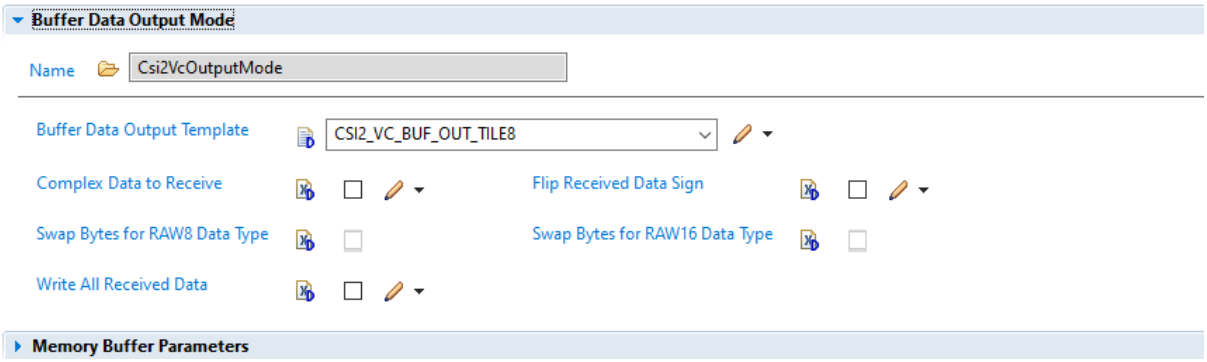


Figure 3.8 : CSI2 Tresos plugin - PostBuild Virtual Channel data buffer setup

- **Buffer Data Output Template** = specify the template to be used when the data is written into the memory :
  - **CSI2_VC_BUF_OUT_ILEAVED** = the data is written as is receives, see above, **Number of channels/antennas**
  - **CSI2_VC_BUF_OUT_TILE8** = the data will be aggregated to have 8 samples from the same ADC together, like this : ADC0_0, ADC0_1, ... , ADC0_7, ADC1_0, ADC1_1, ... , ADC1_7, ADC2_0, ADC2_1, ..., ADC2_7, ADC3_0, ADC3_1, ..., ADC3_7, ADC0_8, ADC0_9, ..., ADC0_15, ...
  - **CSI2_VC_BUF_OUT_TILE16** = the data will be aggregated to have 16 samples from the same ADC together, like this : ADC0_0, ADC0_1, ... , ADC0_15, ADC1_0, ADC1_1, ... , ADC1_15, ADC2_0, ADC2_1, ..., ADC2_15, ADC3_0, ADC3_1, ..., ADC3_15, ADC0_16, ADC0_17, ..., ADC0_31, ...
- **Complex Data to Receive** = specify the type of data to be received
  - **ON** = complex data to receive, this will double the number of above declared antennas/channels. The complex data will be mixed with the real data, it means the received data flow will be : ADC0_real, ADC0_complex, ADC1_real, ADC1_complex, ... which will affect the order in the data buffer specified in the point above
  - **OFF** = only real data received
- **Flip Received Data Sign**
  - **ON** = the adjusted RAW data will have the sign flipped
  - **OFF** = the RAW data sign will not be flipped
- **Swap Bytes for RAW8 Data Type** = how to write the received RAW8 data into the memory
  - **ON** = for RAW8 data only, the samples will be put in the buffer in a reversed order, to allow reversing the 16 bits values if a 16 bits value was sent as RAW8, from MSB to LSB or from LSB to MSB
  - **OFF** = RAW8 data put in the memory in the normal order
- **Swap Bytes for RAW16 Data Type** = same as above, but referring to RAW16 data stream
- **Write All Received Data** = how to manage the data received before **Frame Start**
  - **ON** = the data will be written into the memory as received, even Frame Start was not received and possible some of the chirps too; this can affect the first line written into the buffer for the next frame
  - **OFF** = the data will be written into the memory only if a Frame Start is received

Buffer Data parameters for Virtual Channel

For each **VC** data to be received is necessary to specify the buffer parameters :

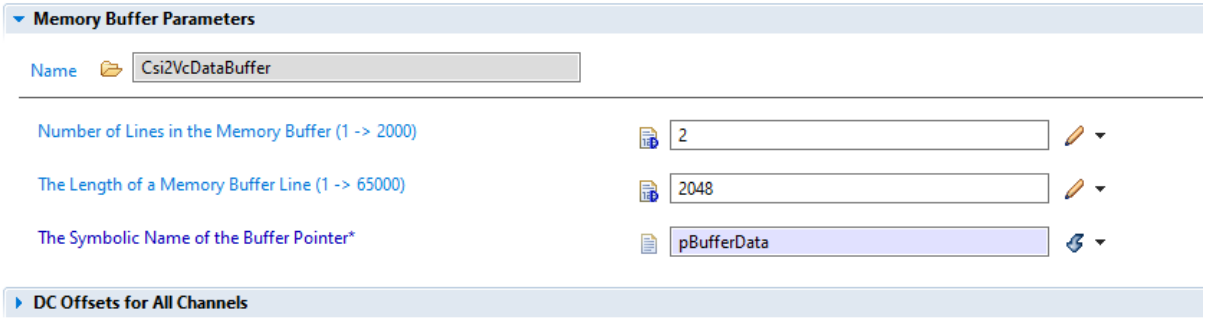


Figure 3.9 : CSI2 Tresos plugin - PostBuild Virtual Channel data buffer parameters

- **Number of Lines in the Memory Buffer** = the number of lines in the buffer; if the specified number of lines is N (the lines are numbered from 1 to N), after writing the line N, line 1 will be written; this means, only if N is the numbers of chirps to be received the data will not be overwritten for a frame.
- **The Length of a Memory Buffer Line** = the bytes reserved for a chirp; be careful :
  - the length of data is generally : $<$ number_of_channels $>$ $*$ $<$ number_of_samples_per_channel $>$ $*$ $<$ bytes_per_sample_in_memory $>$
  - if statistical data is required to be added at the end of the chirp, 80 bytes must be added to the length above
- **The Symbolic Name of the Buffer Pointer** = the buffer name to be used in the application; the buffer will be defined in Csi2_PBCfg.c file.

DC offset parameters for Virtual Channel

For each received channel is possible to set a DC offset. The DC offset can be a fixed value, set at the initialization or is possible to be updated after each frame if automatic DC offset is set, see Tresos plugin - PreCompile parameters. The value specified as DC offset is subtract from the received values, with an exception : the value 32767, which is the maximum positive value to be set for a 16 bits signed value. This value require for that channel to adjust the DC offset after each frame (automatic DC offset). This is done per channel, so only the channels with this DC offset will be updated automatically.

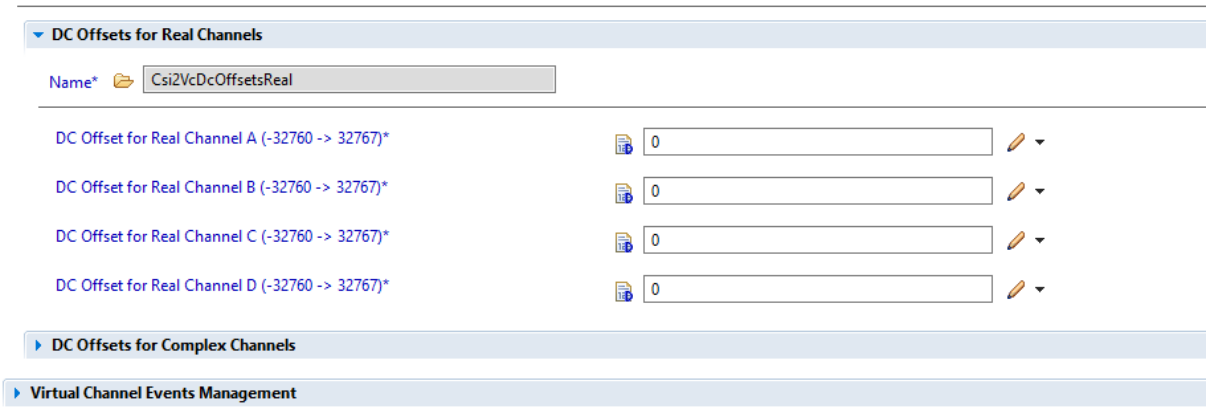


Figure 3.10 : CSI2 Tresos plugin - PostBuild Virtual Channel channels DC offsets

If the complex data is received, DC management is similar.

Events for Virtual Channel

For each defined Virtual Channel is possible to request some events signals. The field Csi2_ErrorReportType↩
::evtMaskVC is signaling the detected signals.
There is a big difference between errors and signals :

- any error is detected, is reported to the application using Csi2_ErrorReportType structure and the appropriate callback
- an event is reported to the application using the same Csi2_ErrorReportType, but only if required at setup/initialization time; if the event is not required, the application will not receive a signal if the event is detected

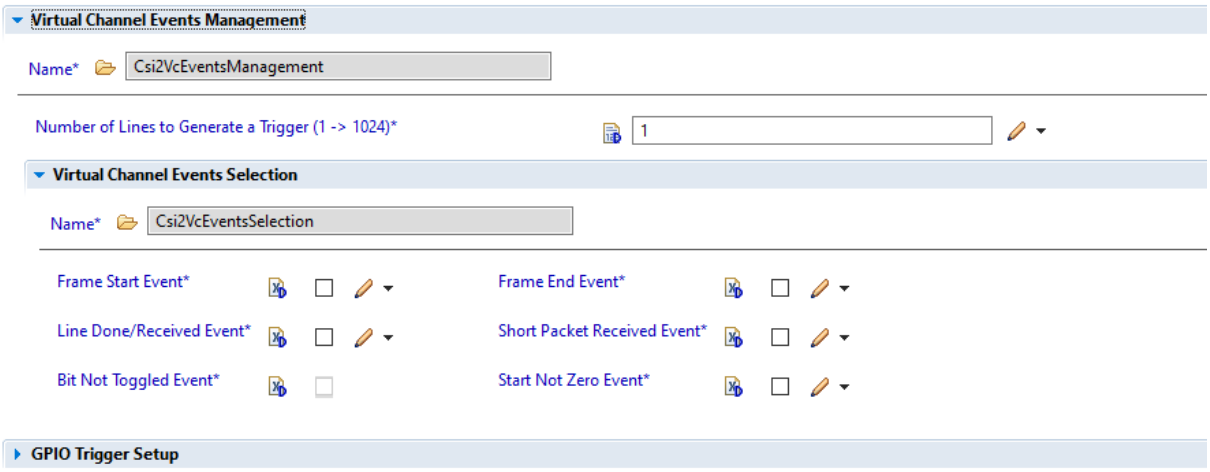


Figure 3.11 : CSI2 Tresos plugin - PostBuild Virtual Channel events

Is possible to request these events to be reported :

- **Frame Start Event** = this event is reported when a **Frame Start** packet is received
- **Frame End Event** = this event is reported when a **Frame End** packet is received
- **Line Done/Received Event** = this event is reported when is received the requested amount of chirps : a multiple of the parameter :
  - **Number of Lines to Generate a Trigger** = this is an important value and has two different meanings :
    - the number of chirps received at which the driver reports a **Line Done Event**
    - the number of chirps received at which SPT will receive a CSI2 line received event
      So, set the appropriate value to this parameter.
- **Short Packet Received Event** = event signaled when an unexpected short packet is received. The expected short packets are : Frame Start, Frame End.
- **Bit Not Toggled Event** = this event signal that a specific bit, for a specific channel, is not changed over the entire chirp. This event can be reported only if Statistical data is reported.
- **Start Not Zero Event** = this event is reported if at Frame End the next line to receive data is not the first line in the buffer

GPIO triggers for Virtual Channel

The Packet Processor interface is able to trigger an output GPIO signal, using the dedicated PE[6] output pad. Is possible to trigger an output signal for many CSI2/PPE internal events :

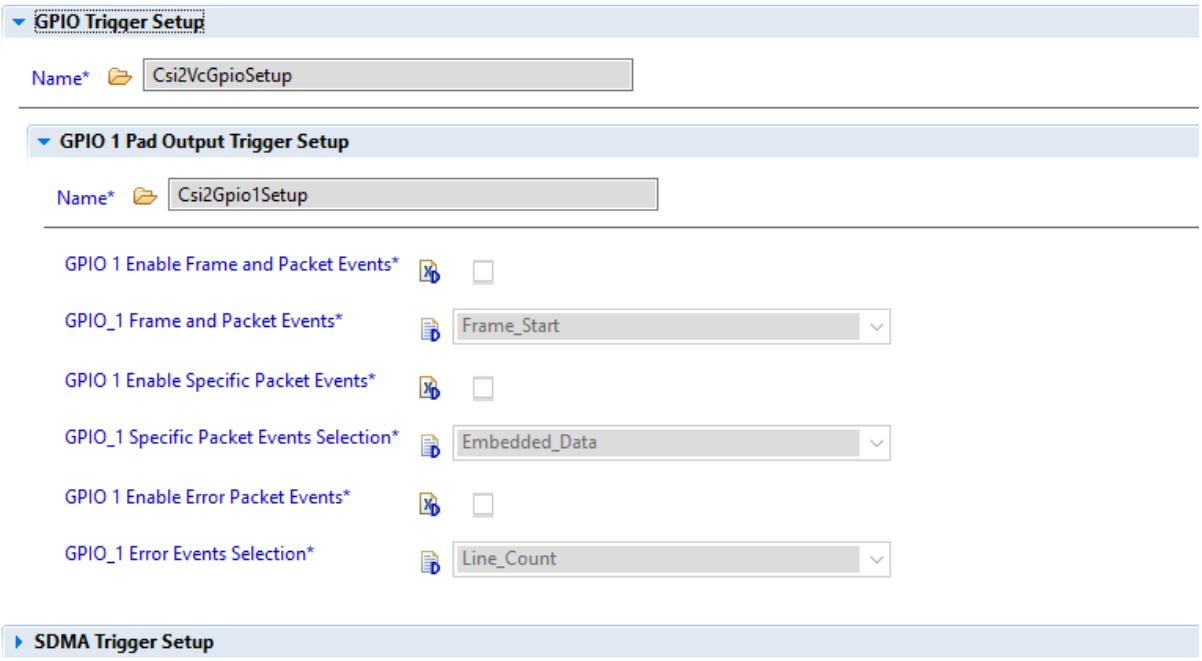


Figure 3.12 : CSI2 Tresos plugin - PostBuild Virtual Channel GPIO trigger setup

- **GPIO 1 Enable Frame and Packet Events** = GPIO triggers at some specific moments, related to packet protocol level :
  - **OFF** = no triggers generated for these events
  - **ON** = triggers generated for the selected event :
    - **Frame_Start**
    - **Frame_End**
    - **Packet_Start**
    - **Packet_End**
- **GPIO 1 Enable Specific Packet Events** = triggers generated when the specific data packet is received
  - **OFF** = no triggers generated for these events
  - **ON** = triggers generated for the selected event :
    - **Embedded_Data**
    - **User_Defined_Data**
    - **RAW_Data**
    - **RGB_Data**
- **GPIO 1 Enable Error Packet Events** = triggers generated when the specific error is detected
  - **Line_Count**
  - **Line_Length**
  - **CRC_or_ECC2bits**
  - **No_Synchronization_on_DPHY**

SDMA triggers for Virtual Channel

The Packet Processor interface is able to trigger an SDMA signal and start a SDMA data transfer if requested :

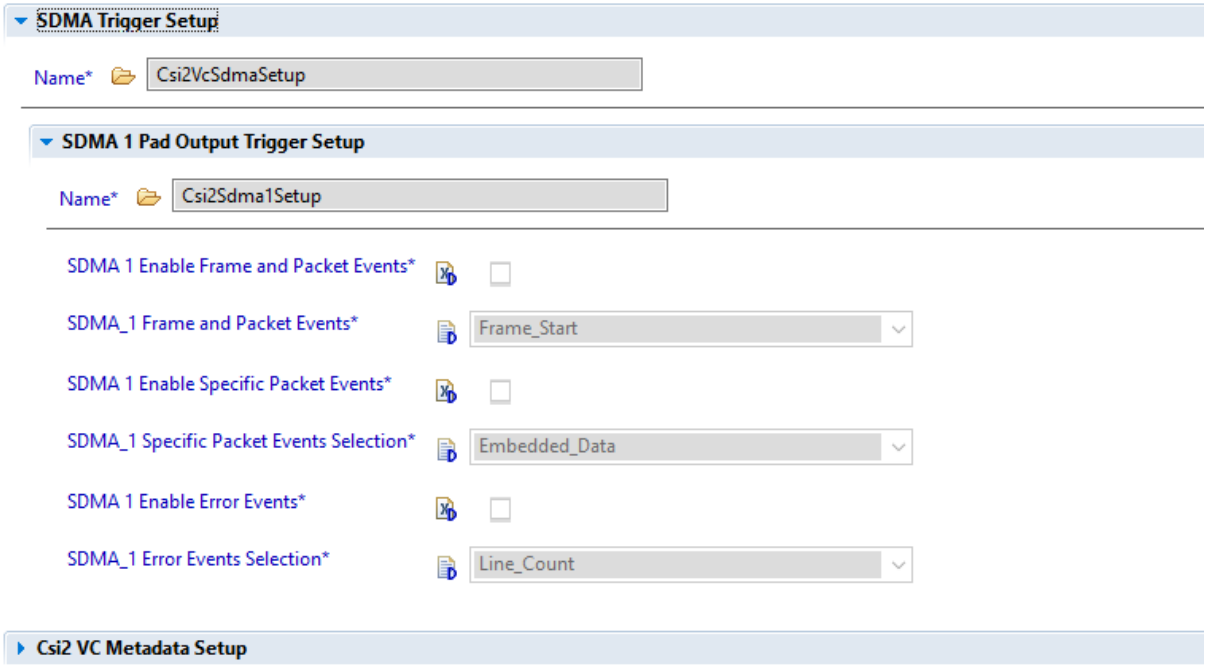


Figure 3.13 : CSI2 Tresos plugin - PostBuild Virtual Channel SDMA trigger setup

- **SDMA_1 Frame and Packet Events** = SDMA triggers at some specific moments, related to packet protocol level :
  - **OFF** = no triggers generated for these events
  - **ON** = triggers generated for the selected event :
    - **Frame_Start**
    - **Frame_End**
    - **Packet_Start**
    - **Packet_End**
- **SDMA_1 Specific Packet Events Selection** = triggers generated when the specific data packet is received
  - **OFF** = no triggers generated for these events
  - **ON** = triggers generated for the selected event :
    - **Embedded_Data**
    - **User_Defined_Data**
    - **RAW_Data**
    - **RGB_Data**
- **SDMA 1 Enable Error Events** = triggers generated when the specific error is detected
  - **Line_Count**
  - **Line_Length**
  - **CRC_or_ECC2bits**
  - **No_Synchronization_on_DPHY**

Metadata for Virtual Channel

The Packet Processor interface is able to receive Metadata. The Metadata is number of chirps received usually at the end of the frame. The data type is different of the normal radar data and can be selected from a limited range of data types. Generally, this kind of data is restrictive, but allow to receive some limited specific data at the end of the frame, like the Front-End general status.
This feature is not active if **Csi2 Driver MetaData Usage** in Tresos plugin - PreCompile parameters is not set to **ON**.

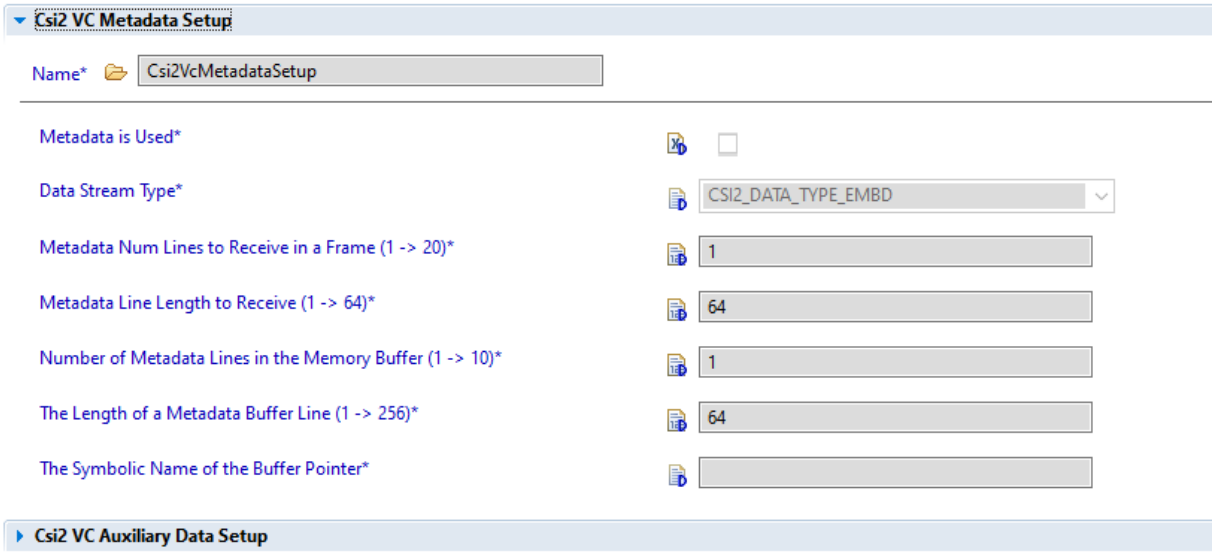


Figure 3.14 : CSI2 Tresos plugin - PostBuild Virtual Channel Metadata parameters

- **Metadata is Used**
  - **OFF** = Metadata is not used on this Virtual Channel received data
  - **ON** = Metadata is used for this Virtual Channel received data
- **Data Stream Type** = the data type to be received as Metadata, it must be different than the normal data received for this Virtual Channel; the list is similar to the one for **Data Stream Type**.
- **Metadata Num Lines to Receive in a Frame** = the number of lines to be received as Metadata
- **Metadata Line Length to Receive** = the bytes amount of Metadata to receive
- **Number of Metadata Lines in the Memory Buffer** = the number of lines in buffer to receive Metadata; the buffer is a circular buffer, as for normal data
- **The Length of a Metadata Buffer Line** = the length of line buffer data; the buffer line length must respect the same rules as for normal data, for alignment and supplementary 80 bytes if statistical data activated
- **The Symbolic Name of the Buffer Pointer** = the Metadata buffer name to be used in application

Auxiliary data for Virtual Channel

The Packet Processor interface is able to receive Auxiliary data. The Auxiliary data is a part of the received data : the received data is split in two parts - the normal radar data and Auxiliary data. Usually the ratio is 50-50, but please check the Reference Manual for the possible schemas. The main point : the data is written into the memory in distinct buffers. This feature is not active if **Csi2 Driver Auxiliary Data Usage** in Tresos plugin - PreCompile parameters is not set to **ON**.

Figure 3.15 : CSI2 Tresos plugin - PostBuild Virtual Channel Auxiliary data parameters

- **Auxiliary Data is Used**
  - **OFF** = Auxiliary data is not used on this Virtual Channel received data
  - **ON** = Auxiliary data is used for this Virtual Channel received data
- **Data Stream Drop Schema** = the schema to split the received data and to write it into the Auxiliary data buffer :
  - **CSI2_DATA_TYPE_AUX_0_NO_DROP** = the first schema, all the Auxiliary data received is written into the memory
  - **CSI2_DATA_TYPE_AUX_0_DR_1OF2** = the second schema, only a half of the Auxiliary data received is written into the memory
  - **CSI2_DATA_TYPE_AUX_0_DR_3OF4** = the third schema, only a quarter of the Auxiliary data received is written into the memory
  - **CSI2_DATA_TYPE_AUX_1_NO_DROP** = the fourth schema, the data is not split 50-50, but like (4 data samples + 2 Auxiliary data samples), and all Auxiliary data is written into the memory
- **Number of Auxiliary Data Lines in the Memory Buffer** = the number of lines in buffer to receive Auxiliary data; the buffer is a circular buffer, as for normal data
- **The Length of an Auxiliary Data Buffer Line** = the length of line buffer data; the buffer line length must respect the same rules as for normal data, for alignment and supplementary 80 bytes if statistical data activated
- **The Symbolic Name of the Buffer Pointer** = the Auxiliary data buffer name to be used in application

3.4.2 S32DS driver configuration

The driver configuration for **S32DS** is offering the same functionality as the Tresos plugin, the only difference is the interface used, adapted to the eclipse usage. CSI2 configuration interface is offering the possibility to setup the CSI2 driver at two levels :

- PreCompile (**PC**) - **CDD_Csi2_PCCfg.h** and **CDD_Csi2_PCCbk.h** files are produced
- PostBuild (**PB**) - **Csi2_PBCfg.c** file is produced, which contains all required configuration parameters; at least one complete parameters structure must be defined, but is possible to define as many structures as necessary for the application.

Note

To enable/add the delivered S32DS RadarSDK plugins/peripherals, please do :

- use python to run the delivered file : < rsdk_root_folder >/autosar_plugins/S32R41/S32DS/Radar↩ SDK_install_to_DS.py
- use as parameters : absolute path to the S32DS root folder and absolute/relative path to the RadarSDK root folder
- restart S32DS if necessary

3.4.2.1 CSI2 configuration - adding the configuration to the Peripherals

As the CSI2 module is not defined as default for the projects, it is necessary to add it to the configuration interface. So, please follow the three steps described in figure 15 :

- click on ADD button for Drivers
- select CSI2 module
- click OK button The CSI2 module must be shown on the Drivers interface and now is possible to configure it.

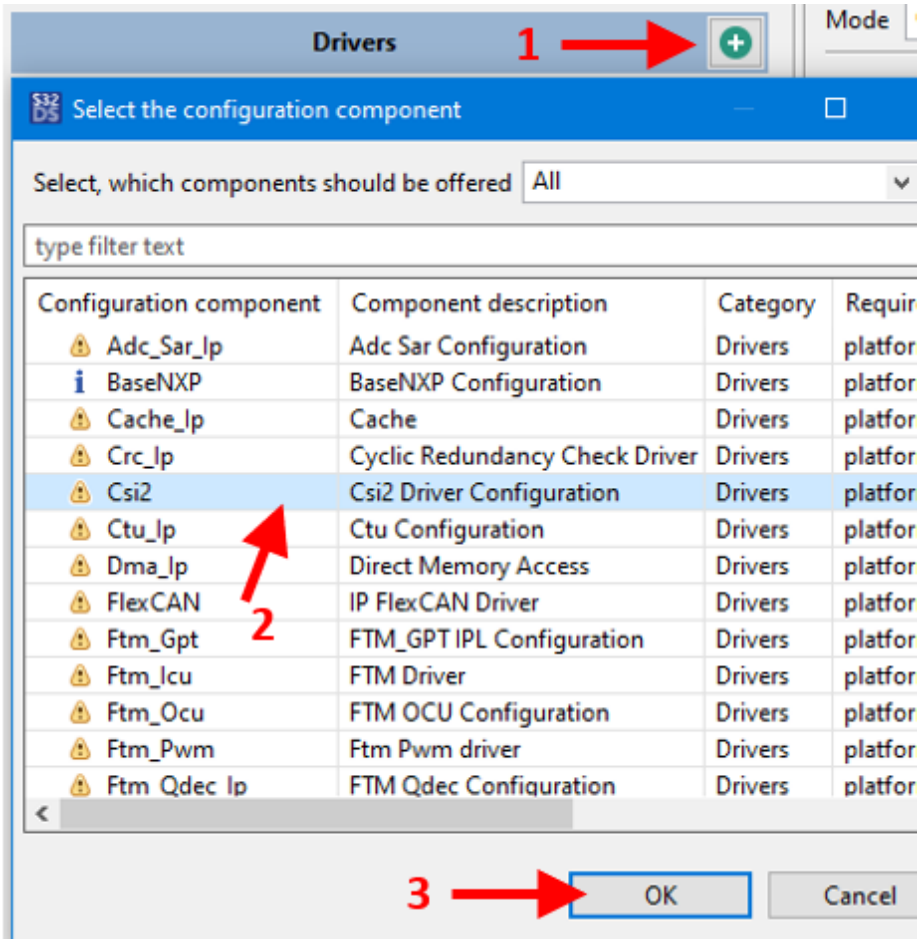


Figure 3.16 : CSI2 S32DS configuration - adding the module to the Peripherals

3.4.2.2 CSI2 configuration - PreCompile parameters

In the figure 16 is pictured the plugin interface for the PreCompile setup. All necessary **PC** parameters are there.

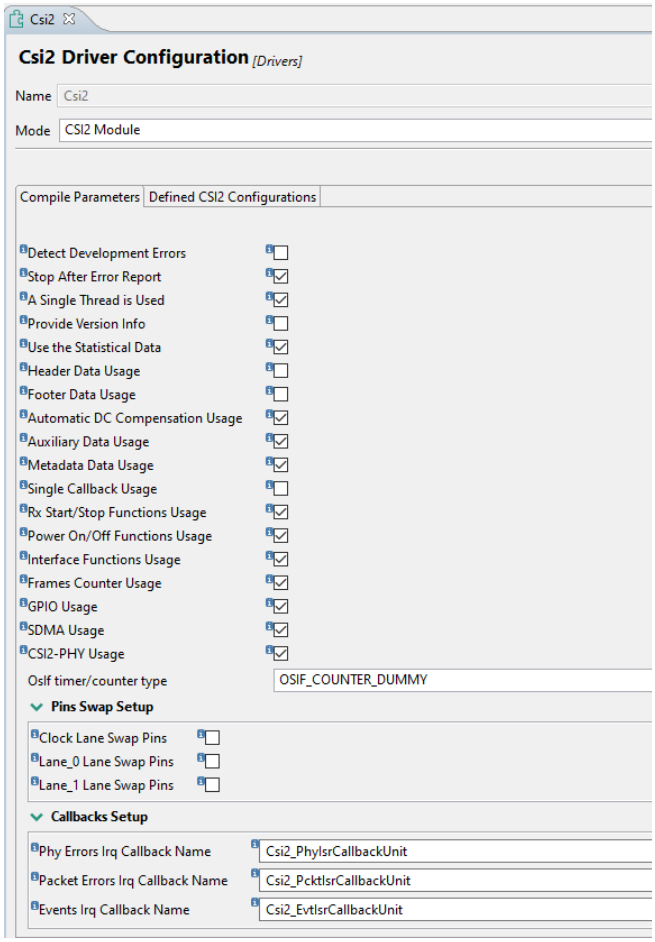


Figure 3.17 : CSI2 configuration - PreCompile page

The image display the default setup parameters, at the first usage of the plugin. All can be changed, but it could be some links between some parameters:

- **Detection of Development Errors**
  - **ON** = the input parameters will be checked before used
  - **OFF** = the input parameters will not be checked, the necessary code for this is removed before compile time
- **Stop Execution After Error Report**
  - **ON** = the execution of the code stops immediately after the error was reported to **Det**
  - **OFF** = the execution doesn't stop after error is reported to Det
- **Csi2 Driver Single Thread Used**
  - **ON** = the CSI2 driver is used in only one thread, the normal and the recommended usage
  - **OFF** = the CSI2 driver is used in more threads
- **Csi2 Driver Version Info Available**, the AUTOSAR specific parameter
  - **ON** = the driver reports the version info
  - **OFF** = the driver doesn't report the version info, the necessary code is removed at compile time
- **Csi2 Driver Statistic Data Usage**
  - **ON** = the statistic data is reported at the end of each chirp:

    - for this reason the buffer line length must be at least 80 bytes more than as the radar data requires
    - all data buffers must align to this requirement
    - it is possible to request automatic DC offset update on the received data
  - **OFF** = the statistical data is not reported
    - the buffer data lines can have only the length necessary to receive the radar data
    - the specific code is removed at compile time

- **Csi2 Driver Auto DC Offset Usage**, can be set only if **Csi2 Driver Statistic Data Usage** is **ON**
  - **ON** = automatic DC offset update for the requested ADC channels, is possible to update the DC offset automatically only for some of the channels
  - **OFF** = no automatic DC offset, this means the DC offset is not changed after setup
- **Csi2 Driver Auxiliary Data Usage**
  - **ON** = auxiliary data will be used
  - **OFF** = auxiliary data will not be used, the code will be removed at compile time
- **Csi2 Driver MetaData Usage**
  - **ON** = metadata will be used
  - **OFF** = metadata will not be used, the code will be removed at compile time
- **Csi2 Driver Single Callback Usage**
  - **ON** = a single callback used for all three interrupts to report the events/errors
  - **OFF** = three different callbacks used, one for each interrupt, see Callback usage
- **Csi2 Driver Rx Start/Stop Usage**
  - **ON** = The calls for Stop/Start reception will be used
  - **OFF** = The calls for Stop/Start reception will not be used, the code will be removed at compile time
- **Csi2 Driver Power On/Off Usage**,
  - **ON** = The calls for PowerOff/PowerOn the PHY interface will be used
  - **OFF** = The calls for PowerOff/PowerOn the PHY interface will not be used, the code will be removed at compile time
- **Csi2 Driver Interface Functions Usage**
  - **ON** = The calls for checking the PHY interface status will be used
  - **OFF** = The calls for checking the PHY interface status will not be used, the code will be removed at compile time
- **Csi2 Driver Frames Counter Usage**
  - **ON** = the driver will maintain the counters for the number of frames received; it is counted the number of Frame End packets received for each channel, the counter is reset at each Setup
  - **OFF** = the driver will not maintain the counters for the number of frames received
- **Csi2 Driver GPIO Usage**
  - **ON** = the hardware feature of triggering the GPIO signal will be used
  - **OFF** = the hardware feature of triggering the GPIO signal will not be used
- **Csi2 Driver SDMA Usage**
  - **ON** = the hardware feature of triggering the DMA transfer will be used
  - **OFF** = the hardware feature of triggering the DMA transfer will not be used
- **OsIf timer/counter type**

  - **OSIF_COUNTER_DUMMY** = the time counting will be done using driver internal loops
  - **OSIF_COUNTER_SYSTEM** = the system counter will be used, it must be implemented by the application
  - **OSIF_COUNTER_CUSTOM** = a custom counter will be used, it must be implemented by the application

- **The Name of the Callback(s) used**
  - **Csi2_SingleIsrCallback** is the default name of the call back, but it can be changed to any valid C name, present only if **Single Callback** selected
  - **Csi2PhyErrorCallbackName** is the default name of the **Phy Error** call back, but it can be changed to any valid C name, present only if **Single Callback** is **not** selected
  - **Csi2PcktErrorCallbackName** is the default name of the **Packet Error** call back, but it can be changed to any valid C name, present only if **Single Callback** is **not** selected
  - **Csi2EvtErrorCallbackName** is the default name of the **Events** call back, but it can be changed to any valid C name, present only if **Single Callback** is **not** selected

3.4.2.3 CSI2 configuration - PostBuild parameters

To use the CSI2/PPE interface, is necessary to define at least one setup structure, to be used in the Csi2_Setup call. But is possible to use as many as necessary different structures if the interface must work in many contexts to receive data. Each defined line in the list on the figure 17 represent a setup structure.

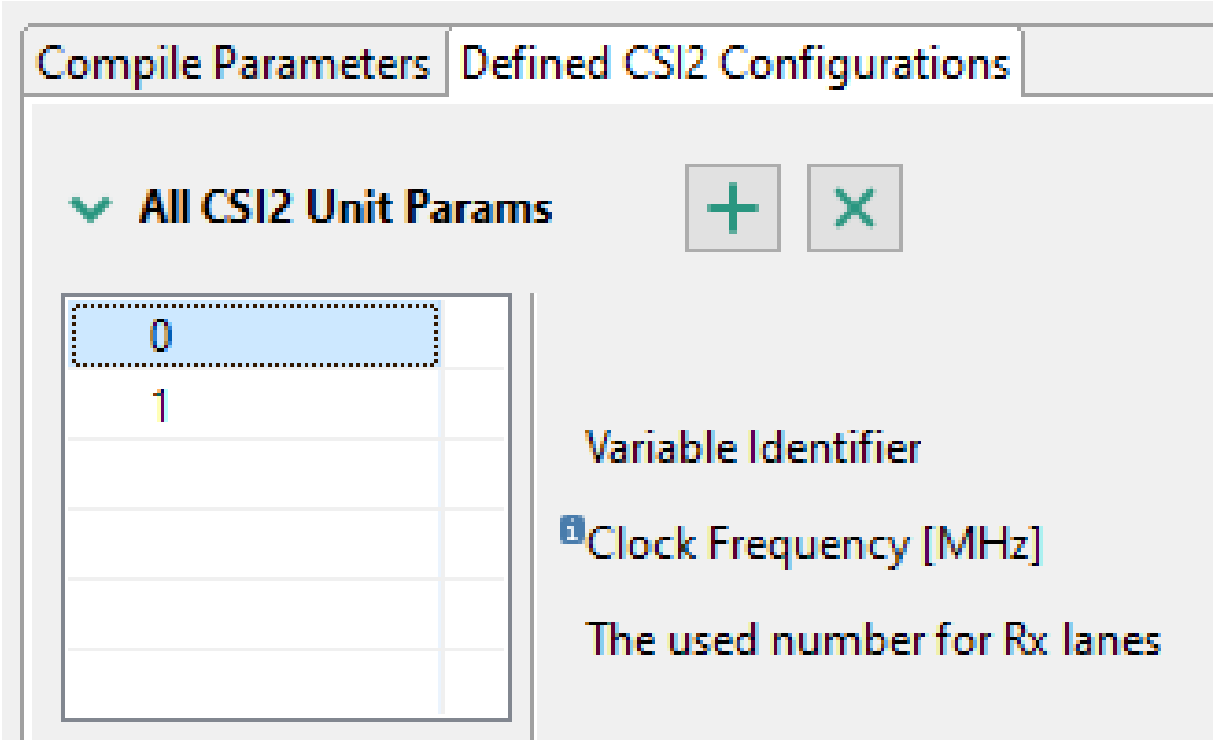


Figure 3.18 : CSI2 S32DS configuration - PostBuild setup parameters list

To define the PostBuild parameters, means to define all necessary functional parameters for the interface to receive correctly the data. For this, is necessary to define two levels of parameters :

- Unit level parameters, to be used at the unit level
- Virtual Channel level parameters, parameters to be used to receive a specific Virtual Channel data

3.4.2.4 CSI2 configuration - CSI2 S32DS configuration - Unit level

For each parameters line below, it is necessary to define the general unit parameters, pictured in figure 18.

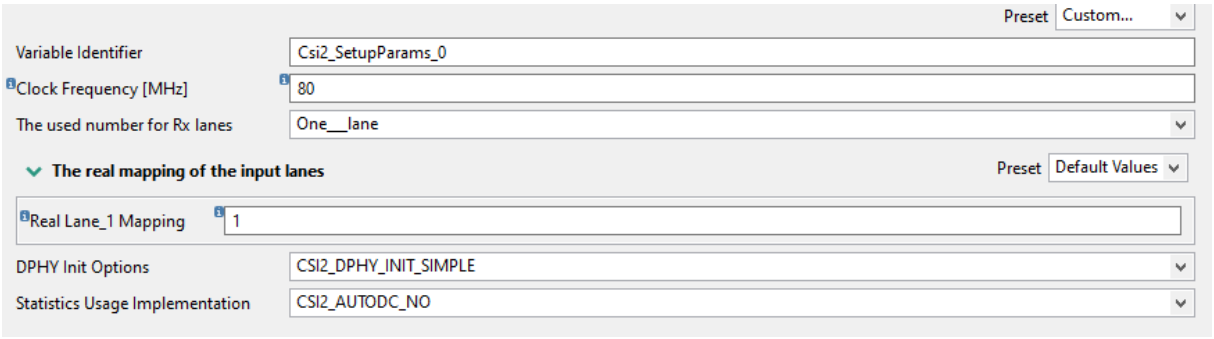


Figure 3.19 : CSI2 S32DS configuration - PostBuild Unit level parameters

The meanings/usage of the parameters :

- **Variable Identifier** = the name to be used for the complete CSI2 unit configuration. The proposed name can be updated, according to the used needs. The only requirement : if many configurations defined, the 'Variable Identifiers' must be distinct.
- **Rx Clock Frequency** = the frequency of the MIPI-CSI2 clock of the transmitter, the unit used is MHz, 2500 in the box represents a 2.5GHz clock. If using an external Radar Front End, 0 must not be used, but in the specific case of SAF85XX 0 means the ADC data must be received over the internal path.
- **Number of Lanes Used** = the number of lines used for receiving data from an external Front-End, possible values :
  - **One___lane**
  - **Two___lanes**
  - **Three_lanes**
  - **Four__lanes**
- **Lanes Mapping** = the way the external input is mapped
- **DPHY Init Options** = the initialization type of CSI2-DHY :
  - **CSI2_DPHY_INIT_SIMPLE** = simple initialization and calibration, all PHY internal values reset to 0
  - **CSI2_DPHY_INIT_SHORT_CALIB** = the quickest initialization and calibration, all PHY internal values updated to the latest used before
  - **CSI2_DPHY_INIT_W_STOP_STATE** = simple initialization and calibration followed by a wait for STOP STATE on the used lanes
  - **CSI2_DPHY_INIT_SHORT_AND_STOP** = the quickest initialization and calibration followed by a wait for STOP STATE on the used lanes
    The time spent to initialize depends of the platform/hardware/initialization type and can be up to 1.2 ms
- **Statistics Usage Implementation** = the necessary selection for the statistical data usage/calculation, please remember that the computation is done in the interrupt handler :
  - **CSI2_AUTODC_NO** = no statistical computation, no automated DC update
  - **CSI2_AUTODC_EVERY_LINE** = the statistical computation will be done using all the statistical data received, for all chirps; this require a long processor time to process data
  - **CSI2_AUTODC_AT_FE** = the computation will be done only at the end of the frame, when the Frame End event is received, using all the chirps/lines in the memory buffer
  - **CSI2_AUTODC_LAST_LINE** = the computation will be done only at the end of the frame, when the Frame End event is received, using only the last chirp/line in the memory buffer

3.4.2.5 CSI2 configuration - PostBuild parameters - Virtual Channel level

For each parameters line in the figure 17, it is necessary to define the Virtual Channel parameters. For each general parameters structure, it is necessary to define from one to four Virtual Channel setup. It is possible to define/use only distinct Virtual Channels in a single setup, numbered from 0 to 3. If this requirement is not respected, the plugin will report an error. A possible list of Virtual Channels setup is pictured in figure 19.

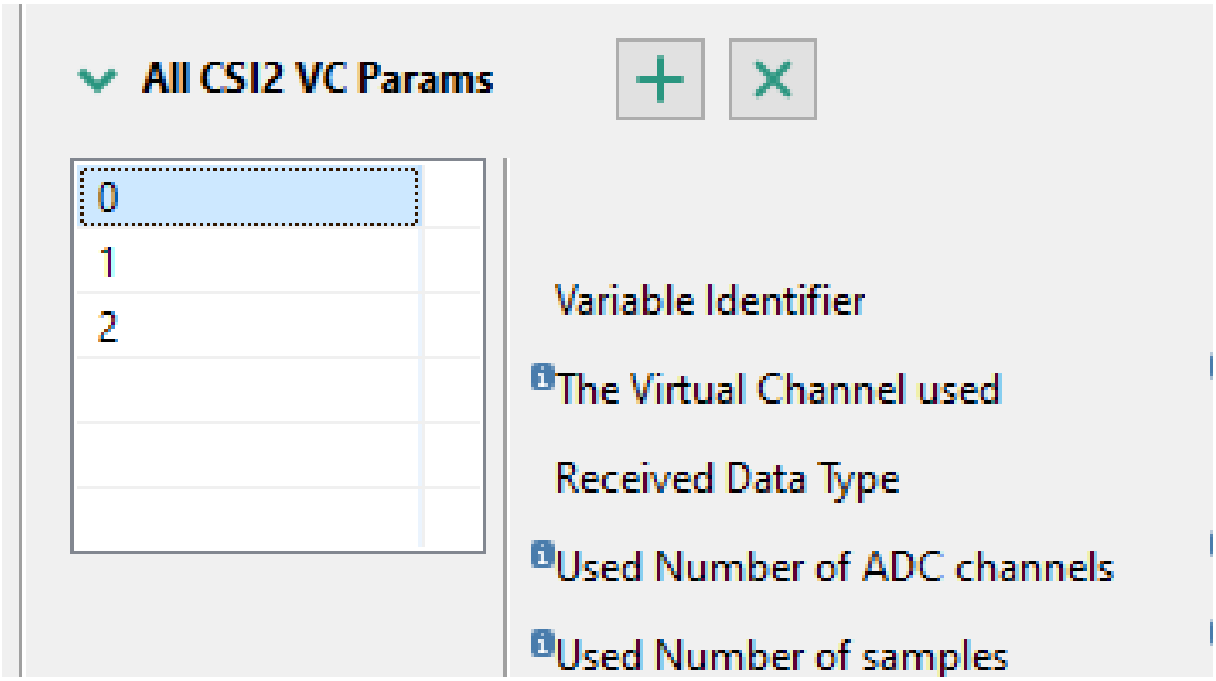


Figure 3.20 : CSI2 S32DS configuration - PostBuild defined Virtual Channel list

For each Virtual Channel used to receive data must be provided the necessary setup data, as above:

General parameters for Virtual Channel

The general **VC** parameters must be provided even all the setup options for PreCompile were set to OFF. The general parameters are pictured below.

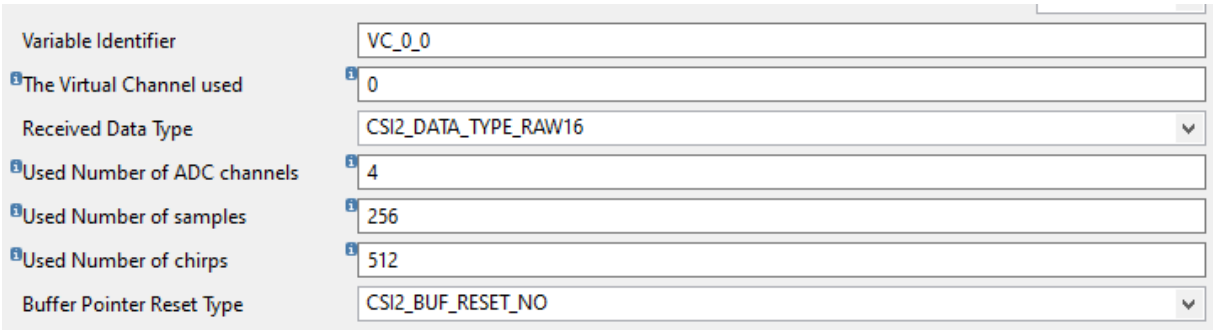


Figure 3.21 : CSI2 S32DS configuration - PostBuild Virtual Channel general parameters

- **Variable Identifier** = defines the variable name for the Virtual Channel structure. It is possible to change the default name to any C variable name, but all defined names for parameters must be unique across entire setup.

- **Virtual Channel Used** = defines the ID of the virtual channel to which this setup is referring. It could be only in range [0...3] and a different value must be used for each **VC** parameters set.
- **Received Data Type** = the stream type to be received using this **VC**. The possible types are aligned with the **RM** defined types and **MIPI-CSI2** documentation; it could be one of :
  - **CSI2_DATA_TYPE_EMBD** = embedded data type (0x12)
  - **CSI2_DATA_TYPE_YUV422_8** = YUV422 video data type, 8 bits version (0x1e)
  - **CSI2_DATA_TYPE_YUV422_10** = YUV422 video data type, 10 bits version (0x1f)
  - **CSI2_DATA_TYPE_RGB565** = RGB565 video data type (0x22)
  - **CSI2_DATA_TYPE_RGB888** = RGB888 video data type (0x24)
  - **CSI2_DATA_TYPE_RAW8** = RAW8 data type (0x2a)
  - **CSI2_DATA_TYPE_RAW10** = RAW10 data type (0x2b)
  - **CSI2_DATA_TYPE_RAW12** = RAW12 data type (0x2c)
  - **CSI2_DATA_TYPE_RAW14** = RAW14 data type (0x2d)
  - **CSI2_DATA_TYPE_RAW16** = RAW16 data type (0x2e)
  - **CSI2_DATA_TYPE_USR0** = user data type (0x30)
  - **CSI2_DATA_TYPE_USR1** = user data type (0x31)
  - **CSI2_DATA_TYPE_USR2** = user data type (0x32)
  - **CSI2_DATA_TYPE_USR3** = user data type (0x33)
  - **CSI2_DATA_TYPE_USR4** = user data type (0x34)
  - **CSI2_DATA_TYPE_USR5** = user data type (0x35)
  - **CSI2_DATA_TYPE_16_FROM_8** = = user data type (0x36), used to convert a 8 bits received data to 16 bits data LSB format
  - **CSI2_DATA_TYPE_USR7** = user data type (0x37)
    For **RAW** data type, at processing time, it is necessary to check the format conversion done in the hardware interface, data alignment and sign extension, please check **RM**, chapter **ADC channel data processing for RAW data**
- **Used Number of ADC channels** = the number of antennas/channels/ADC used to get data; the interface is expecting to receive data samples in this sequence for 4 antennas and real data : ADC0_0, ADC1_0, ADC2_0, ADC3_0, ADC0_1, ADC1_1, ADC2_1, ADC3_1, ADC0_2, ADC1_2, ADC2_2, ADC3_2, ...
- **Used Number of Samples** = the number of samples to receive in a chirp for a single channel/antenna
- **Used Number of Chirps** = the number of the chirps in a frame for this **VC**
- **Buffer Pointer Reset Type** = how to be handled the buffer pointer at Frame Start
  - **CSI2_BUF_RESET_NO** = the buffer pointer continue to write data using the next available line in the buffer. If the last frame was not completely received, the pointer will write the new Frame at another place then the one expected
  - **CSI2_BUF_RESET_FS** = the first chirp of the frame will be written always starting the first line in the data buffer

Buffer Data setup for Virtual Channel

For each **VC** data to be received is necessary to specify the buffer management :

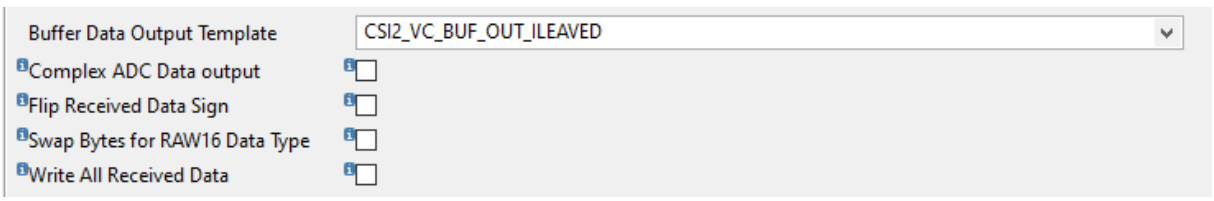


Figure 3.22 : CSI2 S32DS configuration - PostBuild Virtual Channel data buffer setup

- **Buffer Data Output Template** = specify the template to be used when the data is written into the memory :
  - **CSI2_VC_BUF_OUT_ILEAVED** = the data is written as is receives, see above, **Number of channels/antennas**
  - **CSI2_VC_BUF_OUT_TILE8** = the data will be aggregated to have 8 samples from the same ADC together, like this : ADC0_0, ADC0_1, ... , ADC0_7, ADC1_0, ADC1_1, ... , ADC1_7, ADC2_0, ADC2_1, ..., ADC2_7, ADC3_0, ADC3_1, ..., ADC3_7, ADC0_8, ADC0_9, ..., ADC0_15, ...
  - **CSI2_VC_BUF_OUT_TILE16** = the data will be aggregated to have 16 samples from the same ADC together, like this : ADC0_0, ADC0_1, ... , ADC0_15, ADC1_0, ADC1_1, ... , ADC1_15, ADC2_0, ADC2_1, ..., ADC2_15, ADC3_0, ADC3_1, ..., ADC3_15, ADC0_16, ADC0_17, ..., ADC0_31, ...
- **Complex Data to Receive** = specify the type of data to be received
  - **ON** = complex data to receive, this will double the number of above declared antennas/channels. The complex data will be mixed with the real data, it means the received data flow will be : ADC0_real, ADC0_complex, ADC1_real, ADC1_complex, ... which will affect the order in the data buffer specified in the point above
  - **OFF** = only real data received
- **Flip Received Data Sign**
  - **ON** = the adjusted RAW data will have the sign flipped
  - **OFF** = the RAW data sign will not be flipped
- **Swap Bytes for RAW8 Data Type** = how to write the received RAW8 data into the memory
  - **ON** = for RAW8 data only, the samples will be put in the buffer in a reversed order, to allow reversing the 16 bits values if a 16 bits value was sent as RAW8, from MSB to LSB or from LSB to MSB
  - **OFF** = RAW8 data put in the memory in the normal order
- **Swap Bytes for RAW16 Data Type** = same as above, but referring to RAW16 data stream
- **Write All Received Data** = how to manage the data received before **Frame Start**
  - **ON** = the data will be written into the memory as received, even Frame Start was not received and possible some of the chirps too; this can affect the first line written into the buffer for the next frame
  - **OFF** = the data will be written into the memory only if a Frame Start is received

Buffer Data parameters for Virtual Channel

For each **VC** data to be received is necessary to specify the buffer parameters :

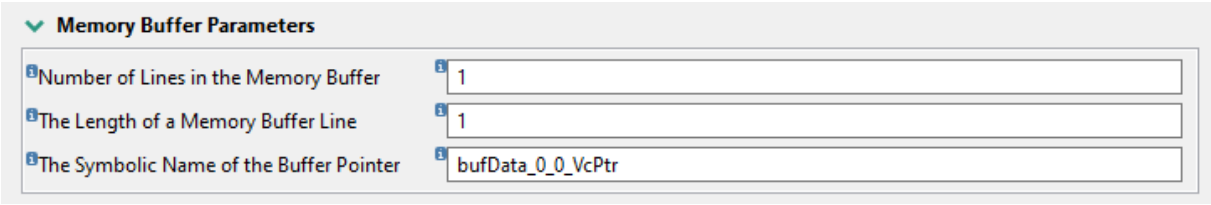


Figure 3.23 : CSI2 S32DS configuration - PostBuild Virtual Channel data buffer parameters

- **Number of Lines in the Memory Buffer** = the number of lines in the buffer; if the specified number of lines is N (the lines are numbered from 1 to N), after writing the line N, line 1 will be written; this means, only if N is the numbers of chirps to be received the data will not be overwritten for a frame.
- **The Length of a Memory Buffer Line** = the bytes reserved for a chirp; be careful :

  - the length of data is generally : < number_of_channels > ∗ < number_of_samples_per_channel > ∗ < bytes_per_sample_in_memory >
  - if statistical data is required to be added at the end of the chirp, 80 bytes must be added to the length above

- **The Symbolic Name of the Buffer Pointer** = the buffer name to be used in the application; the buffer will be defined in Csi2_PBCfg.c file.

DC offset parameters for Virtual Channel

For each received channel is possible to set a DC offset. The DC offset can be a fixed value, set at the initialization or is possible to be updated after each frame if automatic DC offset is set, see CSI2 configuration - PreCompile parameters.
The value specified as DC offset is subtract from the received values, with an exception : the value 32767, which is the maximum positive value to be set for a 16 bits signed value. This value require for that channel to adjust the DC offset after each frame (automatic DC offset). This is done per channel, so only the channels with this DC offset will be updated automatically.

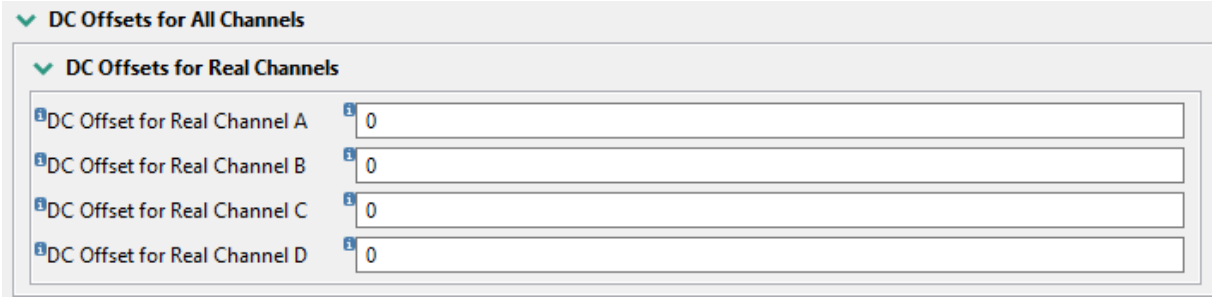


Figure 3.24 : CSI2 S32DS configuration - PostBuild Virtual Channel channels DC offsets

If the complex data is received, DC management is similar.

Events for Virtual Channel

For each defined Virtual Channel is possible to request some events signals. The field Csi2_ErrorReportType↩ ::evtMaskVC is signaling the detected signals.
There is a big difference between errors and signals :

- any error is detected, is reported to the application using Csi2_ErrorReportType structure and the appropriate callback
- an event is reported to the application using the same Csi2_ErrorReportType, but only if required at setup/initialization time; if the event is not required, the application will not receive a signal if the event is detected

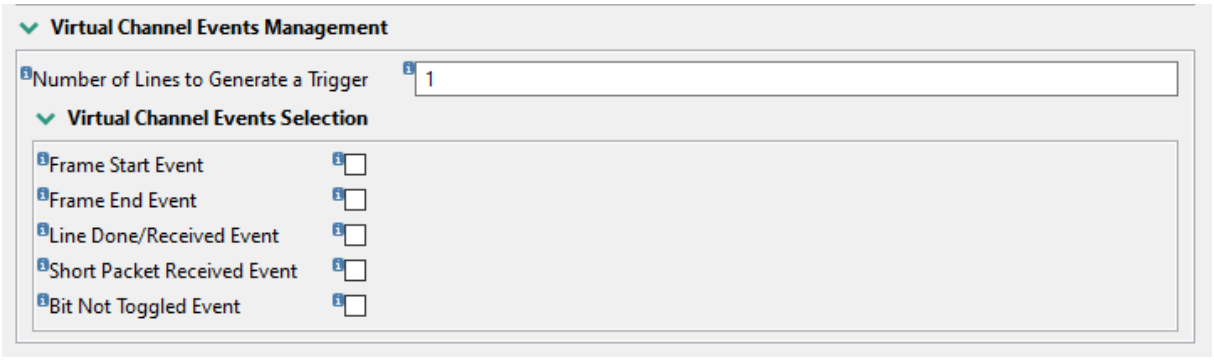


Figure 3.25 : CSI2 S32DS configuration - PostBuild Virtual Channel events

Is possible to request these events to be reported :

- **Frame Start Event** = this event is reported when a **Frame Start** packet is received
- **Frame End Event** = this event is reported when a **Frame End** packet is received
- **Line Done/Received Event** = this event is reported when is received the requested amount of chirps : a multiple of the parameter :
  - **Number of Lines to Generate a Trigger** = this is an important value and has two different meanings :
    - the number of chirps received at which the driver reports a **Line Done Event**
    - the number of chirps received at which **SPT** will receive a CSI2 line received event
      So, set the appropriate value to this parameter.
- **Short Packet Received Event** = event signaled when an unexpected short packet is received. The expected short packets are : Frame Start, Frame End.
- **Bit Not Toggled Event** = this event signal that a specific bit, for a specific channel, is not changed over the entire chirp. This event can be reported only if Statistical data is reported.
- **Start Not Zero Event** = this event is reported if at Frame End the next line to receive data is not the first line in the buffer

GPIO triggers for Virtual Channel

The Packet Processor interface is able to trigger an output GPIO signal, using the dedicated PE[6] output pad. Is possible to trigger an output signal for many CSI2/PPE internal events :

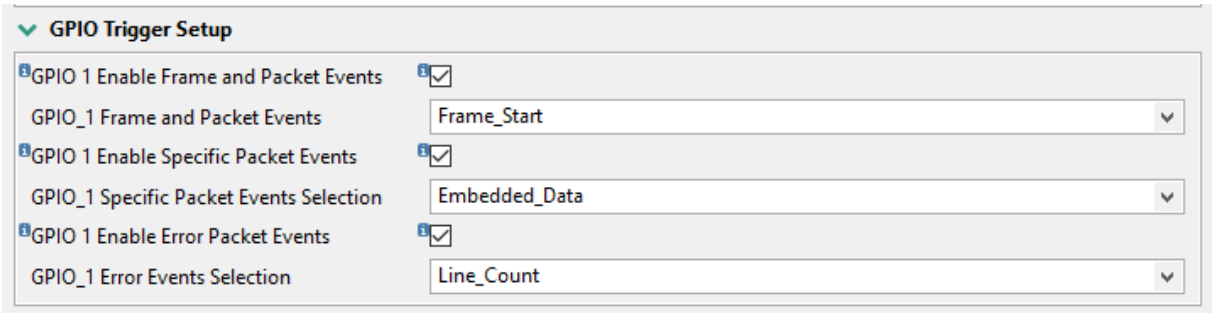


Figure 3.26 : CSI2 S32DS configuration - PostBuild Virtual Channel GPIO trigger setup

- **GPIO 1 Enable Frame and Packet Events** = GPIO triggers at some specific moments, related to packet protocol level :
  - **OFF** = no triggers generated for these events
  - **ON** = triggers generated for the selected event :
    - **Frame_Start**
    - **Frame_End**
    - **Packet_Start**
    - **Packet_End**
- **GPIO 1 Enable Specific Packet Events** = triggers generated when the specific data packet is received
  - **OFF** = no triggers generated for these events
  - **ON** = triggers generated for the selected event :
    - **Embedded_Data**
    - **User_Defined_Data**

        - **RAW_Data**
        - **RGB_Data**

- **GPIO 1 Enable Error Packet Events** = triggers generated when the specific error is detected
    - **Line_Count**
    - **Line_Length**
    - **CRC_or_ECC2bits**
    - **No_Synchronization_on_DPHY**

SDMA triggers for Virtual Channel

The Packet Processor interface is able to trigger an SDMA signal and start a SDMA data transfer if requested :

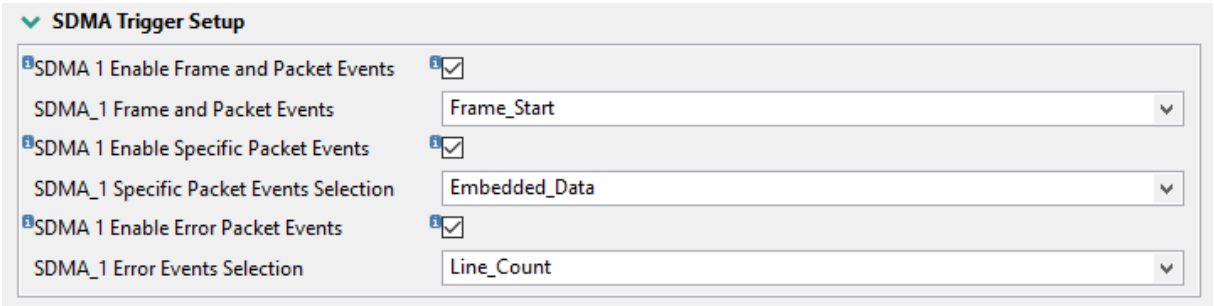


Figure 3.27 : CSI2 S32DS configuration - PostBuild Virtual Channel SDMA trigger setup

- **SDMA_1 Frame and Packet Events** = SDMA triggers at some specific moments, related to packet protocol level :
    - **OFF** = no triggers generated for these events
    - **ON** = triggers generated for the selected event :
        - **Frame_Start**
        - **Frame_End**
        - **Packet_Start**
        - **Packet_End**
- **SDMA_1 Specific Packet Events Selection** = triggers generated when the specific data packet is received
    - **OFF** = no triggers generated for these events
    - **ON** = triggers generated for the selected event :
        - **Embedded_Data**
        - **User_Defined_Data**
        - **RAW_Data**
        - **RGB_Data**
- **SDMA 1 Enable Error Events** = triggers generated when the specific error is detected
    - **Line_Count**
    - **Line_Length**
    - **CRC_or_ECC2bits**
    - **No_Synchronization_on_DPHY**

Metadata for Virtual Channel

The Packet Processor interface is able to receive Metadata. The Metadata is number of chirps received usually at the end of the frame. The data type is different of the normal radar data and can be selected from a limited range of data types. Generally, this kind of data is restrictive, but allow to receive some limited specific data at the end of the frame, like the Front-End general status.
This feature is not active if **Csi2 Driver MetaData Usage** in CSI2 configuration - PreCompile parameters is not set to **ON**.

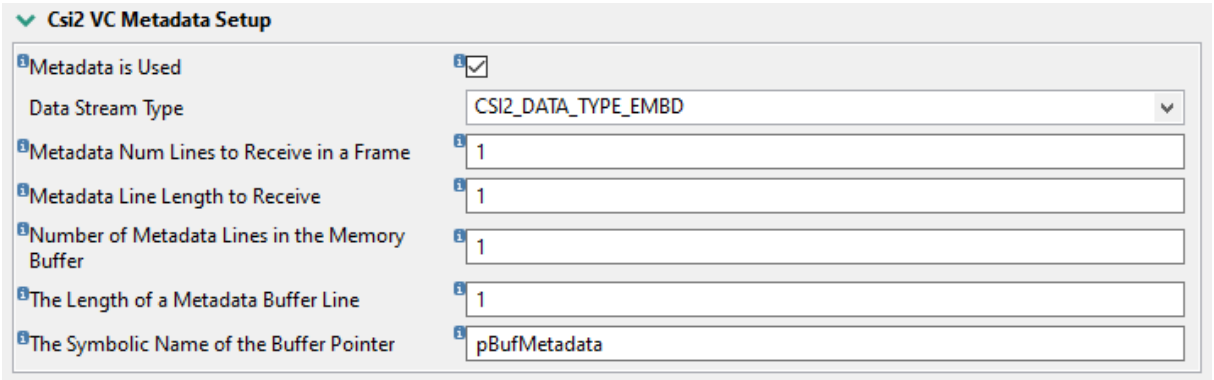


Figure 3.28 : CSI2 S32DS configuration - PostBuild Virtual Channel Metadata parameters

- **Metadata is Used**
  - **OFF** = Metadata is not used on this Virtual Channel received data
  - **ON** = Metadata is used for this Virtual Channel received data
- **Data Stream Type** = the data type to be received as Metadata, it must be different than the normal data received for this Virtual Channel; the list is similar to the one for **Data Stream Type**.
- **Metadata Num Lines to Receive in a Frame** = the number of lines to be received as Metadata
- **Metadata Line Length to Receive** = the bytes amount of Metadata to receive
- **Number of Metadata Lines in the Memory Buffer** = the number of lines in buffer to receive Metadata; the buffer is a circular buffer, as for normal data
- **The Length of a Metadata Buffer Line** = the length of line buffer data; the buffer line length must respect the same rules as for normal data, for alignment and supplementary 80 bytes if statistical data activated
- **The Symbolic Name of the Buffer Pointer** = the Metadata buffer name to be used in application

Auxiliary data for Virtual Channel

The Packet Processor interface is able to receive Auxiliary data. The Auxiliary data is a part of the received data : the received data is split in two parts - the normal radar data and Auxiliary data. Usually the ratio is 50-50, but please check the Reference Manual for the possible schemas. The main point : the data is written into the memory in distinct buffers. This feature is not active if **Csi2 Driver Auxiliary Data Usage** in CSI2 configuration - PreCompile parameters is not set to **ON**.

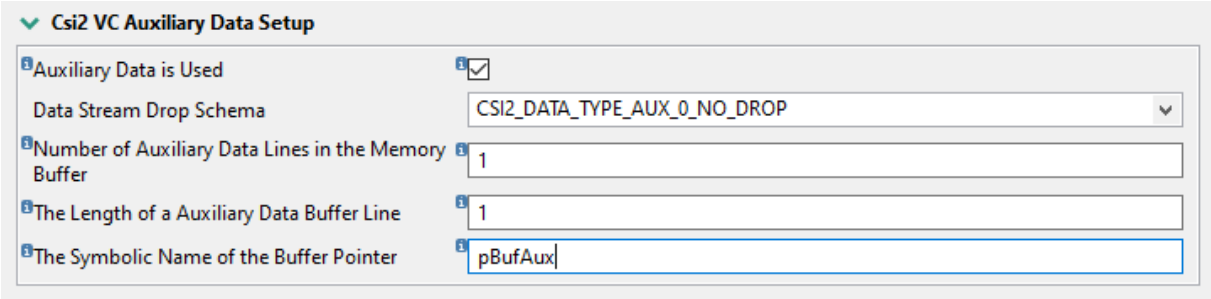


Figure 3.29 : CSI2 S32DS configuration - PostBuild Virtual Channel Auxiliary data parameters

- **Auxiliary Data is Used**
  - **OFF** = Auxiliary data is not used on this Virtual Channel received data
  - **ON** = Auxiliary data is used for this Virtual Channel received data
- **Data Stream Drop Schema** = the schema to split the received data and to write it into the Auxiliary data buffer :
  - **CSI2_DATA_TYPE_AUX_0_NO_DROP** = the first schema, all the Auxiliary data received is written into the memory
  - **CSI2_DATA_TYPE_AUX_0_DR_1OF2** = the second schema, only a half of the Auxiliary data received is written into the memory
  - **CSI2_DATA_TYPE_AUX_0_DR_3OF4** = the third schema, only a quarter of the Auxiliary data received is written into the memory
  - **CSI2_DATA_TYPE_AUX_1_NO_DROP** = the fourth schema, the data is not split 50-50, but like (4 data samples + 2 Auxiliary data samples), and all Auxiliary data is written into the memory
- **Number of Auxiliary Data Lines in the Memory Buffer** = the number of lines in buffer to receive Auxiliary data; the buffer is a circular buffer, as for normal data
- **The Length of an Auxiliary Data Buffer Line** = the length of line buffer data; the buffer line length must respect the same rules as for normal data, for alignment and supplementary 80 bytes if statistical data activated
- **The Symbolic Name of the Buffer Pointer** = the Auxiliary data buffer name to be used in application

Chapter 4

CTE User Guide

4.1 Introduction

This is a detailed description of the CTE module, which provide all necessary functions/procedures to initialize the CTE unit. More details can be found at CTE User Guide and cte_asr_api_func.

This section presents the **Radar SDK** software components provided for enablement of the **Cross Trigger Engine** module.
It provides details about their functionality and programming modes. It contains all guidelines for an easy integration. Cross Trigger Engine can be used on the RadarSDK environment to generate more complex hardware trigger events, related or not to other received hardware events. For example, the data processing synchronization is done by CTE, generating specific SPT signals (Radar Frame Start (RFS), Radar Chirp Start (RCS) or SPT events (0...3) and not waiting a very specific MIPI-CSI2 signal.

4.2 Module description

The RSDK CTE module consists of the CTE Driver, software component.

The CTE Driver is an independent module that the application can use to trigger various asynchronous events mainly inside the Radar subsystem (MIPI-CSI2, DSP and SPT), but GPIO could be included too.
The CTE Driver serves as signaling interface in three different contexts : software only managed with no hardware input, external signal processed and internal signal processed. The driver has the same API for all platforms using the same Operating Systems, but some data structures could be different, adjusted to the working platform. For a detailed description of the driver, see cte.
For platform implementation see the appropriate platform Reference Manual.

The rsdk_sampleapps show how to integrate these software components into the user code.

4.3 CTE Module Usage

4.3.1 Environment

The tools and dependencies used for building and executing the RSDK modules are listed in Deployment environment and dependenci section

4.3.2 CTE driver usage

The CTE Driver is not built into a separate library. As usually AUTOSAR usage, the driver files must be integrated directly as source code in top-level applications deployed on the M7 core. Before building an application which include the CTE driver, there are 4 conditions for ensuring the proper functionality of the driver:

- All source files must be added and built along with the application
- The driver's interrupt handlers must be registered accordingly
- The interrupt callback must be defined at application level
- The necessary configuration build parameters must be provided, to respect the necessary functionalities and code footprint

It is recommended to use the plugin configurator to provide the CTE driver setup parameters, see details in CTE plugin usage. The setup parameters provided as example are generated using the plugin. But even not using the plugins to configure the driver, the files CDD_Cte_PCCfg.h and CDD_Cte_PCCbk.h must be provided if at compile time **RSDK_AUTOSAR** is defined.

For **S32R41** platform there are some restrictions in the hardware setup/working :

- if the input CTE clock (CTE_CLK) is 80 MHz, the table duration, for any Time Table 0 or Time Table 1, can't be 1; the driver accept at setup time a table duration of 1, but the hardware will have the table duration set to 2, no error being returned
- if a CTEP output signal is set to clock mode, the frequency of the CTEP clock can be up to half of the CTE_CLK; the driver is calculating the required frequency and if the result is higher then CTE_CLK/2 limit the output to CTE_CLK/2, no error being returned
- if CTE is working in **SLAVE** mode and a RFS is required to reset the timing to Time Table 0 start, it must be done only in these conditions :
  - CTE must be working, an error is reported if the driver is not initialized or not running
  - CTE timing at RFS request time must be in Time Table 0; the driver check this before and after RFS trigger and raise a specific error if not in TT0 execution before or in TT1 execution after

    Note

    If CTE is working in **SLAVE** mode using external signals, if the external RFS signal is received when TT1 is executing, it is possible to corrupt the CTE signal timing, only a CTE reset solving this issue. In this case, if two tables are used, it is necessary to have a good timing correlation between CTE table timing and the external signals timing.

4.3.3 Normal processing

The following image shows the available interface states and the possible transitions.

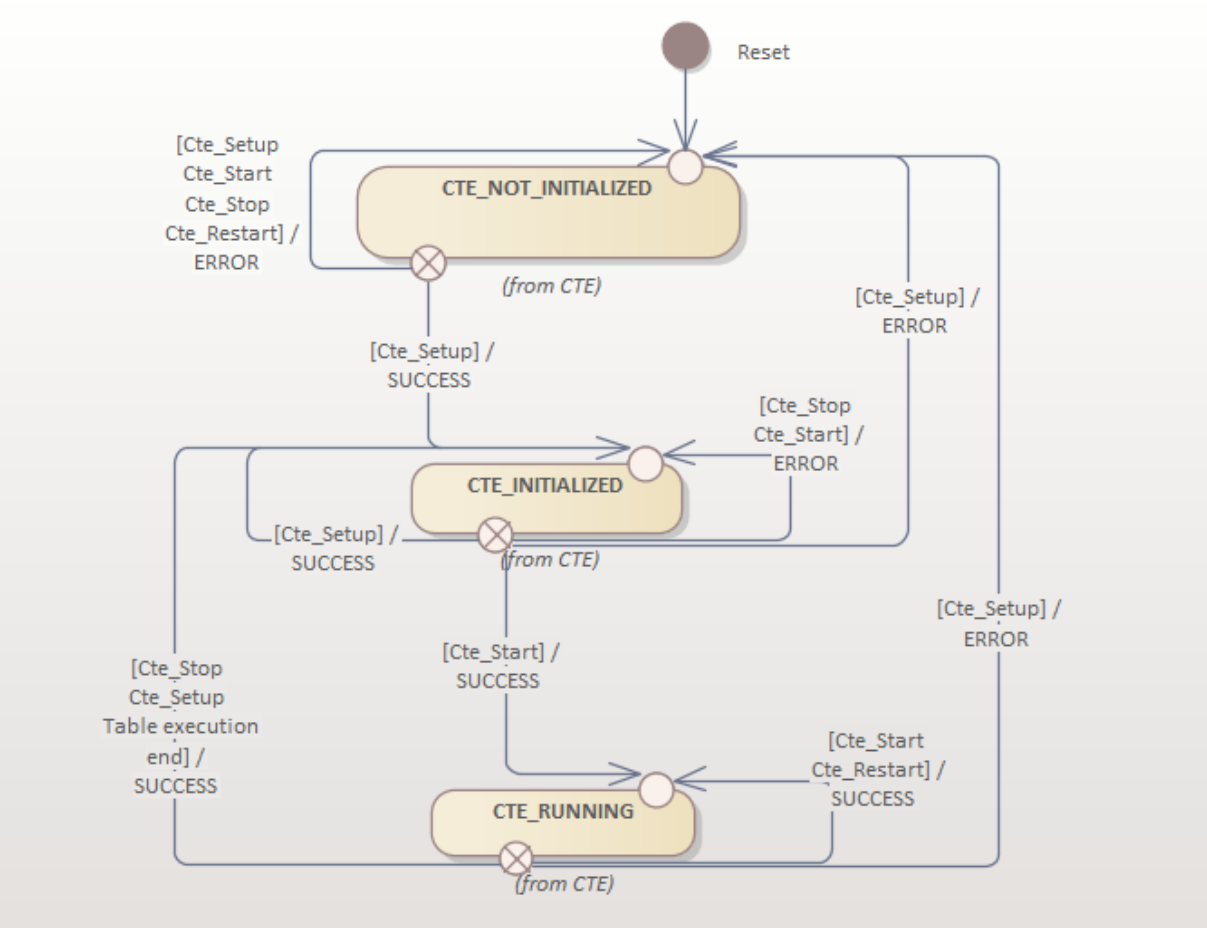


Figure 4.1 : CTE States Diagram

After the interface is initialized, only few other commands are accepted:

Cte_Start	The real functional start for the CTE unit
Cte_Stop	Stop the CTE unit
Cte_Restart	Restart the CTE unit, done by applying stop and start functions
Cte_RfsGenerate	Used only in Master mode to restart table execution
Cte_UpdateTables	Update only the timing table(s), the output settings remains the same. If necessary, the internal clocks can be changed.

4.3.4 General table execution

The driver target is to generate some output signals at the specified time. The events array is called "timing table". The hardware can process one table or two tables. The following images show how this happens, the exact table execution is detailed in Specific table execution.

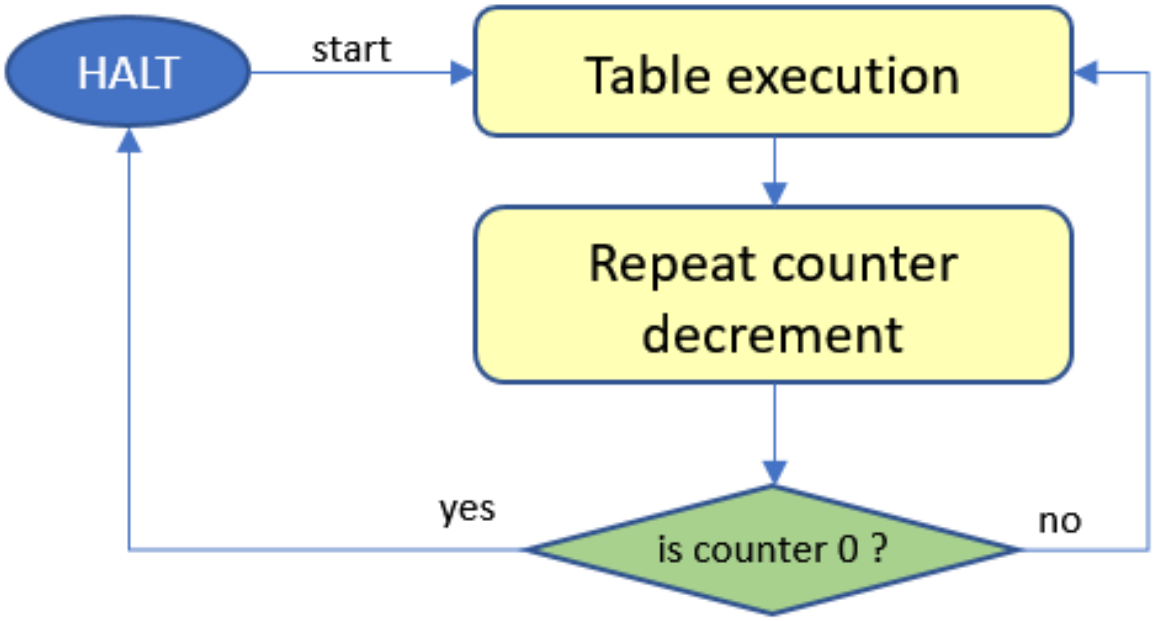


Figure 4.2 : One table execution

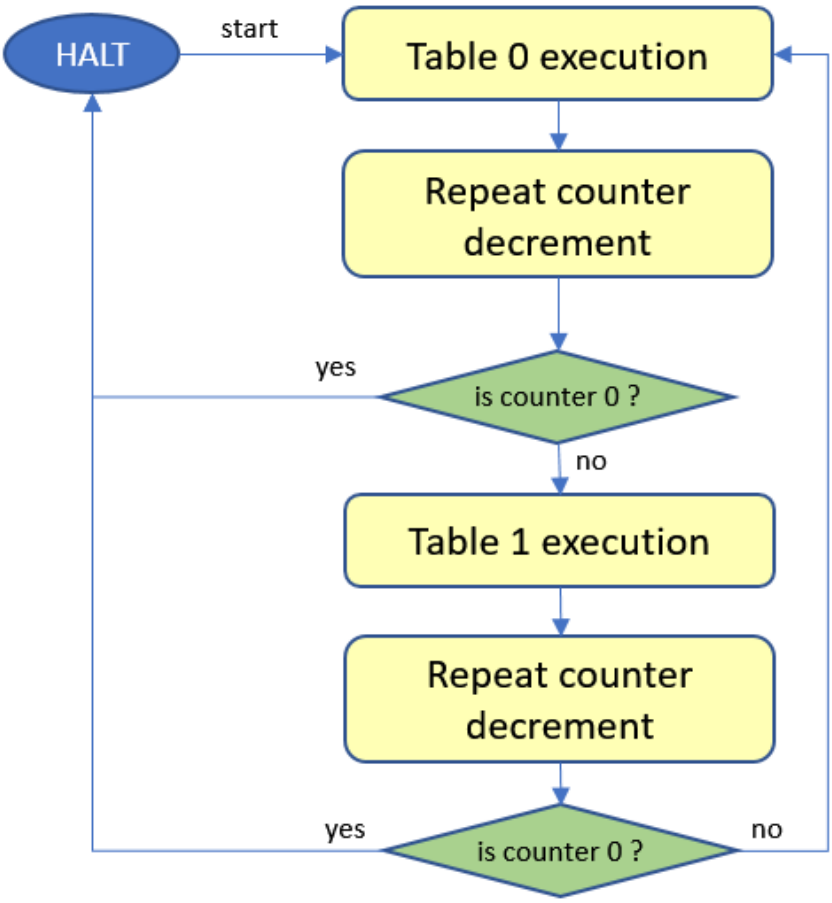


Figure 4.3 : Two tables execution

Note

Timer counter set to 0 means continuous table execution. In this case only the software can stop the CTE to work. If the counter has another value, is possible that CTE to have a hardware stop. The repeat counter is only 16 bits wide, so the maximum value is 65535.

4.3.5 Specific table execution

CTE hardware can be in three different states :

- first is DISABLED : the hardware is not enabled to work; normally this state is used to setup the working parameters
- second is RUNNING : after all necessary parameters are setup, when the CTE module is "enabled" it starts to work, according to the current setup
- third is HALT : at the end of RUNNING sequence (when the table repetition counter became 0), the CTE module is still "enabled", but any execution is stopped Further, only HALT status will be used, but DISABLED has the same significance CTE hardware can work in two different contexts : Master or Slave mode.
  In **Master** mode, the hardware doesn't accept any input signals. Only software commands manage the execution : start/stop. The specific table execution is detailed in the following image.

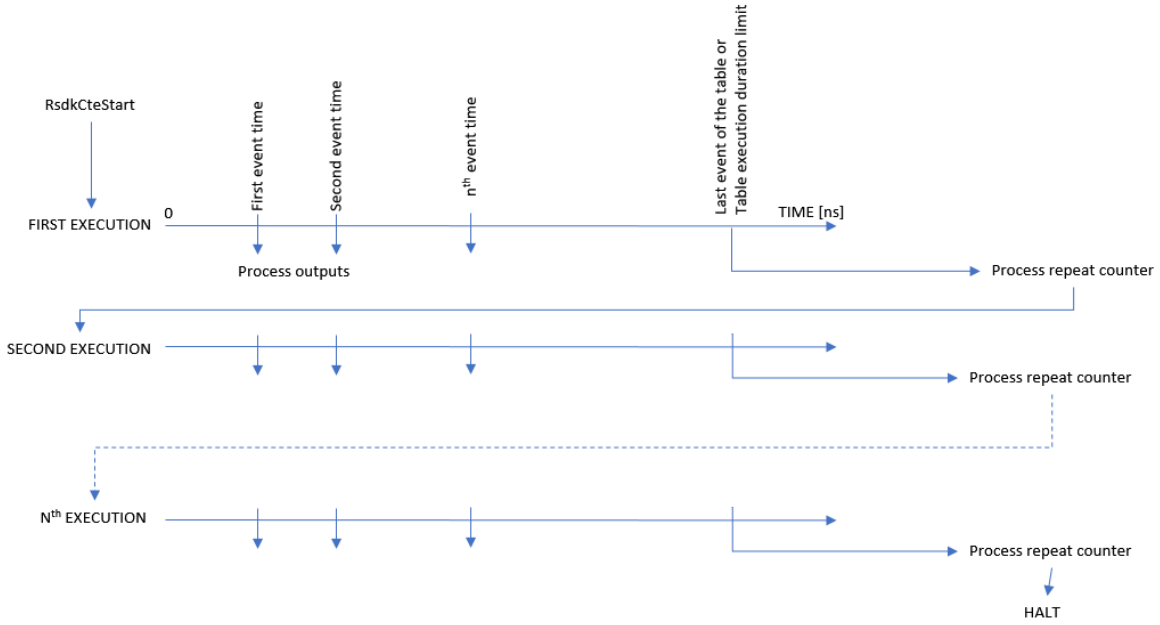


Figure 4.4 : Specific table execution in Master mode

In **Slave** mode, the table execution is triggered by the internal (MIPI-CSI2) or external (GPIO) signals. The call for Cte_Start only change the **HALT** status of the CTE, but doesn't really start the table execution. The table execution is started by the RCS signal. The RFS signal reset only the table execution index and put CTE in wait for RCS signal. The RFS signal can be software emulated. If two tables are executed, the control remains to the current table. The following image explain the table execution in **Slave** mode.

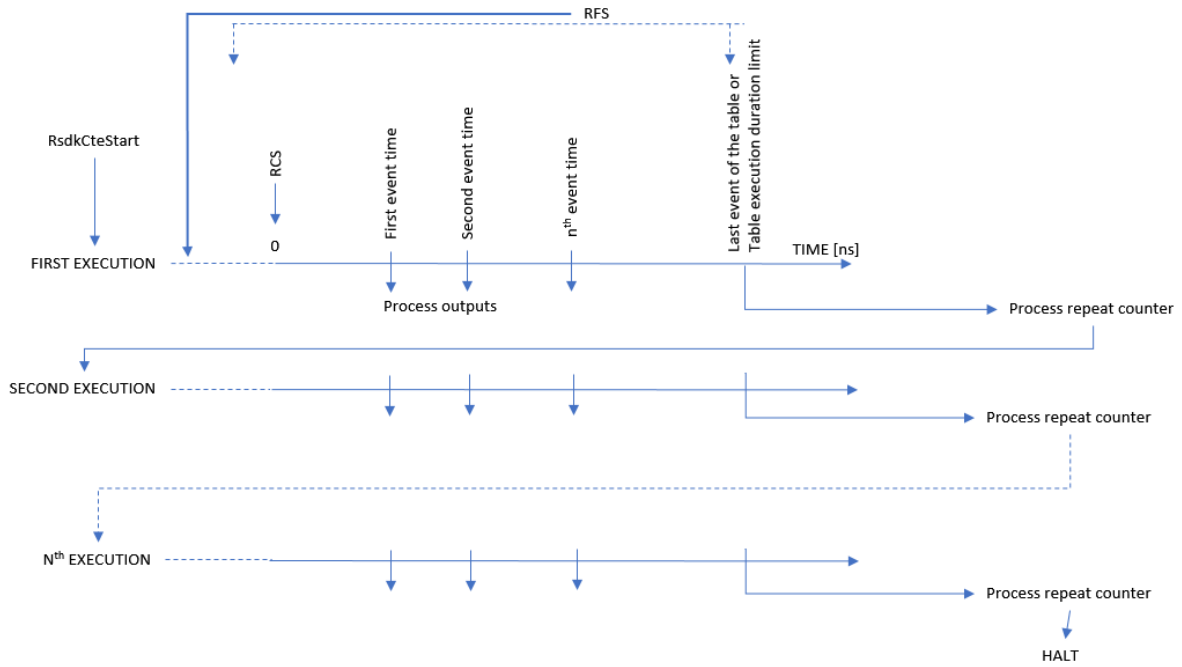


Figure 4.5 : Specific table execution in Slave mode

4.3.6 Callback usage

The only interface interactions with the application are the events. The driver are using the specified callbacks to announce the application about the events occurred on the interface. The callback will be used for every event occurred, each time. The event reported are the events set at CTE output, not at input. As example, if the CTE is working in Slave mode with CSI2 as input trigger, the RFS or RCS start reported are not the ones coming from CSI2, but the ones set by CTE to SPT; if these signals (RCS and/or RFS) are not defined in the timing table, the events will not be reported. The events reported by interrupt/callback are:

Bit	Meaning
CTE_IRQ_TT0_START	Table 0 started to execute. In **Slave** mode, it indicates the receiving of RCS signal. In **Master** mode, it only notifies the start of Table 0 execution.
CTE_IRQ_TT1_START	Table 1 started to execute. See the above explanation.
CTE_IRQ_TT0_END	Table 0 finished. It indicates that the last event was processed or the execution duration limit was counted.
CTE_IRQ_TT1_END	Table 1 finished. See the above explanation.
CTE_IRQ_RCS	The RCS signal generated. This can be used to synchronize the M7 core application to SPT execution
CTE_IRQ_RFS	The RFS signal generated. This can be used to synchronize the M7 core application to SPT execution
CTE_IRQ_TABLE_EXEC_END	The tables execution finished. It signals that CTE is in **HALT** status from this moment.

Note

The callback routine is used only if an event request was provided (Cte_SetupParamsType::cteIrqEvents is not 0).

To simplify the checks done at configuration time, the callback must be defined even no events report is required.

The driver/hardware reports no execution errors. The driver check at Cte_Start the input parameters and report any detected problem/error.

But after this point, the tables are initialized and the execution of the tables only check the counters (timing counters, table execution counters, execution duration, etc.).

Most importantly, do not set the first event time to 0!

If set to 0, the table execution hangs because the time counting is done by decreasing the specified event time (in this case 0) till it gets to 0.

4.3.7 CTE Code Usage

The necessary steps to use the CTE driver are :

- use the Tresos application and the CTE plugin to generate the necessary setup parameters :
  - configuration parameters, defined in CDD_Cte_PCCfg.h
  - callback functions, as the customer can use other names than the fixed ones, defined in CDD_Cte_↩ PCCbk.h
  - setup parameters, defined in Cte_PBCfg.c :
    - output signals setup
    - timing tables
- include into the application build files Cte_PBCfg.c and the files from < rsdk >/CTE/CTE_driver/src/low_level
- adjust the include build paths to include < rsdk >/CTE/CTE_driver/include/low_level and the Tresos plugin generated < output >/include files
- use Cte_Setup function in the application code to setup the interface and an executive function like Cte_Start

The generated CDD_Cte_PCCfg.h parameters looks like :

```
/*==================================================================================================
*                                       DEFINES AND MACROS
==================================================================================================*/
/* Pre-processor switch to enable/disable development error detection for Cte API */
#define CTE_DEV_ERROR_DETECT                STD_ON

/* Pre-processor switch to enable/disable stop execution after error detection for Cte API */
#define CTE_DEV_HALT_ON_ERROR               STD_OFF

/* Pre-processor switch to define single Cte management thread or multiple Cte management threads.
 * If single thread (which is the normal approach) - there are no necessary exclusive areas for the driver
      */
#define CTE_SINGLE_MANAGEMENT_THREADS       STD_ON

/* Pre-processor switch to enable/disable version info report for Cte API                    */
#define CTE_VERSION_INFO_API                STD_ON

/* Pre-processor switch to enable/disable Rx start/stop usage in CTE API                     */
#define CTE_START_STOP_USAGE                STD_ON
```

The generated CDD_Cte_PCCfg.h parameters looks like :

```
/*
 * Output Configurations
 */

Cte_SingleOutputDefType outputDefTable_0[] = {
    { CTE_OUTPUT_CTEP_1, CTE_OUT_HIZ, 0u},
    { CTE_OUTPUT_SPT_RCS, CTE_OUT_LOGIC, 0u },
    { CTE_OUTPUT_FLEX_0, CTE_OUT_LOGIC, 0u },
    { CTE_OUTPUT_MAX, CTE_OUT_LOGIC, 0u}
```

```
};

/*
 * Time Tables Single Events Definitions
 */
Cte_ActionType table_0_Action_0[] = {
    { CTE_OUTPUT_SPT_0, { CTE_LOGIC_SET_TO_HIGH_Z}},
    { CTE_OUTPUT_SPT_1, { CTE_LOGIC_SET_TO_LOW}},
    { CTE_OUTPUT_CTEP_0, { CTE_CLOCK_SET_TO_HIGH}},
    { CTE_OUTPUT_MAX, {CTE_OUT_LOGIC}}
};

Cte_ActionType table_0_Action_1[] = {
    { CTE_OUTPUT_SPT_0, { CTE_LOGIC_SET_TO_HIGH}},
    { CTE_OUTPUT_MAX, {CTE_OUT_LOGIC}}
};

Cte_ActionType table_0_Action_2[] = {
    { CTE_OUTPUT_SPT_0, { CTE_LOGIC_SET_TO_LOW}},
    { CTE_OUTPUT_SPT_1, { CTE_LOGIC_SET_TO_HIGH}},
    { CTE_OUTPUT_MAX, {CTE_OUT_LOGIC}}
};

Cte_ActionType table_1_Action_0[] = {
    { CTE_OUTPUT_CTEP_4, { CTE_LOGIC_SET_TO_LOW}},
    { CTE_OUTPUT_MAX, {CTE_OUT_LOGIC}}
};

/*
 * Time Tables Events Definitions
 */
Cte_TimingEventType listEventsTable_0[] = {
    {1000u, table_0_Action_0},
    {2000u, table_0_Action_1},
    {3000u, table_0_Action_2}
};

Cte_TimingEventType listEventsTable_1[] = {
    {1000u, table_1_Action_0}
};

/*
 * Time Tables Definitions
 */
Cte_TimeTableDefType timeTable_0 = {
    3u,
    1000000u,
    listEventsTable_0
};

Cte_TimeTableDefType timeTable_1 = {
    1u,
    1000000u,
    listEventsTable_1
};

/*
 * Complete CTE configurations
 */
Cte_SetupParamsType cteSetupParam_0 = {
    .cteClockFrecq = 40000000u,
    .cteMode = { CTE_SLAVE_CSI2, { .cteCsi2Unit = CTE_CSI2_UNIT_0}, { .cteCsi2Vc = CTE_CSI2_VC_0}},
    .repeatCount = 0u,
    .pSignalDef0 = outputDefTable_0,
    .pSignalDef1 = outputDefTable_0,
    .pTimeTable0 = &timeTable_0,
    .pTimeTable1 = &timeTable_1,
    .cteIrqEvents = 0u
};
```

4.4 CTE plugin usage

This section explain how to use the CTE provided plugins, for ElectroBit Tresos and for S32 Design Studio.

4.4.1 Tresos plugin

CTE plugin for Tresos is offering the possibility to setup the CTE driver at two levels :

- PreCompile (**PC**) - only **CDD_Cte_PCCfg.h** and **CDD_Csi2_PCCbk.h** files are produced
- PostBuild (**PB**) - there are produced the two files specified above and the necessary **Cte_PBCfg.c** file, which contains all required configuration parameters; at least one complete parameters structure must be defined, but is possible to define as many structures as necessary

4.4.1.1 Tresos plugin - PreCompile parameters

In the figure 6 is pictured the plugin interface for the PreCompile setup. All necessary **PC** parameters are there.

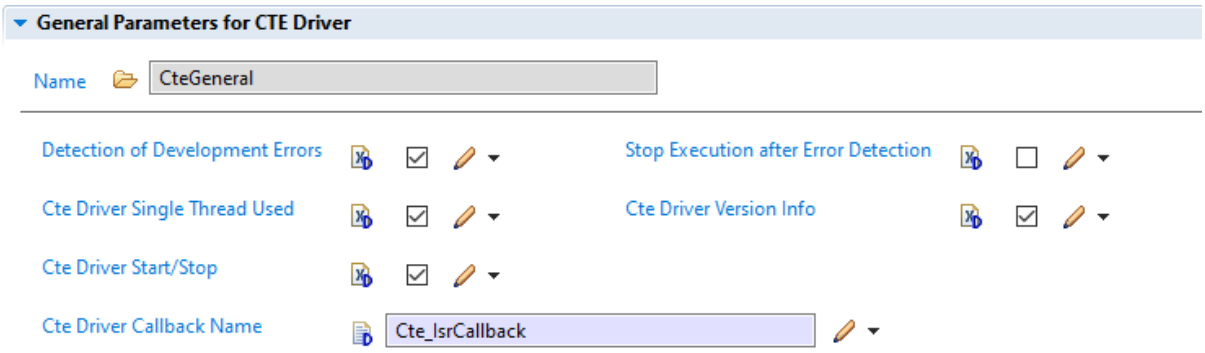


Figure 4.6 : CTE Tresos plugin - PreCompile page

The image display the default setup parameters, at the first usage of the plugin. All can be changed, but it could be some links between some parameters:

- **Detection of Development Errors**
  - **ON** = the input parameters will be checked before used
  - **OFF** = the input parameters will not be checked, the necessary code for this is removed before compile time
- **Stop Execution After Error Report**
  - **ON** = the execution of the code stops immediately after the error was reported to **Det**
  - **OFF** = the execution doesn't stop after error is reported to Det
- **Csi2 Driver Single Thread Used**
  - **ON** = the CTE driver is used in only one thread, the normal and the recommended usage
  - **OFF** = the CTE driver is used in more threads
- **Cte Driver Version Info Available**, the AUTOSAR specific parameter
  - **ON** = the driver reports the version info
  - **OFF** = the driver doesn't report the version info, the necessary code is removed at compile time
- **Cte Driver Callback Name**
  - the name of the used callback function; the callback function will be used to report only CTE requested events, but must be implemented in the code even no events are required

4.4.1.2 Tresos plugin - Output Setup List

To setup the driver, at least one setup parameter structure must be defined. According to the necessary usage, can be defined as many setup structures as needed for the normal usage. Each defined setup structure must be used in the application code with the Cte_Setup function.

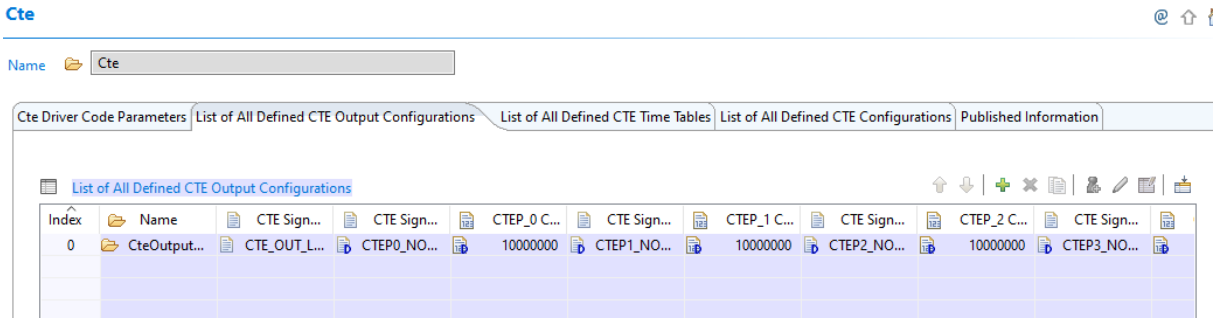


Figure 4.7 : CTE Tresos plugin - CTE Output Configuration List

The image display only one defined setup parameter structure. To edit the structure is recommended to use the detailed setup - see the next subsection - and not this interface.

4.4.1.3 Tresos plugin - Output Setup Detailed

In the following figure all the possible output parameters are shown.

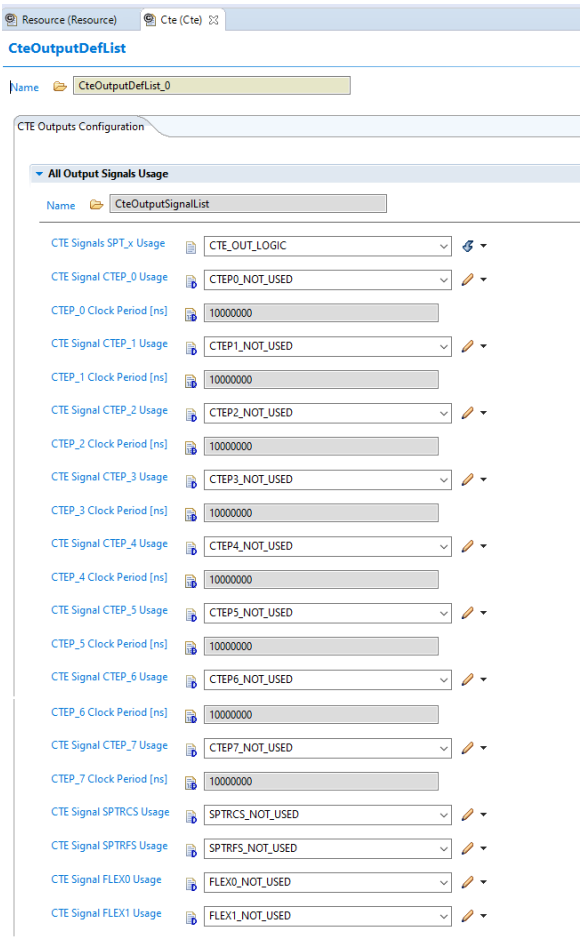


Figure 4.8 : CTE Tresos plugin - CTE Output Detailed Configuration

Only the necessary outputs must be define, but at least one output must be defined:

- **CTE Signals SPT_x Usage** - all four SPT signals must be used in the same way, one of:
  - **SPT_SIGNALS_NOT_USED** = if the signals are not used, the default value
  - **CTE_OUT_LOGIC** = the signals are used as logic output, the logic value of 0 or 1 will be maintained until the next state change will be required in the time table
  - **CTE_OUT_PULSE** = only a short pulse will be generated at output
- **CTE Signal CTEP_x Usage** - each CTEP output signal can be setup individual using one of :
  - **CTEPx_NOT_USED** = the signal is not used, the default value
  - **CTE_OUT_HIZ** = High Impedance input, practically not used, but the acting as a high impedance input
  - **CTE_OUT_TOGGLE** = the output is changing the state after each command, from 0 to 1 and from 1 to 0
  - **CTE_OUT_CLOCK** = the output is generating a clock signal, wit5 50% duty cycle, when activated (set to 1); only in this case the next control is activated
  - **CTE_OUT_LOGIC** = used as logic output, similar to CTE Signals
- **CTEP_x Clock Period [ns]** - the required output frequency if the CTEP_x signal is used as clock output; it is activated only in this case
  - the specified value is the required clock period in nanoseconds; depending the possible internal CTE clock setup, the closest possible frequency is used
- **CTE Signal SPTRCS Usage** - the usage og the generated SPT-Radar Chirp Start is used:
  - **SPTRCS_NOT_USED** = signal not used, the default value
  - **CTE_OUT_LOGIC** = signal used as logic signal, similar to CTE Signals
- **CTE Signal SPTRFS Usage** - the usage og the generated SPT-Radar Frame Start is used:
  - **SPTRFS_NOT_USED** = signal not used, the default value
  - **CTE_OUT_LOGIC** = signal used as logic signal, similar to CTE Signals
- **CTE Signal FLEXx Usage** - the usage of the signal to the FLEX interface
  - **FLEXx_NOT_USED** = the signal is not used, the default value
  - **CTE_OUT_LOGIC** = signal used as logic signal, similar to CTE Signals

4.4.1.4 Tresos plugin - Time Table List

To setup the driver, at least one time table must be defined. According to the necessary usage, can be defined as many time tables as needed for the normal usage. Each defined time table must be used in the application code with the Cte_Setup function. It is possible to combine Time Tables and Outputs as needed, as long as all necessary outputs in the Time Table are set to use in the Output setup. Also, the same Time Table can be used with different Outputs Setup, but the real output signals could be different.

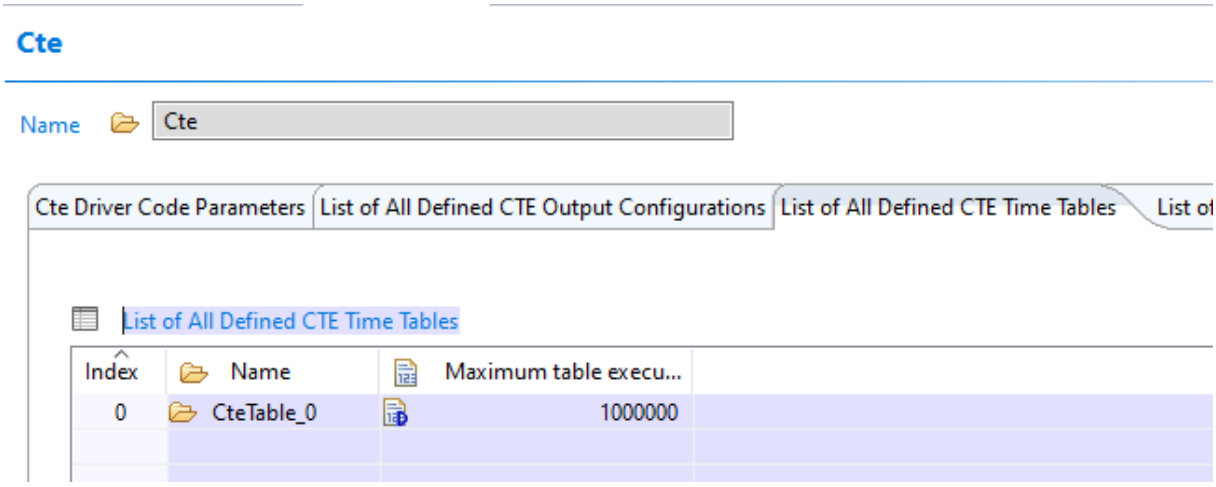


Figure 4.9 : CTE Tresos plugin - CTE Time Tables List

The image display only one defined time table. To edit the structure the detailed setup must be used, as only the execution time limit is available directly in this list.

4.4.1.5 Tresos plugin - Time Table Detailed

The first parameter to be set is the table duration:

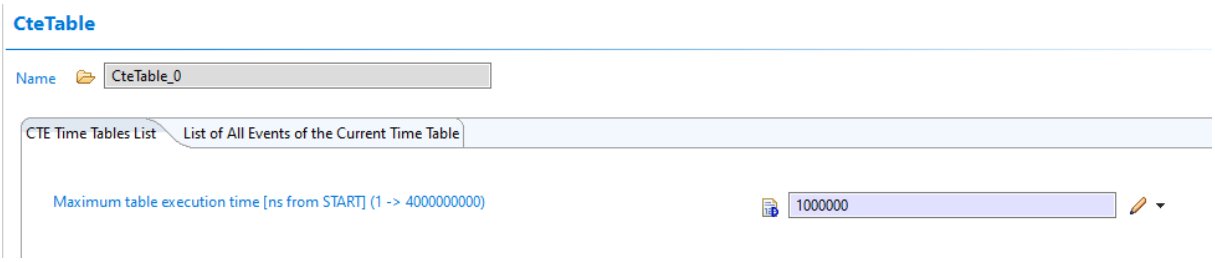


Figure 4.10 : CTE Tresos plugin - CTE Time Table Duration

This is the absolute time from START when the table execution finish. The time value is defined in ns (nanoseconds). The real time execution for a table is the minimum of this value and the last defined event in the events list.
The other parameters necessary to be set are the events associated to the table. For each table event is necessary to define a single events list :

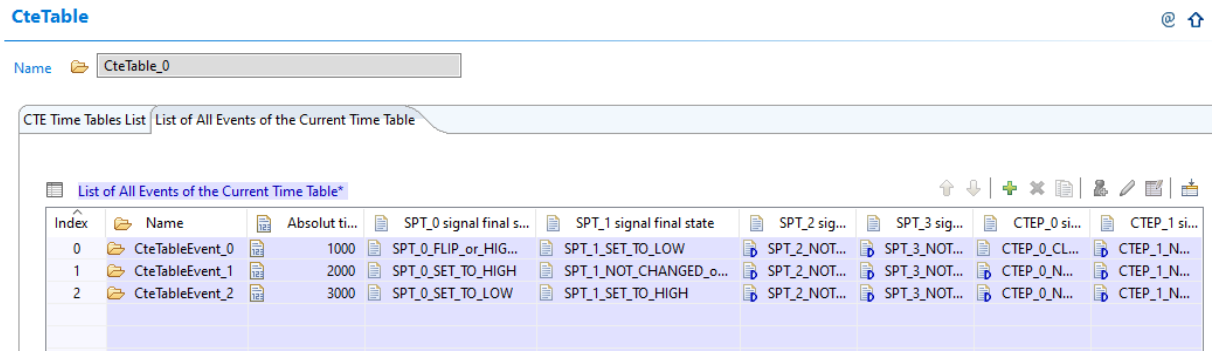


Figure 4.11 : CTE Tresos plugin - CTE Time Table Events List

For each event it must be defined the minimum parameters:

- the event time
- at least an output signal with a defined status; it is recommended to have at least output status change else the event will be "not used" and a table can manage only a limited number or events

A single table can have up to 64 events, but for two tables execution is possible to define up to 32 events for each table.

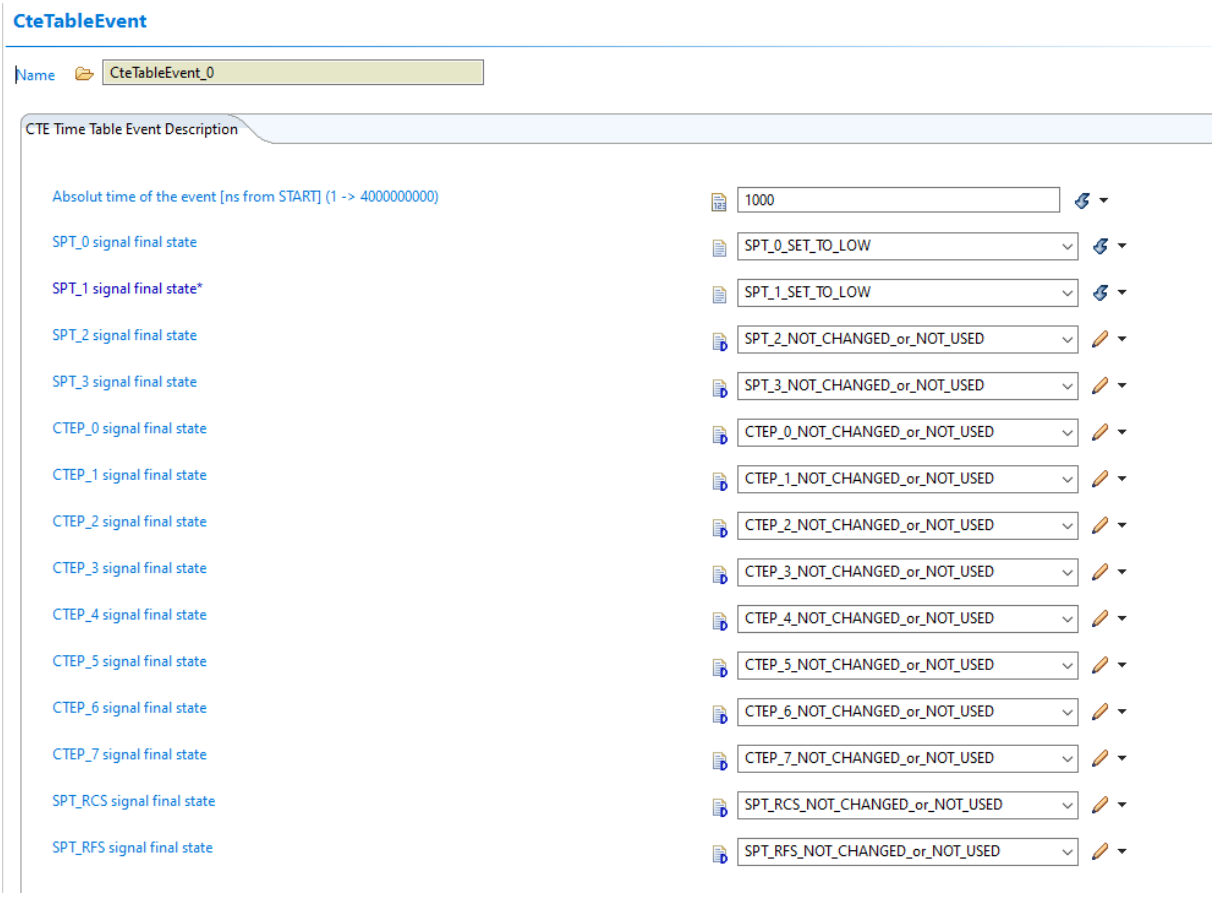


Figure 4.12 : CTE Tresos plugin - CTE Time Table Event Definition

- **Absolut time of the event** - the absolte time of the event, in nanoseconds; the times must be in increasing order else an error will be generated
- **SPT_x signal final state** - the output SPT signal status at the specified time, it can be
  - **SPT_x_NOT_CHANGED_or_NOT_USED** = the status is not changed, the signal keep the previous state
  - **SPT_x_SET_TO_LOW** = the signal is set to low
  - **SPT_x_SET_TO_HIGH** = the signal is set to high
  - **SPT_x_FLIP_or_HIGH_Z** = the signal is set to be changed or to remain high impedance, depending of the output used setup
- **CTEP_x signal final state** - the final state of the CTEP x external signal
  - **CTEP_x_NOT_CHANGED_or_NOT_USED** = signal not changed from the existing state or not used at all
  - **CTEP_x_SET_TO_LOW** = the signal is set to low
  - **CTEP_x_SET_TO_HIGH** = the signal is set to high

  - **CTEP_x_SET_TO_HIGH_or_ACTIVE_SYNC** = the signal is set to high or toggle
  - **CTEP_x_FLIP_or_HIGH_Z_or_CLOCK_ACTIVE** = the signal flip, activate clock output or set to high impedance
  - **CTEP_x_CLOCK_SET_TO_HIGH** = only clock output is set to high
- **SPT_RCS signal final state** or **SPT_RFS signal final state** - the final state for the specified generated signals for SPT
  - **SPT_R.S_NOT_CHANGED_or_NOT_USED** = signal not changed or not used

Note

The specified event times are processed to detect the best suitable dividers combination, the main target is to allow the execution of the last defined event.

For this reson, it is possible to not get the real exact moment but an aproximative one.

Depending the event times is possible to not get a dividers combination and the setup process will return an error.

4.4.1.6 Tresos plugin - Complete Configuration List

This is the setup which connect the other elements defined before :

- the Outputs to be used
- the Events to be used It is possible to define as many configurations as necessary, depending the application context and necessary events and timing to be used. The list of configurations is available in the fourth tab, as shown in the picture:

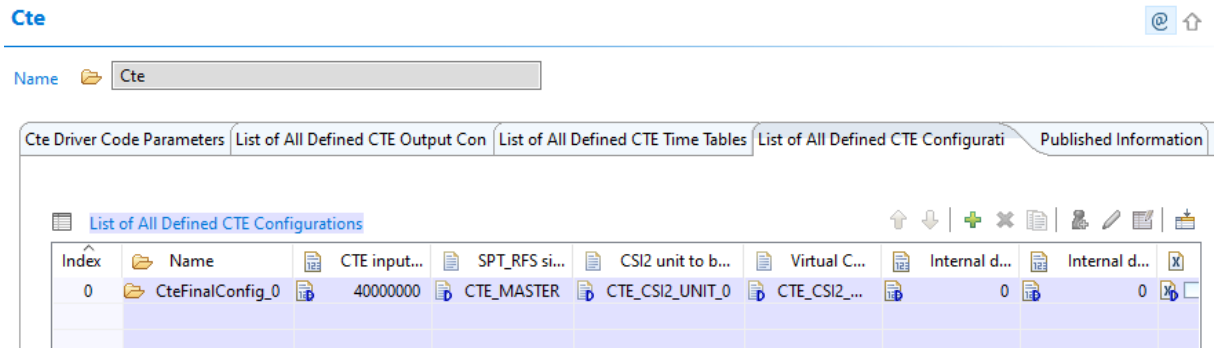


Figure 4.13 : CTE Tresos plugin - CTE Configurations List

There is necessary to add into the list as many configurations items as necessary, but at least one. After the definition, it is necessary to set the correct parameters for each element.

4.4.1.7 Tresos plugin - Complete Configuration Detailed

To put together the defined Outputs and Events, some other parameters must be defined too. The complete list of the necessary parameters can be found in figure 14.

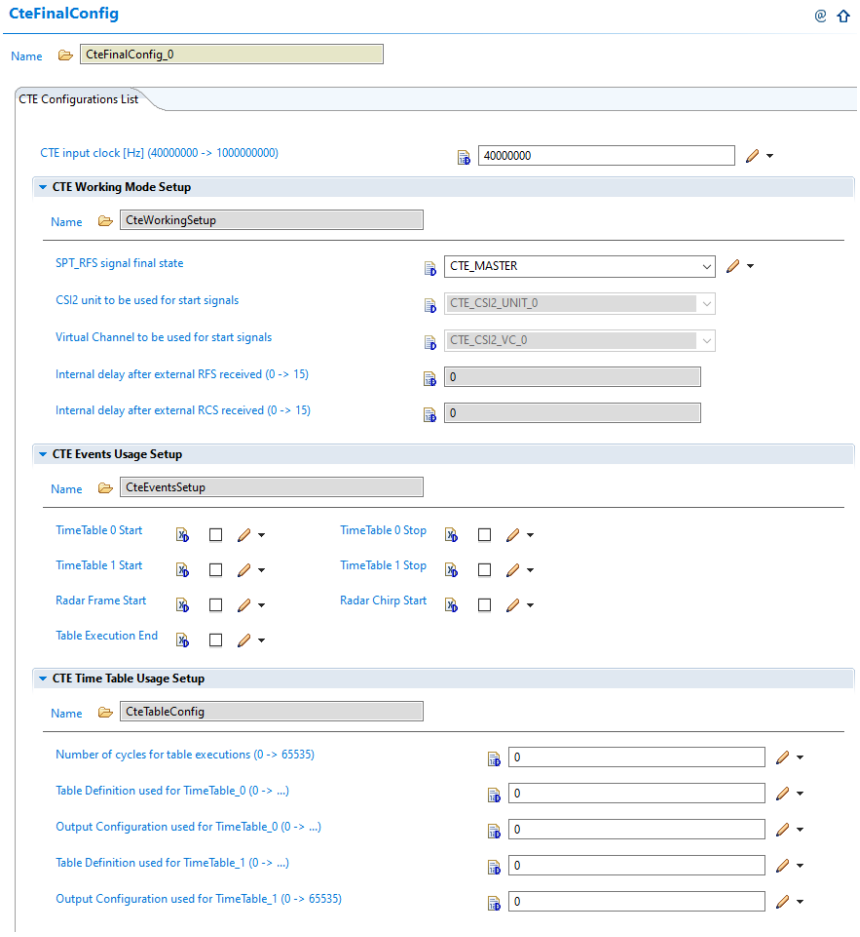


Figure 4.14 : CTE Tresos plugin - CTE Configurations Detailed

- **CTE input clock [Hz]** - this is the CTE working clock (**CTE_CLK**, as defined in the Reference Manual for the SoC), as resulted after the platform is setup
- **CTE Working Mode Setup** - the necessary parameters to define the CTE working mode :
  - **SPT_RFS signal** = select the trigger input signal
    - **CTE_MASTER** = CTE is only software commanded, Cte_Start starts the table execution too
    - **CTE_SLAVE_EXTERNAL** = CTE triggers are done by the external RFS/RCS signals; in this case, two more elements can be used :
      - **Internal delay after external RFS received** = the internal delay between the signal reception and the real start, this is specified in CTE_CLK periods and is only up to 15
      - **Internal delay after external RCS received** = the internal delay between the signal reception and the real start, this is specified in CTE_CLK periods and is only up to 15
    - **CTE_SLAVE_CSI2** = the triggers are generated by the CSI2 data reception; it is possible to specify/use only a single root of the signal :
      - **CSI2 unit to be used for start signals** = the CSI2 unit to be the source of signals, it must be one of :
        CTE_CSI2_UNIT_0
        CTE_CSI2_UNIT_1

  - **Virtual Channel to be used for start signals** - the Virtual Channel reception to trigger the time table execution, one of :
    CTE_CSI2_VC_0
    CTE_CSI2_VC_1
    CTE_CSI2_VC_2
    CTE_CSI2_VC_3

- **CTE Events Usage Setup** - the events required to be reported to the application, using the callback
  - **TimeTable 0 Start** = Time Table 0 start to be reported
  - **TimeTable 0 Stop** = Time Table 0 stop to be reported
  - **TimeTable 1 Start** = Time Table 1 start to be reported
  - **TimeTable 1 Stop** = Time Table 1 stop to be reported
  - **Radar Frame Start** = Radar Frame Start signal received
  - **Radar Chirp Start** = Radar Chirp Start signal received
  - **Table Execution End** = Table Execution stopped and CTE entering HALT mode
- **CTE Time Table Usage Setup** - the necessary combination between Time Tables and Outputs
  - **Number of cycles for table executions** = the number of times the tables are executed, 0 means "continuously", 1...65535 are the other possible values
  - **Table Definition used for TimeTable_0** = the position of the Time Table List to be used as Time Table 0, the ending number in the identifier, for CteTable_x x must be used
  - **Output Configuration used for TimeTable_0** = the Output configuration position to be used for TT_0 execution, the ending number in identifier to be uesed, as above
  - **Table Definition used for TimeTable_1** = the position of the Time Table List to be used as Time Table 1
  - **Output Configuration used for TimeTable_1** = the Output configuration position to be used for TT_1 execution

4.4.2 S32DS Plugin

CTE plugin for Design Studio offers the same functionality as the Tresos plugin, but adapted to the Eclipse environment.
At least one complete set of parameters must be defined, but is possible to define as many structures as necessary.

4.4.2.1 1. Adding the component to the Peripherals

The component is not added by default, therefore:

- click on **ADD** button for Drivers
- select **CTE** component (see figure below)
- click **OK** button

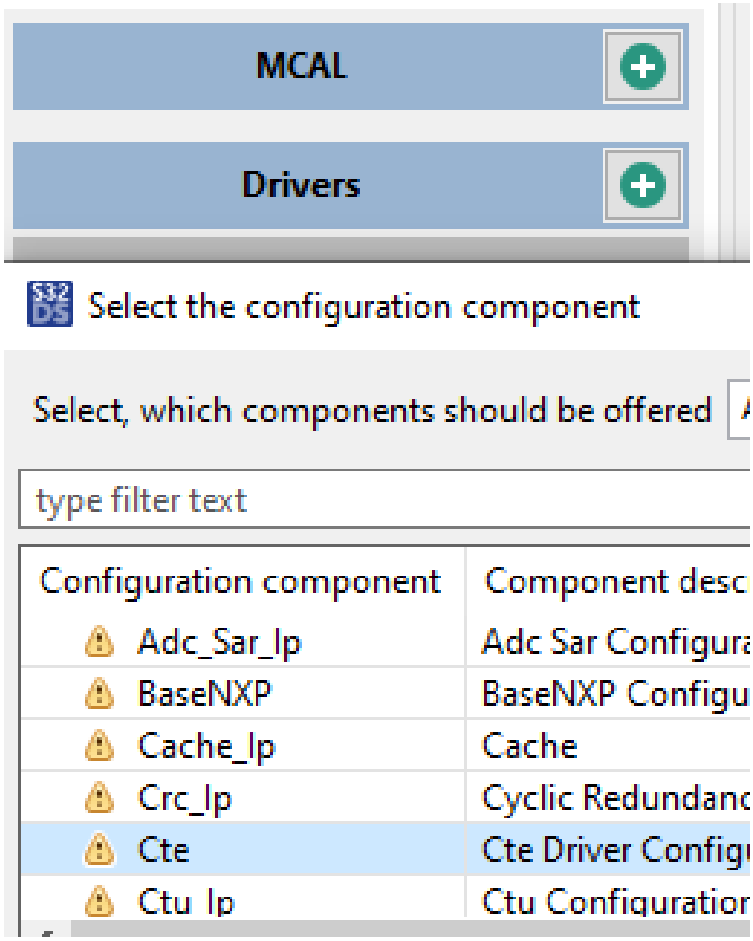


Figure 4.15 : CTE S32DS Plugin - Adding the component to the Peripherals

4.4.2.2 2. Configuring the PreCompile parameters

The parameters from Figure 16 work as follows (hovering over each parameter also shows details related to it):

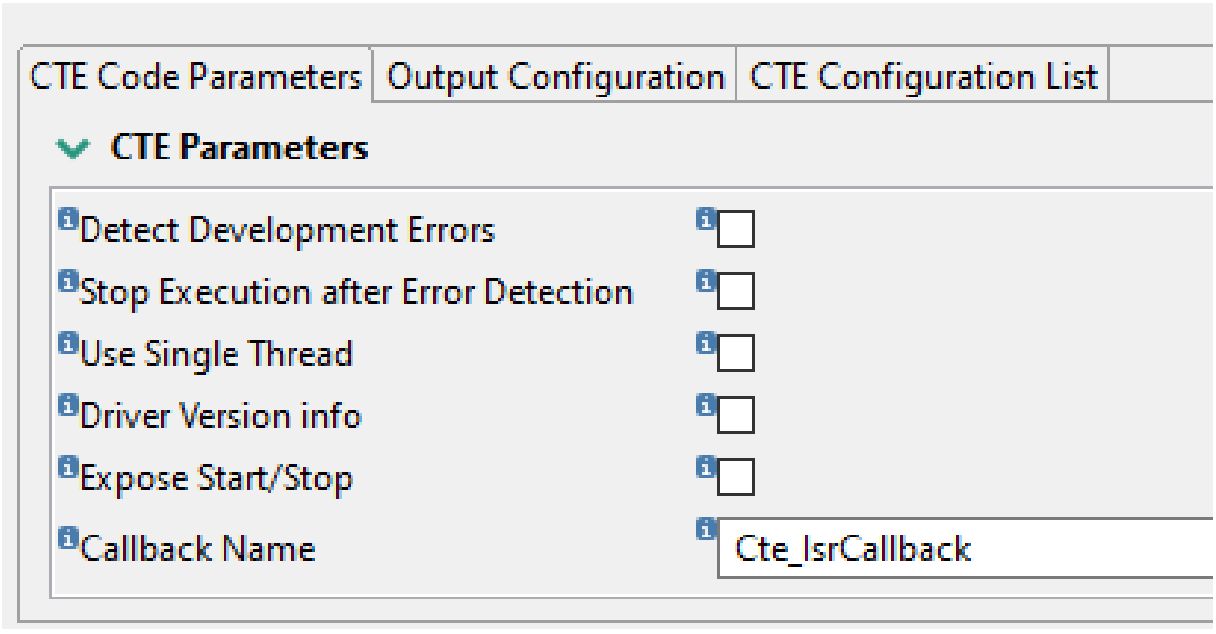


Figure 4.16 : CTE S32DS Plugin - PreCompile Parameters

- **Detect Development Errors**
  - **TRUE** = Development error detection enabled
  - **FALSE** = Development error detection disabled
- **Stop Execution after Error Detection**
  - **TRUE** = Stops execution after the error report.
  - **FALSE** = Does not stop execution after the error report.
- **Use Single Thread**
  - **TRUE** = Single threaded.
  - **FALSE** = Multi threaded.
- **Driver Version info**
  - **TRUE** = VersionInfo function exposed.
  - **FALSE** = VersionInfo function not exposed.
- **xpose Start/Stop**
  - **TRUE** = Start/Stop functions exposed.
  - **FALSE** = Start/Stop functions not exposed.
- **Callback Name**
  - This is the name of the driver callback.
    It is used for events reporting. The call is done from the CTE events interrupt handler.
    To avoid any issues, this callback must be implemented.

4.4.2.3 3. Configuring the CTE Outputs

The parameters from Figure 17 have a ierarhical structure, each "structure" defined by a number next to it. Below, there are details on how they interact between each other, but as an example: whichever output signal (number 1 in the image below) is set to a different value than "*componentName*_NOT_USED", it will influence what is allowed to choose in "Output Type" drop-down menu from "Action List" (number 4 in the image below).

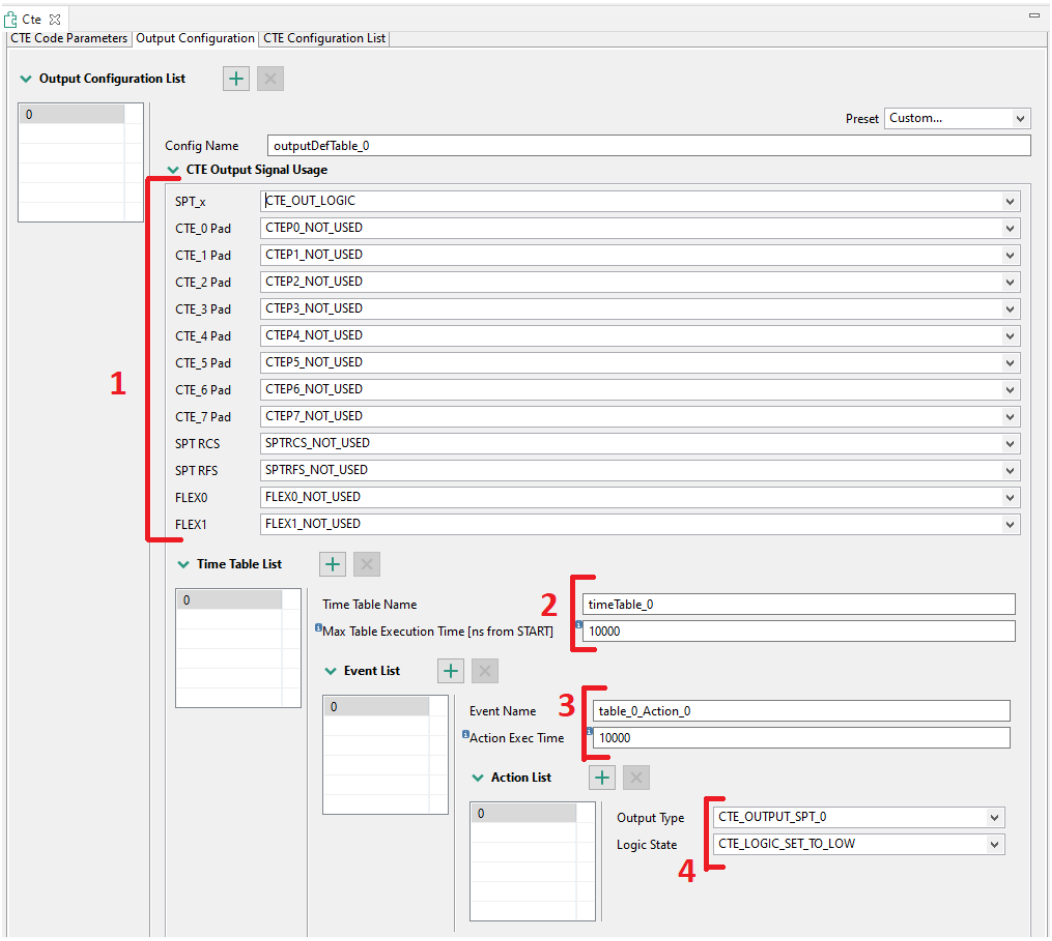


Figure 4.17 : CTE S32DS Plugin - Output Parameters

- **CTE Output Signal Usage** - marked by **number 1 in Figure 17**, defines which output signals will be available for changing and also selects the type of change to occur (Toggle, Logic, Clock - depending on the output). Multiple outputs can be selected.
- **Time Table List** - marked by **number 2 in Figure 17**. Each time table can hold up to 32 Events (or up to 64 Events if a single Time Table is used). Here you can set a maximum absolute time from START (in nanoseconds), when the time table execution will finish. The real execution time will be the smallest between this time, and the last defined Event in the "Event List" (number 3 in Figure 17). You can create as many Time Tables as desired, but which Time Table will be used will depend on which one is selected in "CTE Configuration List" tab -> "Time Table Selection x" drop-down menu (see Figure 18)
- **Event List** - marked by **number 3 in Figure 17** will hold all the Actions defined in "Action List". Here you can define at which time from START (in nanoseconds) should the event occur.
- **Action List** - marked by **number 4 in Figure 17** defines which output should change and to which state. Selections chosen here should match the selections chosen in "CTE Output Signal Usage" (number 1 in Figure 17), otherwise it will cause an error and configuration cannot be generated.

- **Output Signal Usage selections:**
  - **SPT_x Usage** - all four SPT signals must be used in the same way, one of:
    - **SPT_SIGNALS_NOT_USED** = the signals are not used
    - **CTE_OUT_LOGIC** = the signals are used as logic output, the logic value of 0 or 1 will be maintained until the next state change will be required in the time table
    - **CTE_OUT_PULSE** = only a short pulse will be generated at output

- **CTE_x Pad Signal Usage** - each CTE Pad output signal can be setup individual using one of :
  - **CTEPx_NOT_USED** = the signal is not used, the default value
  - **CTE_OUT_HIZ** = High Impedance input, practically not used, but the acting as a high impedance input
  - **CTE_OUT_TOGGLE** = the output is changing the state after each command, from 0 to 1 and from 1 to 0
  - **CTE_OUT_CLOCK** = the output is generating a clock signal, with 50% duty cycle, when activated (set to 1); only in this case the next control is activated
  - **CTE_OUT_LOGIC** = used as logic output, similar to CTE Signals
- **CTEP_x Clock Period [ns]** - the required output frequency if the CTEP_x signal is used as clock output; it is activated only in this case
  - the specified value is the required clock period in nanoseconds; depending the possible internal CTE clock setup, the closest possible frequency is used
- **SPTRCS Signal Usage** - the usage of the generated SPT-Radar Chirp Start is used:
  - **SPTRCS_NOT_USED** = signal not used, the default value
  - **CTE_OUT_LOGIC** = signal used as logic signal, similar to CTE Signals
- **SPTRFS Signal Usage** - the usage of the generated SPT-Radar Frame Start is used:
  - **SPTRFS_NOT_USED** = signal not used, the default value
  - **CTE_OUT_LOGIC** = signal used as logic signal, similar to CTE Signals
- **CTE Signal FLEXx Usage** - the usage of the signal to the FLEX interface
  - **FLEXx_NOT_USED** = the signal is not used, the default value
  - **CTE_OUT_LOGIC** = signal used as logic signal, similar to CTE Signals

4.4.2.4 4. Completing the configuration

The final steps of the CTE configuration are as follows:

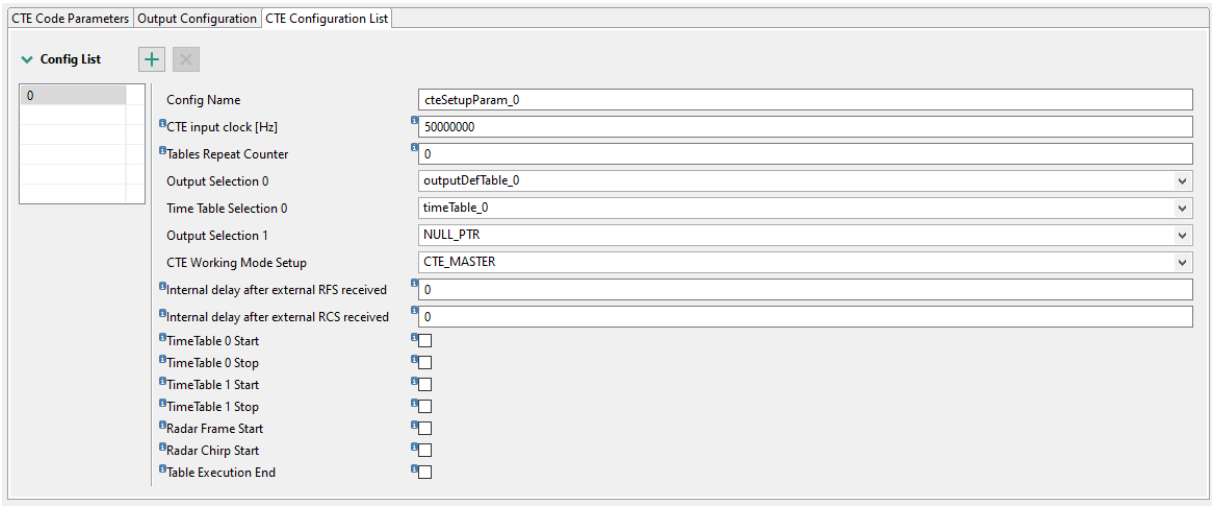


Figure 4.18 : CTE S32DS Plugin - Finalizing Configuration

- **CTE input clock [Hz]** - this is the CTE working clock (**CTE_CLK**, as defined in the Reference Manual for the SoC), as resulted after the platform is setup
- **Tables Repeat Counter** - The number of times the table execution will be repeated until going to HALT state. 0 (zero) means "forever". If two alternative tables used, each table execution will decrease the counter.

- **Output Selection 0** - Selects which Output Configuration to be used. The selections come from "Output Configuration List" (see Figure 17)
- **Time Table Selection 0** - The selections available will depend on how many "Time Tables" are defined for the "Output Selection" from above
- **Output Selection 1** - This is similar to "Output Selection 0". If there is only 1 element defined in "Output Configuration List" (see Figure 17), this should be set to "NULL_PTR". Otherwise, one of the other output configurations can be selected (this will cause a "Time Table Selection 1" to also appear).
- **CTE Working Mode Setup** - the necessary parameters to define the CTE working mode :
  - **CTE_MASTER** = CTE is only software commanded, Cte_Start starts the table execution too
  - **CTE_SLAVE_EXTERNAL** = CTE triggers are done by the external RFS/RCS signals; in this case, two more elements can be used :
    - **Internal delay after external RFS received** = the internal delay between the signal reception and the real start, this is specified in CTE_CLK periods and is only up to 15
    - **Internal delay after external RCS received** = the internal delay between the signal reception and the real start, this is specified in CTE_CLK periods and is only up to 15
  - **CTE_SLAVE_CSI2** = the triggers are generated by the CSI2 data reception; it is possible to specify/use only a single root of the signal :
    - **Virtual Channel to be used for start signals** - the Virtual Channel reception to trigger the time table execution, one of :
      CTE_CSI2_VC_0
      CTE_CSI2_VC_1
      CTE_CSI2_VC_2
      CTE_CSI2_VC_3
- **CTE Events Usage Setup** - the events required to be reported to the application, using the callback
  - **TimeTable 0 Start** = Time Table 0 start to be reported
  - **TimeTable 0 Stop** = Time Table 0 stop to be reported
  - **TimeTable 1 Start** = Time Table 1 start to be reported
  - **TimeTable 1 Stop** = Time Table 1 stop to be reported
  - **Radar Frame Start** = Radar Frame Start signal received
  - **Radar Chirp Start** = Radar Chirp Start signal received
  - **Table Execution End** = Table Execution stopped and CTE entering HALT mode

Chapter 5

SPT User Guide

5.1 Introduction

This section describes the SPT Driver and SPT Kernels software components.

It provides details about their functionality, operating modes, directory structure and the steps needed to build the binaries. It also contains guidelines and examples for easy integration. They are described together in a single chapter, due to their tight functional coupling.

5.2 Module description

The SPT module consists of the SPT Driver and SPT Kernels software components.

The SPT Driver serves as an interface between the user application running on the host CPU cores and the SPT hardware. Its purpose is to enable the integration and execution of low-level microcode kernels on the SPT (Signal Processing Toolbox) accelerator for baseband radar signal processing.

The SPT Kernels are individual pre-compiled routines of SPT code packed into a library, each implementing a part of the radar processing flow.

Together, the SPT Driver and SPT Kernels library provide a software abstraction of the built-in SPT hardware functions (e.g. FFT, Maximum Search, Histogram etc) combined in a series of algorithms.

The SPT Driver, kernels and the DSP executable image can be integrated in user applications running on the S32R41 M7 core in an AUTOSAR environment (using the dedicated NXP RTOS).

The following figure shows a high-level deployment diagram. The rsdk_sampleapps demonstrate how to integrate the all the software components into the user code.

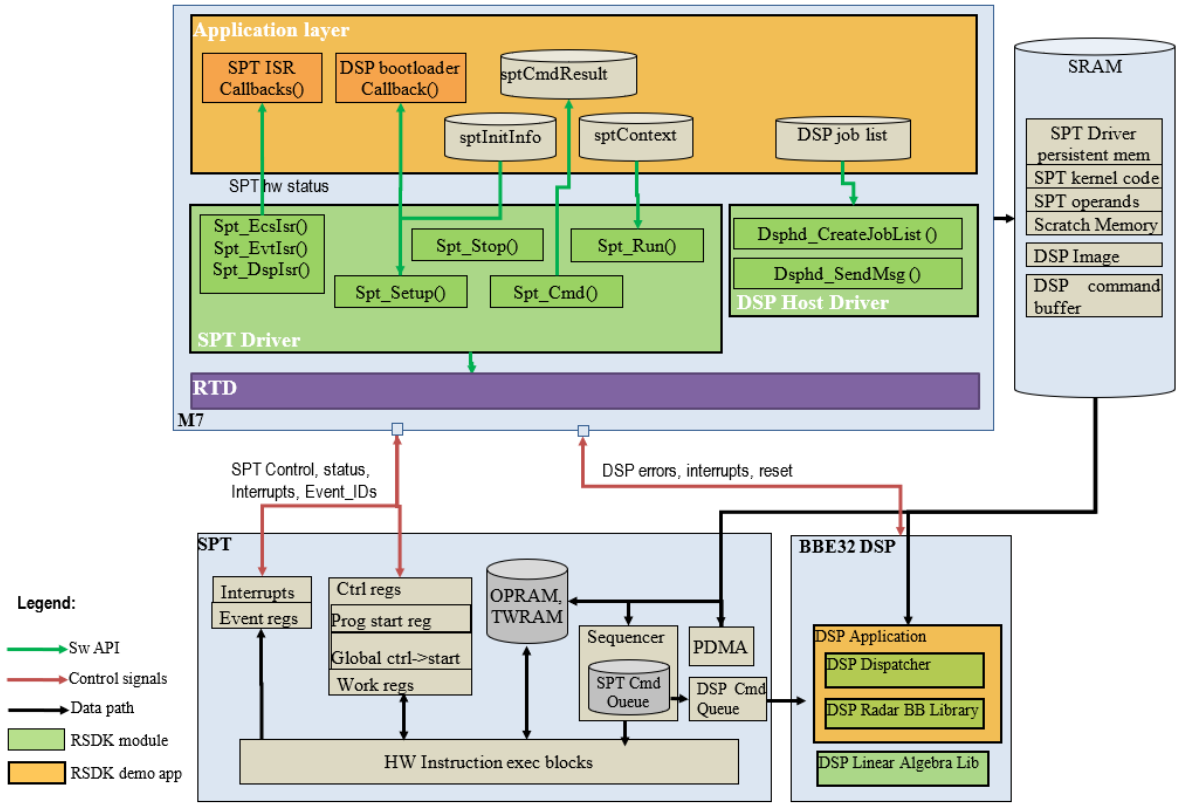


Figure 5.1 SPT & DSP software architecture

5.3 Build and Execution

5.3.1 Environment

The tools and dependencies used for building and executing the RSDK modules are listed in RSDK Environment Dependencies section.

5.3.2 Building the SPT Driver

The SPT Driver is not built into a separate library, but integrated directly as source code in top-level applications deployed on the M7 core. When building an application for SAF85XX, there are a few conditions for ensuring the proper functionality of the SPT driver:

- All SPT CDD source files must be added and built along with the application
- The SPT driver's interrupt handlers must be registered accordingly by at application level (Interrupt IDs)
- All files generated through a Tresos SPT configuration must be also made available for the application (Using the SPT Tresos plugin or Using the SPT S32DS plugin)
- SPT driver's callbacks must be implemented by the application (callbacks' names are fixed - Interrupt callbacks)

The build options used for this software component are provided in the Build configuration paragraph.

code is provided in rsdk_multi_ex.

5.3.2.1 Using the SPT Tresos plugin

The SPT Tresos plugin is provided in order to allow SPT driver's configuration at application level.

The first step is to add the SPT plugin into a Tresos configuration project. For this: right click on the project -> "Module Configurations..." -> select SPT from the list:

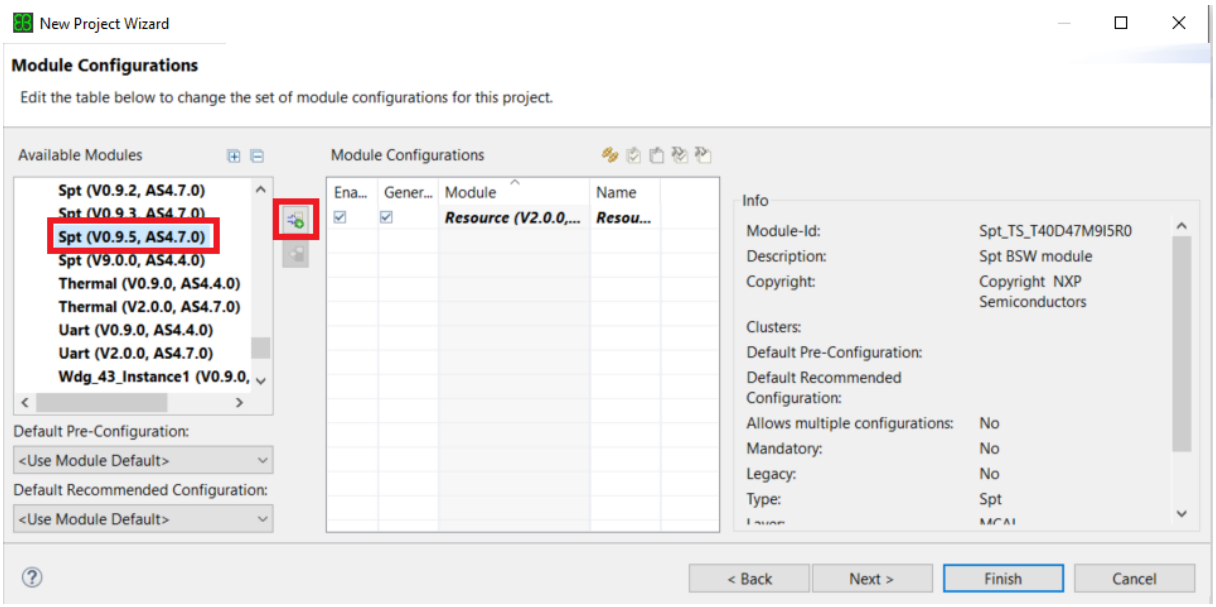


Figure 5.2 Adding the SPT plugin to a Tresos project

After the plugin is imported, the SPT configuration window can be opened in order to make necessary changes.

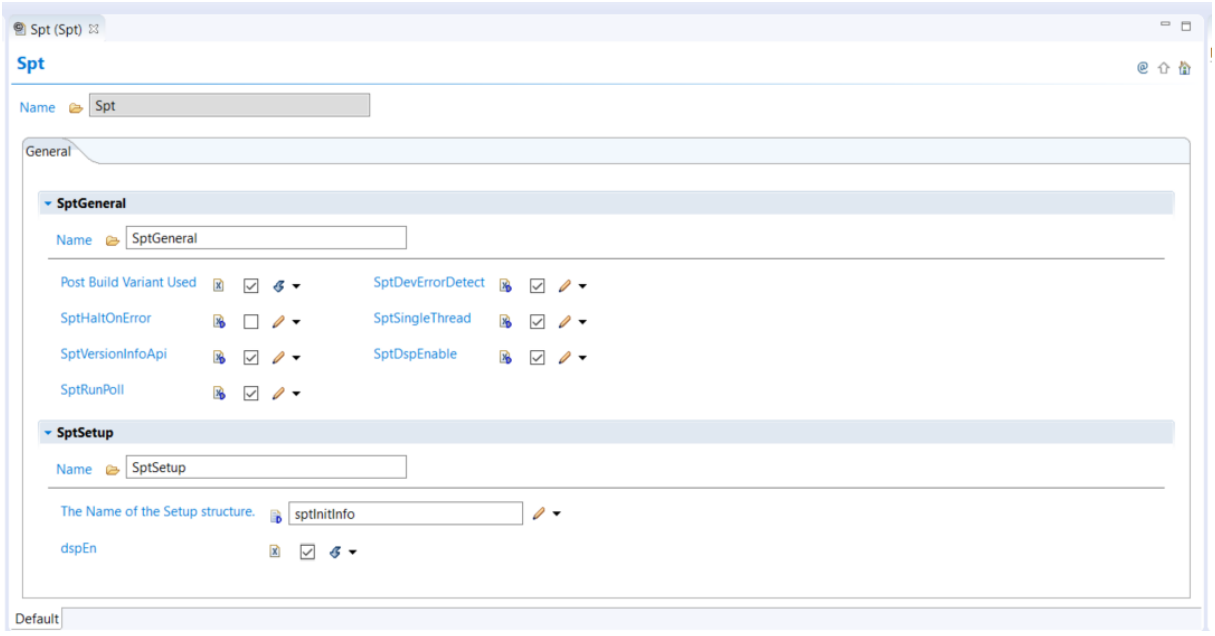


Figure 5.3 SPT driver's configuration options

The options under the "SptGeneral" container represents the PreCompile **(PC)** options available, while the "Spt↩ Setup" one corresponds to the PostBuild **(PB)** configuration.

Note

The "SptRunPoll" options does not control the way the SPT driver runs (polling vs interrupt based). This options is specified in Spt_DriverContextType::opMode (details in Running the SPT) and, as a conseqence, can be different for different SPT kernels. The "SptRunPoll" plugin option allows to remove the polling support from the SPT code ONLY if the operating mode passed to the all Spt_DriverContextType structures is with interrupts.

See SPT Driver Autosar Configuration Parameters for a full description of the Tresos configuration parameters, extracted from the corresponding ∗.epd file.

After the configuration is set, right click on the SPT project entry and press "Generate Configuration" in order to generate the files required by the application.

Note

RSDK provides a configuration project example along with rsdk_multi_ex.

5.3.2.2 Using the SPT S32DS plugin

The SPT S32DS plugin is also provided as another alternative to allow SPT driver's configuration at application level directly from S32 Design Studio.

The first step is to add SPT into the list of peripherals in an S32DS configuration project. For this: open the project in the peripherals perspective -> click on the "+" icon -> select SPT from the list:

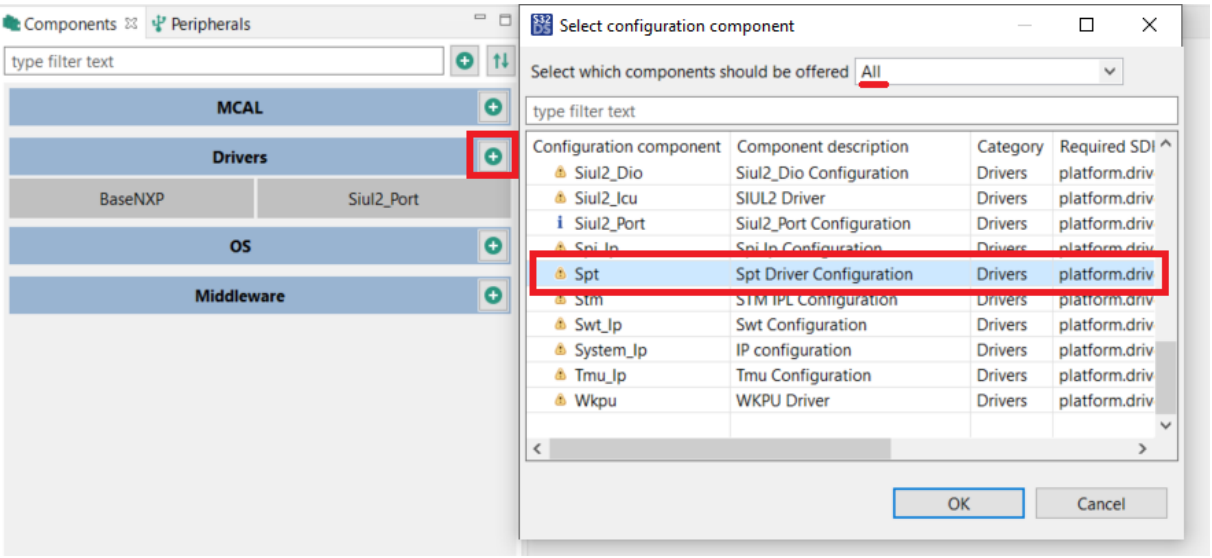


Figure 5.4 Adding the SPT plugin to an S32DS project

After SPT has been added, the configuration window can be opened in order to make necessary changes.

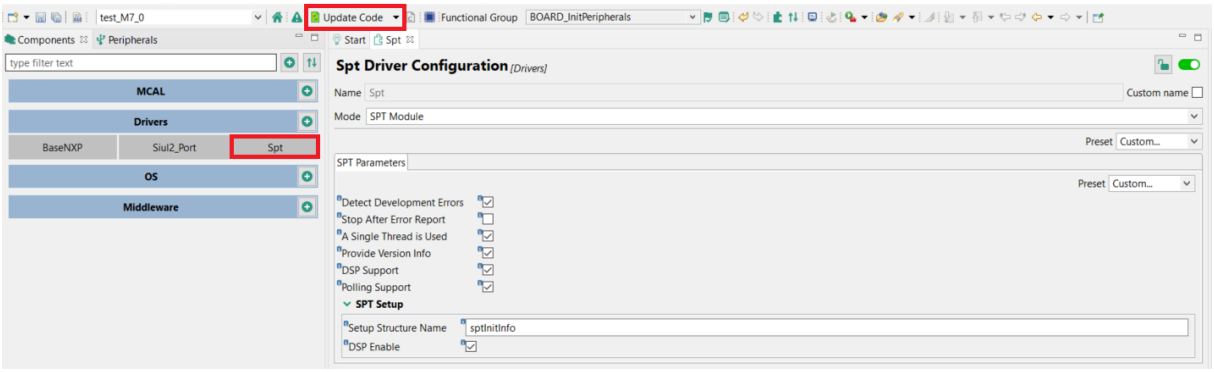


Figure 5.5 SPT driver's configuration options

After the configuration is set, click on "Update Code" in order to generate the files required by the application.

Note

All parameters are the same as for the Tresos plugin. This means that all notes under Using the SPT Tresos plugin also apply to the **S32DS** SPT plugin and, also, a more detailed description for all available options can be found under SPT Driver Autosar Configuration Parameters.

5.3.3 Building the SPT Kernels library

The SPT Kernels are packed in two libraries:

- **librsdk_SPT_kernels_offline_gcc_S32R41.a** : Version which is used in the offline mode of RSDK Offline Example , allowing execution on the S32R41 without a radar frontend
- **librsdk_SPT_kernels_gcc_S32R41.a** : Standard version, used in rsdk_multi_ex, running with a connected radar front-end

To build the SPT kernels, open DesignStudio, import the .project file located at < rsdk_path >/SPT/SPT_↩ kernels/project/< hw_arch >, then press 'Project -> Build All'. Alternately, each target can be selected and built individually (Project -> Build Configurations -> Set Active, then Project -> Build Project). After the build process is finished, the libraries will be located in < rsdk_path >/SPT/SPT_kernels/bin.

You may also use the make command to build the library - from msys or Ubuntu shell on Windows systems, because the build process uses executables from the DesignStudio Windows installation folder. The makefile is located in < rsdk_path >/SPT/SPT_kernels. It uses some information defined in < rsdk_path >/compilerdefs.mak, make sure you set all the variables properly on your system before building (see compilerdefs.mak file header for details).

The make command line is:

```
< rsdk_path >/SPT/SPT_kernels$ make all PLATFORM=S32R41 OSENV=sa
```

5.4 SPT Operation manual

This section describes the proper usage of the SPT_Driver, which includes initialization, control and runtime processing.

5.4.1 Program Flow

The general program flow of the SPT Driver and Kernels is shown below:

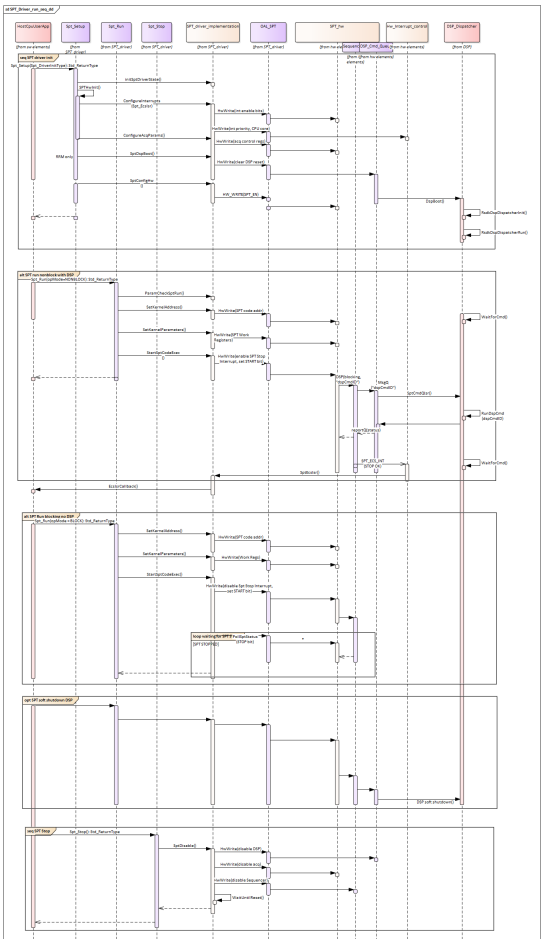


Figure 5.6 SPT execution sequence

5.4.1.1 Initialization

Initialization of the SPT Driver and BBE32 boot-up are done by calling Spt_Setup(). The initialization stage sets the internal state of the SPT Driver as well as the SPT hardware registers. The SPT Driver initialization stage is the first operation that needs to happen after system boot or after a change in the Radar front-end acquisition parameters, before the SPT can be called to perform any data processing. It is also recommended to re-initialize the SPT every time after an SPT hardware error occurs.

5.4.1.2 Booting the BBE32:

At this stage the user has an API option to bring the **BBE32 DSP** out of reset and allow it to boot from the DSP's own Instruction RAM. For this purpose the driver calls the Spt_DspBootloaderCb (which must be provided by the user) to load the BBE32 image into memory.

Memory must be allocated by the user before copying the image. RSDK provides additional tools to create the BBE32 image automatically when building the executable. See Building the DSP image for details.

The role of the Spt_DspBootloaderCb callback is to place all code and data sections of the BBE32 image at their proper memory addresses. Its implementation is tightly coupled with the BBE32 image format and memory map, since the code and data sections must be copied in memory at the exact locations specified in the linker file of the DSP executable.

After booting the DSP will start running its own program, which is expected to call RsdkDspDispatcherInit() and then go into RsdkDspDispatcherRun() where it will wait indefinitely for incoming commands (through the SPT command queue) and execute them. Before calling Spt_Setup() , the user must check if the preconditions are met and set the necessary control parameters:

- allocate and initialize one instance of Spt_DriverInitType (can be generated using the SPT Tresos plugin - PostBuild configuration)
- if the DSP is enabled, then the DSP image must also be included as a source or object file in the in the host application build.

Example integration code:

```
  Spt_DriverInitType     sptInitInfo;
  Std_ReturnType         sptStatus = RSDK_SUCCESS;
  sptInitInfo.hwPlatSpec.dspEn = TRUE;

/* Spt_DspBootloaderCb() and Spt_DspIsrCb() callbacks must be implemented because are used by Spt_Setup() */
  sptStatus = Spt_Setup(&sptInitInfo);
  statusCheck("Spt_Setup", stdStatus);
```

5.4.1.3 Running the SPT

The user can execute a certain SPT Kernel by passing the Spt_DriverContextType::kernelCodeAddr and the kernel parameters to Spt_Run().

The SPT Driver itself does not provide any service for synchronization with the chirp and frame acquisition signals (e.g. for chirp-by-chirp range processing). In case the execution of the SPT kernel needs to be synchronized, this can be done either inside the SPT Kernel code (using WAIT instructions) The user is responsible to detect if data is not received from the Radar front-end. Usually this is done by guarding the SPT synchronous kernel execution with a system-level watchdog or by monitoring the frame acquisition signals generated by MIPI-CSI2.

Before calling Spt_Run() , the caller must check if the preconditions are met and set the necessary control parameters:

- all SPT kernels used via the SPT Driver must be included in the linking phase of application build. Mind that SPT driver relies on definition of #RSDK_SPT_CUBE_BASE_ADDR and #RSDK_SPT_OTHER_BASE_↩ ADDR symbols which must point to App SRAM memory. They are used to integrate RSDK SPT Driver and Kernels, by setting a common base for host CPU and SPT PDMA accesses to SRAM. They can be defined in the linker file used to build the application.
- allocate one global 32-bit integer to hold the SPT kernel return values and pass it to Spt_DriverContext↩ Type::kernelRetPar. If no return parameter is expected from the SPT kernel, then it can be initialized to NULL (0).
- allocate and initialize one instance of Spt_DriverContextType

The SPT operation can be considered complete when either the Spt_Run() returns from a blocking call, or the Spt_EcsIsrCb is called during an interrupt after a non-blocking call to Spt_Run().

The side-effects of the Spt_Run() depend on the operating mode) Spt_DriverContextType::opMode). The difference between the 2 modes comes from the actions taken after the SPT kernel processing is started, as shown in Figure 3:

- **blocking mode**: After triggering the SPT kernel execution, Spt_Run() waits in a polling loop until:
  - either SPT_CS_STATUS0[PS_STOP] bit is set in hardware,
  - or the internal timeout expires,
  - or the SPT hardware signals an error during kernel execution, in one of the dedicated error registers.

- **non-blocking mode**: After triggering the SPT kernel execution, Spt_Run() exits immediately to allow other processing on the CPU core. SPT kernel completion is signalled by the Spt_EcsIsrCb , which is called during the SPT ECS interrupt triggered by the SPT_CS_STATUS0[PS_STOP] bit. While the SPT is busy processing a kernel, Spt_Run() prevents other SPT kernels from being launched until the SPT hardware finishes the current one, by returning #SPT_E_WARN_HW_BUSY status to the caller to announce that it is 'busy'.

5.4.1.4 Synchronizing with chirp data acquisition

S32R Radar applications usually configure the input data acquisition buffer to contain only a small number of chirps (less than 1 frame), due to memory and speed constraints. This buffer is filled-in by the CSI2 in a circular manner and it is consumed by the SPT range FFT processing. The SPT needs to be notified every time a new chirp is loaded in the buffer, such that it can process the data before it gets overwritten. The CSI2 and CTE modules can send such signals to the SPT, which can synchronize on them using the "WAIT" instruction. The CPU is not involved in the synchronization procedure.

Note

The RSDK SPT Range FFT kernels are hardcoded to use the CSI2 synchronization signals and an input buffer size to contain 2 chirps' worth of data. Each SPT kernel and thread can synchronize with a different CSI2 unit and Virtual Channel, depending on the encoding of the "WAIT" instruction, which is set at assembly time. The corresponding CSI2 units must be configured to send the proper signals to the SPT, through the csi2 (see Csi2_VCParamsType::bufNumLinesTrigger).

Example code:

```
//--- Init SPT runtime context for Range & Doppler Initialization Kernel
    ------------------------------------------------
memset(&sptContext, 0, sizeof(sptContext));

sptContext.kernelCodeAddr = (uintptr_t)gSptModuleCodeRelocBufH.phyAddr;

sptContext.opMode = SPT_OP_MODE;
sptContext.kernelRetPar = NULL;  //this kernel has no return value
sptContext.checkKernelWatermark = CHECK_KERNEL_WATERMARK;

//First SPT Kernel to be run is the initialization of Range & Doppler
//FFTs for the 128chirps x 512 samples use case
RelocSptCode(gSptModuleCodeRelocBufH.virtAddr, RsdkSptInit512smp128crp4ch,
             RSDK_SPT_GET_KERNEL_SIZE(RsdkSptInit512smp128crp4ch));

//SPT parameter list:
sptContext.kernelParList[np].paramType = SPT_PARAM_TYPE_ADDR;
sptContext.kernelParList[np++].paramValue = (uintptr_t)(gFft512TwiddleFactorsBufH.phyAddr);

sptContext.kernelParList[np].paramType = SPT_PARAM_TYPE_ADDR;
sptContext.kernelParList[np++].paramValue = (uintptr_t)gFft128TwiddleFactorsBufH.phyAddr;

sptContext.kernelParList[np].paramType = SPT_PARAM_TYPE_ADDR;
sptContext.kernelParList[np++].paramValue = (uintptr_t)gFft512BlackmanWindowBufH.phyAddr;

sptContext.kernelParList[np].paramType = SPT_PARAM_TYPE_ADDR;
sptContext.kernelParList[np++].paramValue = (uintptr_t)gFft128BlackmanWindowBufH.phyAddr;

sptContext.kernelParList[np].paramType = SPT_PARAM_TYPE_ADDR;
sptContext.kernelParList[np++].paramValue = (uintptr_t)gFft16TwiddleFactorsBufH.phyAddr;

sptContext.kernelParList[np].paramType = SPT_PARAM_TYPE_ADDR;
sptContext.kernelParList[np++].paramValue = (uintptr_t)gDbfSteeringVectorsBufH.phyAddr;

sptContext.kernelParList[np].paramType = SPT_PARAM_TYPE_LAST;

//Driver call to SPT - load twiddles and window coefficients
//Spt_EcsIsrCb() and Spt_EvtIsrCb() should also be implemented by the application
sptStatus = Spt_Run(sptContext);
statusCheck("Spt_Run", sptStatus);

//Wait for init to be done
WaitForSPT(sptContext.opMode);
```

5.4.1.5 Interrupt handling

The SPT Driver uses the following interrupt event lines coming from the SPT through the interrupt controller:

- SPT_IRQ_ECS - the general interrupt line which includes the command sequencer status interrupts (e.g. "SPT done" signal) as well as all the hardware error and overflow interrupts

- SPT_IRQ_EVT - for signalling SPT events generated using the "EVT" instruction, towards the CPU
- SPT_IRQ_DSP - for signalling events and errors generated by the BBE32 towards the CPU

The application must register SPT's interrupts. The interrupt line numbers and the interrupt handlers are provided in the SPT driver.

Moreover, the application must also implement a set of callbacks for each interrupt line described above (callbacks' declarations are provided through the SPT plugin configuration).

Note

An example integration of the SPT control and data flow is provided in the source code of the RSDK Offline Example app.

5.4.1.6 Trace log

By default the SPT Driver library is built with activated Trace Log events, in order to measure the execution time of API functions and associated SPT interrupts. This feature is controlled by the "TRACE_ENABLE" preprocessor macro; to disable it, the macro must be undefined and the library must be rebuilt.

5.4.2 SPT Calling Convention

The SPT hardware is designed to execute groups of microcoded instructions (referred to as 'SPT kernels', but with different meaning than the 'OS kernel') that can be placed directly in system memory and passed to the SPT Command Sequencer at runtime. From the point of view of the CPU, each SPT kernel is simply an array of characters. Before starting SPT execution, the SPT Driver writes the address of the kernel into the SPT Program Start Address Register (SPT_CS_PG_ST_ADDR). However, the SPT hardware binary interface does not enforce any means of passing parameters between code running on the CPU and the SPT instructions. The "RSDK SPT Calling Convention" addresses this issue by creating a set of rules which allow basic parameter exchange using the SPT Working Registers:

- **WR0**: passes a 32-bit return value from SPT code to the CPU. The most significant 8 bits of the return value are read from WR0_IM LSB and the remaining 24 bits from WR0_RE
- **WR1..10**: Are used to pass input arguments to the SPT kernel. These can be interpreted as 32-bit integer value or address types (see definition of Spt_ParamType). The maximum number of parameters (SPT↩ _MAX_N_PAR) was chosen as a compromise between the maximum number of parameters that may be required by an SPT kernel and the number of available Working Registers.

5.4.3 SPT Input Arguments

The SPT instructions cannot handle full-width 32bit addresses, instead they work with a 24-bit address offset and an implicit memory base address. Therefore, 'user-space' address arguments must be pre-processed by the SPT_↩ Driver before passing them to the SPT. This raises the need for differentiation between 'address' and 'data' argument types.

The type of input argument is specified by Spt_DrvParamType.

In case of integer-type arguments, the SPT Driver places the most significant 8 bits in the imaginary part and the remaining 24 bits in the real part of the corresponding Work Register.

In case of address-type arguments, the SPT Driver selects and subtracts the static memory base address (e.g. SRAM), then writes the resulting 24-bit offset into the imaginary part of the corresponding Work Register.

The significance of each input argument (address or data) is decided by the user of the SPT_Driver. For any new SPT module its creator can choose a different meaning of the parameters. Assuming all WRs have identical behavior, there is no need to reserve some for addresses and others for data.

In addition to input and output parameters, each SPT module might need additional resources like persistent or scratch memory. These will be managed by the higher application layers, as required by each module. Pointers to different memory areas can be passed as regular input parameters to the SPT modules.

5.5 AUTOSAR integration guidelines

5.5.1 API Function Call Sequence

This section lists the SPT Driver API functions to be called during EcuM start-up, wake-up, run and shutdown:

EcuM State	Function calls
Startup	none
Wakeup	none
Run	Spt_Setup(), Spt_Run(), Spt_Command(), Spt_Stop()
Shutdown	none

5.5.2 Interrupt IDs

The SPT Driver uses the following interrupt IDs:

- #SPT_INTC_OFFSET_ECS
- #SPT_INTC_OFFSET_EVT1
- #SPT_INTC_OFFSET_DSP_ERR

5.5.3 Interrupt callbacks

The SPT Driver uses the following interrupt callbacks:

- **Spt_EcsIsrCb**
- **Spt_EvtIsrCb**
- **Spt_DspIsrCb**
- **Spt_DspBootloaderCb**

5.5.4 Runtime Errors

The SPT Driver can generate the following runtime errors, reported through the Autosar DET module:

- #SPT_E_INVALID_PARAM
- #SPT_E_INVALID_STATE
- #SPT_E_TIMEOUT_BLOCK
- #SPT_E_WARN_HW_BUSY
- #SPT_E_MEM
- #SPT_E_DMA
- #SPT_E_HW_ACC
- #SPT_E_ILLOP
- #SPT_E_TIMEOUT_START
- #SPT_E_TIMEOUT_STOP
- #SPT_E_WARN_UNEXPECTED_STOP
- #SPT_E_HIST_OVF0
- #SPT_E_HIST_OVF1
- #SPT_E_IRQ_REG
- #SPT_E_UNMAP_SPT_MEM
- #SPT_E_WR
- #SPT_E_ILLOP_SCS0
- #SPT_E_ILLOP_SCS1
- #SPT_E_WARN_DRV_BUSY
- #SPT_E_API_INIT_LOCK_FAIL
- #SPT_E_API_ENTER_LOCK_FAIL
- #SPT_E_API_ENTER_UNLOCK_FAIL
- #SPT_E_API_EXIT_LOCK_FAIL
- #SPT_E_API_EXIT_UNLOCK_FAIL
- #SPT_E_THR_CREATE
- #SPT_E_THR_TERM
- #SPT_E_OAL_COMM_INIT
- #SPT_E_CHECK_WATERMARK
- #SPT_E_INIT_Q_FAIL
- #SPT_E_BBE32_REBOOT
- #SPT_E_INVALID_KERNEL
- #SPT_E_HW_RST
- #SPT_E_OTHER

5.5.5 Used Hardware Resources

The following hardware resources are used by the SPT driver and kernels:

- M7 core
- SPT
- BBE32EP DSP
- SPT DSP Command Queue

Chapter 6

DSP User Guide

6.1 Introduction

This section describes the DSP Dispatcher , DSP Host Driver , DSP Linear Algebra Library and DSP Radar Algo Library software components.

It provides details about their functionality, operating modes, directory structure and the steps needed to build the binaries. It also contains guidelines and examples for easy integration. They are described together in a single chapter, due to their tight functional coupling.

For DSP Linear Algebra Library functions cycle estimates, please see `DSP LAL cycle count estimation`.

6.2 Module Overview

The DSP Dispatcher, DSP Radar Algo Library and DSP Linear Algebra Library are deployed on the BBE32 DSP, enabling execution of radar algorithms with a high degree of parallelism. The algorithm execution can be triggered by the SPT "DSP" command , or by commands coming from the host CPU core, through the DSP Host Driver The "DSP Dispatcher" decodes and executes these commands. A typical command is similar to a remote procedure call, consisting of a function ID with its associated parameters, all encoded in 120 bits. See the DSP Calling Convention for details.

The functions to be executed on the DSP can be provided by the user, or taken from the libraries supplied by the RSDK: DSP Radar Algo Library , dsp_lal_api

The Dispatcher and function libraries must be linked together in an executable image, which must be used to boot-up the DSP. Booting procedure is described in Boot-up and Initialization.

Figure 1: SPT & DSP software architecture shows a high-level deployment diagram of SPT and DSP software components. The rsdk_sampleapps demonstrate how to integrate the all the software components into the user code.

Example code is provided in RSDK Offline Example .

6.2.1 DSP LAL description

The RSDK DSP Linear Algebra Library module consists of functions described in dsp_lal_api. The functions use floating-point data and do computations over 8 parallel streaming data (or "batches").

The functions are individual precompiled routines of BBE32EP code packed into a library, each implementing algorithms of Linear ALgebra.

The RSDK Offline Example application shows how to integrate algorithms from this library into the radar data processing flow.

6.3 Design and Operation

6.3.1 Boot-up and Initialization

The Dispatcher and function libraries must be linked together in an executable image, which must be used to boot-up the DSP.

The DSP boot procedure can be done with the help of the SPT Driver, as shown in Booting the BBE32.

Example code is provided in RSDK Offline Example .

Alternatively the DSP can be booted directly by the user application. In this case the Spt_DriverInitPlatSpecific↩ Type::dspEn must be set to "false", to disable the procedure from the SPT Driver.

After being released from reset state, the DSP starts running directly the executable image supplied by the user. This executable must first configure all hardware blocks in the BBE32 subsystem for proper operation, before starting to run application code. For convenience, the Memory Protection Unit, interrupts and prefetch aggressiveness can be programmed by using RsdkDspDispatcherInit(), which provides a user-configurable API. The RsdkDsp↩ DispatcherInit() also initializes the function table (rsdkDspFuncPtr_t) which allows external hosts to trigger execution of functions on the DSP.

More details about prefetch aggressiveness parameters:

- prefetchAggressionData is from 0x0 to 0xF, where 0x0 means no prefetch and 0xF is the most aggressive
- prefetchAggressionInstr is from 0x0 to 0xF, where 0x0 means no prefetch and 0xF is the most aggressive
- prefetchAggressionSW is from 0x0 to 0xF, where 0x0 means no prefetch and 0xF is the most aggressive

Example code for initialization of the prefetch parameters:

```
dspDispInit_t dispInitInfo;

...

dispInitInfo.prefAggrParams.prefetchAggressionData = 0xFu;
dispInitInfo.prefAggrParams.prefetchAggressionInstr = 0xFu;
dispInitInfo.prefAggrParams.prefetchAggressionSW = 0xFu;

...
```

After initialization, the user must call RsdkDspDispatcherRun() to start the main program loop which decodes and dispatches commands.

The /**Apps/DSP_Dispatcher_example**/ application shows how to create an executable application for the DSP, link the needed function libraries and convert it to binary sections which can be loaded to memory at boot time.

After booting, the DSP Dispatcher will be stuck in a busy wait loop in the CommInit function until the master core (M7/A53) initializez the IPCF communication via Dsphd_Init(). After this initialization is done the user must check if the BBE32 is responsive using the DSPHD_MSG_CHECK_DISPATCHER_ALIVE command via the DSP Host Driver.

6.3.2 Sending commands to the DSP

The SPT and the host CPU (ARM) cores can send commands to the DSP independently, by using dedicated hardware support (SPT-DSP Command Queue and, respectively, the DSP Host Driver functions that call the IPCF layer). The hardware support for SPT Commands imposes specific constraints, as described below.

From a software perspective, commands from SPT to the DSP can be sent using the SPT Driver and SPT Kernels, while commands from CPU to DSP can be sent using the DSP Host Driver. All commands, regardless of origin, are parsed and executed by the DSP Dispatcher.

For simplicity of software design, it was decided to encode the SPT and CPU commands identically, allowing the Dispatcher to treat them in a similar way and reuse the same parser for all commands.

SPT Command Constraints

The SPT sends commands to the BBE32 through a 4∗128-bit hardware Command Queue, using the "DSP" instruction. These commands are intended to trigger execution of certain functions on the BBE32, so they should encode a “function ID” and associated list of arguments.

The SPT "DSP" instruction opcode can only support immediate parameters (compile-time numerical constants). If the DSP command encodes the function IDs and arguments directly at compile time, then the user would have to write a different SPT kernel for each possible combination of IDs and arguments that are used in an application, affecting development and maintenance. But since the DSP itself has direct access to SPT internal memory, Work Registers and System RAM, "DSP" command parameters can also be passed indirectly, through these shared hardware resources, while using the command queue only to encode their location (e.g. certain memory address or Work Register numbers, which are set by the user at compile time).

Sending DSP commands from SPT

If an SPT kernel calls the DSP into action, then it must pass a specific DSP command ID and parameters through the SPT Work Registers, according to the DSP Calling Convention. These parameters are passed from the user application running on the host CPU (ARM), through the SPT Driver (e.g. see /SPT/SPT_kernels/src/src/SPT3/34↩_35/common/dsp_example_block.pspt). The list of all DSP command IDs must be known at compile time on both the M7 host side and the DSP toolchain side, by including a shared file (e.g. /Apps/DSP_Dispatcher_↩example/src/host_integration/dispatcher_func_list.inc).

Note

> If an SPT kernel includes the "DSP" instruction, then it will not complete (not set the SPT_CS_STATUS0[PS↩_STOP] bit or trigger an interrupt) until the BBE32 DSP announces the end of its delegated task by pushing a response in the SPT DSP Response Queue.

Example code for calling a DSP function:

```
Spt_DspCmdType dspCmd;

...

dspCmd.id = (uint16_t)DSP_GET_FUNC_ID(RsdkBbe32CaCfar);
dspCmd.arg = (uint32_t)(uintptr_t)caCfarParamsBufH.phyAddr;

//use the SPT Driver to generate the CRC for this command:
sptCmd.cmdId = SPT_CMD_GEN_DSP_CMD_CRC;
sptCmd.cmdParam = (Spt_DriverCommandParamType)&dspCmd;
stdStatus = Spt_Command(&sptCmd, &sptCmdResult);//Legacy code: sptCmdResult is not used in this command, but
      must not be NULL
```

```
statusCheck("Spt_Command", stdStatus);

//all parameters destined for the BBE32 are passed as "SPT_PARAM_TYPE_VALUE" type,
//even if they are pointers, to avoid the address translation done by SPT Driver on "SPT_PARAM_TYPE_ADDR"
      params.
sptContext.kernelParList[0].paramType = SPT_PARAM_TYPE_VALUE;
sptContext.kernelParList[0].paramValue = dspCmd.arg;

sptContext.kernelParList[1].paramType = SPT_PARAM_TYPE_VALUE;
sptContext.kernelParList[1].paramValue = dspCmd.id;

sptContext.kernelParList[2].paramType = SPT_PARAM_TYPE_VALUE;
sptContext.kernelParList[2].paramValue = dspCmd.crc;

sptContext.kernelParList[3].paramType = SPT_PARAM_TYPE_LAST;

sptStatus = Spt_Run(&sptContext);

...
```

Sending DSP commands from the host CPU

Commands from the host CPU to the DSP are sent using the DSP Host Driver, which acts as a client of the DSP Dispatcher. The procedure has 2 steps: first create a job or list of jobs, using Dsphd_CreateJobList(), then send it to the DSP, using Dsphd_SendMsg().

Upon reception of the message, the DSP can execute the commands immediately (in case of #DSPHD_MSG↩ _RUN_ASYNC_JOB message), or schedule them to be executed periodically, with different priorities (in case of #DSPHD_MSG_UPDATE_RC_JOB_LIST or #DSPHD_MSG_UPDATE_LONG_JOB_LIST).

Example code for calling a DSP function:

```
void AppRunDspAsyncJob(uint8_t* pJobListBuf, uint32_t phyAddr, uint16_t kernelName)
{
    rsdkStatus_t        status = RSDK_SUCCESS;
    Dsphd_JobType job;

    job.id = kernelName;
    job.arg = phyAddr;

    status = Dsphd_CreateJobList(&job, pJobListBuf, 1u, DSPHD_JOB_LIST_CRC_DIS);
    if(status == RSDK_SUCCESS)
    {
        CacheFlush((const char *)pJobListBuf, DSPHD_SIZEOF_JOB_LIST);
    }
    statusCheck("RsdkDspHDCreateJobList", status);
    DbgPrintMsg("ARM->BBE32 sending msg: RSDK_DSPHD_MSG_RUN_ASYNC_JOB.\n");

    status = Dsphd_SendMsg(DSPHD_MSG_RUN_ASYNC_JOB, (uintptr_t)pJobListBuf);
    statusCheck("RsdkDspHDSendMsg", status);
    DbgPrintMsg("ARM->BBE32 msg sent: DSPHD_MSG_RUN_ASYNC_JOB.\n");

    // wait for DSP to finish
    AppCondFlagWait(&bbe32AsyncDone);
    DbgPrintMsg("BBE32->ARM : RSDK_DSPHD_MSG_RUN_ASYNC_JOB done.\n");

}
```

6.3.3 DSP Calling Convention

Given all considerations in section Sending commands to the DSP, Radar SDK defines a customized calling convention which supports direct and indirect encoding of "DSP" command parameters, as shown in the figures below.

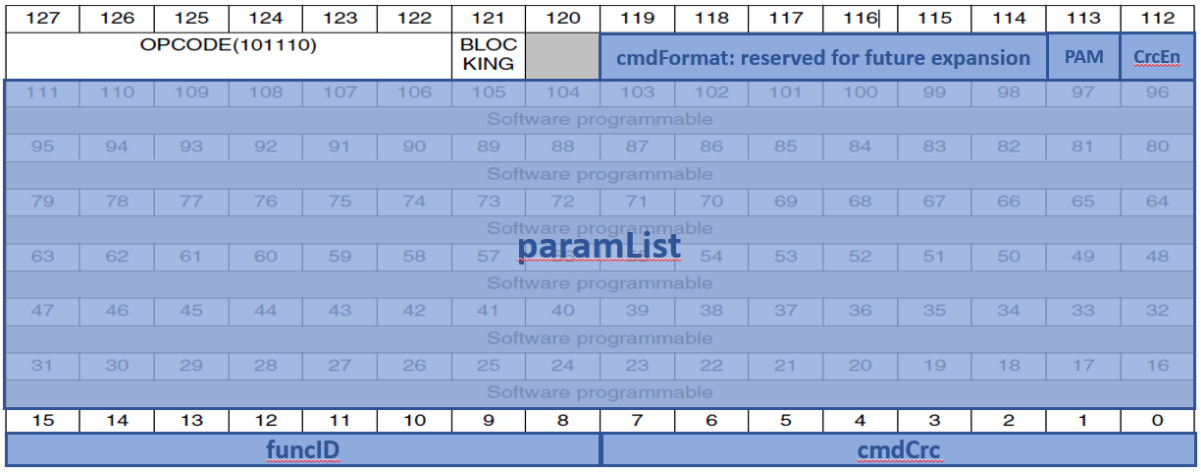
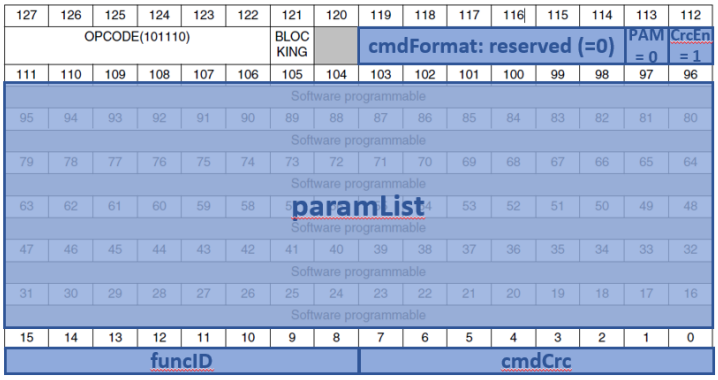


Figure 6.1 DSP Calling Convention overview

Description of command parameters:

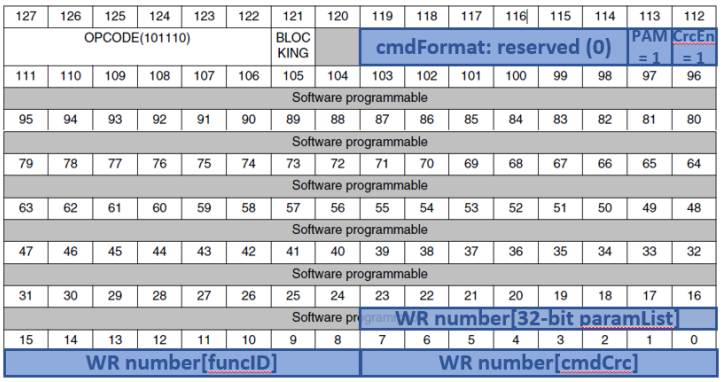
- **cmdFormat::PAM (paramAddrMode)**: specify if paramList and funcID are passed using immediate values or WR number.
- **cmdFormat::CrcEn (CRC enable)**: option to enable/disable CRC checking on DSP side
- **paramList**: DSP function arguments. Content depends on paramAddrMode.
- **funcID**: function identification info. Integer ID preprocessed at compile time, shared with CPU and SPT .
- **cmdCrc**: 8-bit CRC covering the paramList and funcID

Examples of command encoding for direct and indirect parameter passing:



- **cmdFormat::PAM = 0 =>** DSP command parameters values are encoded directly:
  - **paramList** (96 bits): DSP function arguments. They are passed directly to the BBE32 function, without being parsed by the Dispatcher. Can contain anything (including WR numbers)
  - **funcID** (8 bits): DSP function ID.
  - **cmdCrc** (8-bits): CRC covering the paramList and funcID. Must be computed using x^8 +x^2 +x +1 polynomial, on DSP command bits 8:111, with bytes taken in little endian order
- **cmdFormat::CrcEn = 1 => cmdCrc** will be verified by DSP

Figure 6.2 DSP Calling Convention - direct parameter passing



- **cmdFormat::PAM = 1 =>** DSP command parameters values are encoded in Work Registers:
  - **paramList** (32-bits): the DSP function's argument (value or pointer to a param list)
  - **funcID** (8 bits): DSP function ID.
  - **cmdCrc** (8-bits): CRC covering the paramList and funcID
- **cmdFormat::CrcEn = 1 => cmdCrc** will be verified by DSP

Figure 6.3 DSP Calling Convention - indirect parameter passing

Example source code is provided in the SPT kernel library (dsp_example_direct_block.pspt and dsp_example_↩ indirect_block.pspt) and the RSDK Offline Example

6.3.4 DSP Tasks and command scheduling

The DSP Dispatcher implements a preemptive multitasking scheduler driven by the main events in the radar hardware platform:

- interrupts generated by the SPT Command queue
- interrupts generated by the MIPI-CSI2 interface
- interrupts generated by the host CPU core (ARM M7) through IPCF

Each interrupt in the system triggers a processing task with a priority level directly related to the interrupt type. All task priorities are hardcoded in RsdkDspDispatcherRun().

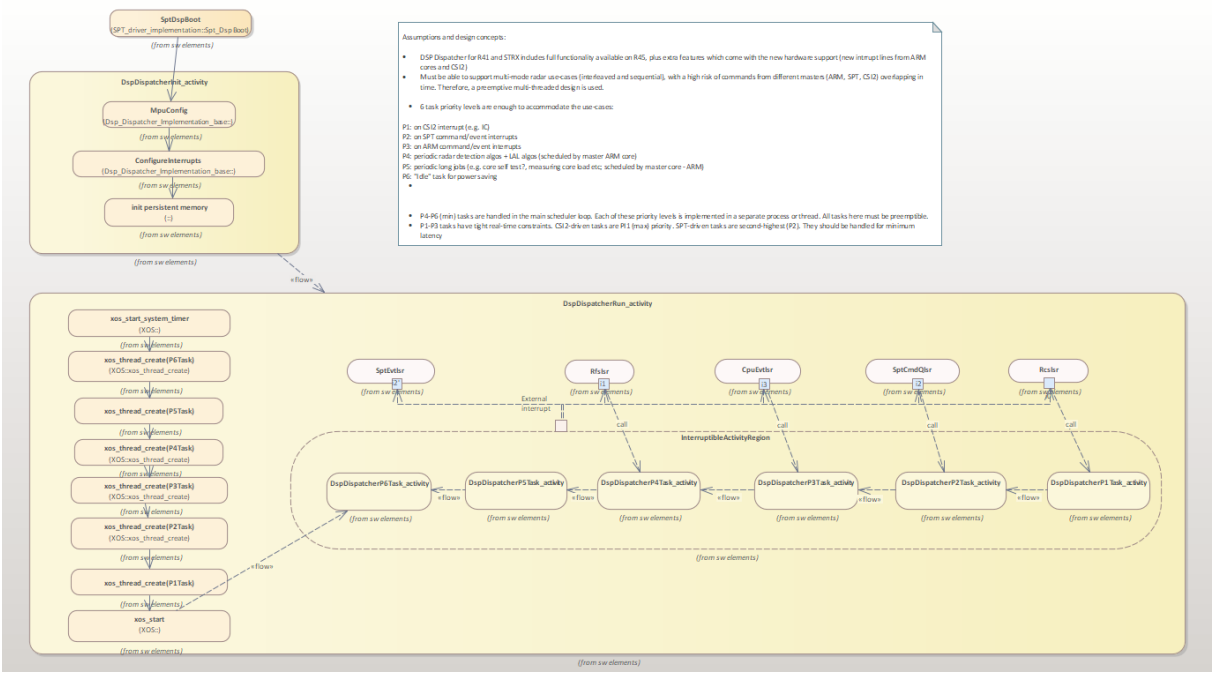


Figure 6.4 DSP Dispatcher Task Scheduling

Assumptions and design concepts for DSP multitasking:

The DSP must be able to support multi-mode radar use-cases (interleaved and sequential chirps in a radar cycle), with a high chance of getting time-overlapping commands from different masters (ARM, SPT, CSI2). Therefore, a preemptive multi-threaded design is used.

The following task priority levels are considered sufficient to accommodate existing radar use-cases on the supported radar hardware platforms:

- P1 - triggered on CSI2 "chirp sync" interrupt: can be used for interference detection, mitigation, or for range processing
- P2 - on SPT command/event interrupts: used for asynchronous commands received through the SPT Command Queue
- P3 - on ARM command/event interrupts: used for asynchronous commands received from the host CPU
- P4 - on CSI2 "frame sync" interrupt: used for periodic radar detection + LAL algorithms, all scheduled by the host CPU core
- P5 - scheduled upon P4 task completion: used for periodic long jobs e.g. core self test, measuring core load etc; scheduled by the host CPU core
- P6 - "Idle" task for power saving

The scheduler is built using the Xtensa® XOS real-time kernel which is supplied as part of the "S32DS BBE32 DSP Add-on package for S32R41".

6.4 Software Configuration

The DSP Host Driver is designed to be configured using the EB Tresos Studio.

The DSP Dispatcher, DSP Radar Algo Library and DSP Linear Algebra Library are configured directly through their C source code APIs.

6.4.1 DSP Host Driver Configuration

6.4.1.1 Tresos plugin setup

Steps to generate the configuration:

- Create a new ∗.link file in `< path to EBT >/EB/tresos/links` folder and add the path to the RSDK plugins: `path=< path to RSDK >/autosar_plugins/S32R41/EBT`
- Open EB Tresos Studio, then open or create a new configuration project
- Add the Dsphd Module to the configuration
- Edit configuration parameters values. See Description of Configuration Parameters for description of parameters.
- Generate the code for your project.

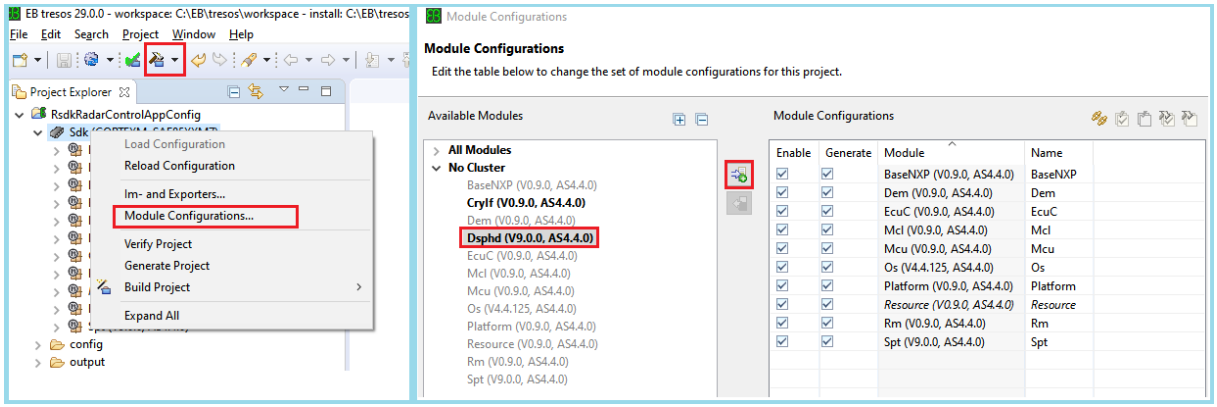


Figure 6.5 Loading DSPHD Configuration in EBT

6.4.1.2 S32DS plugin setup

Steps to generate the configuration:

- Make sure you have run the python script dedicated to add the RSDK S32DS plugins under S32 Design Studio (rsdk/austosar_plugins/<platform>/S32DS/RadarSDK_install_to_DS.py)
- Open S32 Design Studio, then open or create a new configuration project

- Add the Dsphd Module to the configuration by clicking on the "+" icon in the peripherals view
- Edit configuration parameters values. See Description of Configuration Parameters for description of parameters.
- Generate the code for your project by pressing on "Update Code".

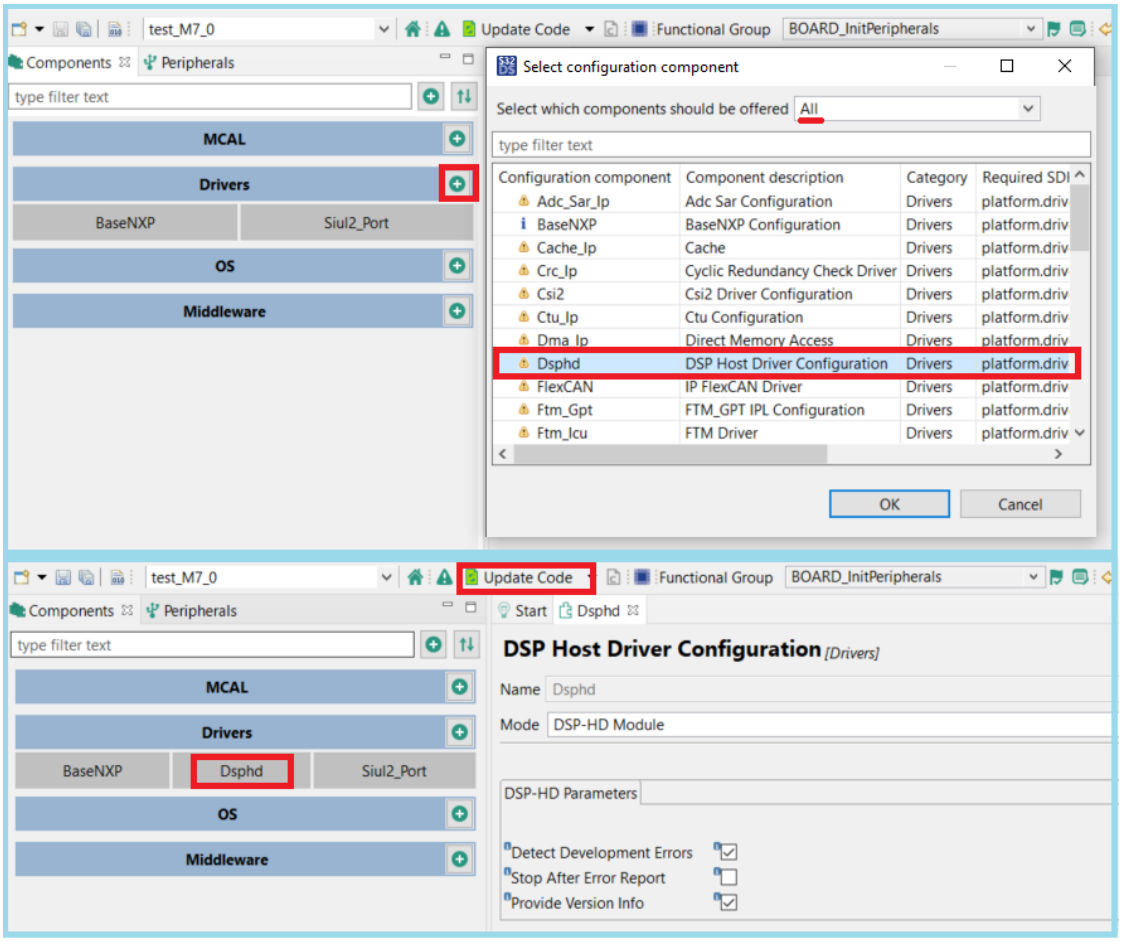


Figure 6.6 Loading DSPHD Configuration in S32DS

6.4.1.3 Description of Configuration Parameters

See Dsp Host Driver Autosar Configuration Parameters for a full description of the DSP Host Driver configuration parameters, extracted from the corresponding ∗.epd file.

6.5 Software Integration

6.5.1 Build instructions

6.5.1.1 Build Tools and Options

The tools and dependencies used for building and executing the RSDK modules are listed in RSDK Environment Dependencies section.

The build options used for validation of the software components are listed in Build configuration.

6.5.1.2 Building the DSP Dispatcher and DSP Algos libraries

The BBE32 uses a proprietary build toolchain, which is integrated in S32 Design Studio.

Each of these projects provides 2 build targets:

- "debug" with no compiler optimizations (-O0) and full debug (-g)
- "release" having full optimization (-O3) and no debug symbols

The libraries can be built in 2 ways:

- either from S32R Design Studio: import the .project file located located in /DSP/< Module_name >/project/ or /DSP/< Module_name >/build, then press the 'Build' button.
- or from the command line terminal, using the makefile located in /DSP/< Module_name >/

For example, the make command line for the DSP Dispatcher is:

```
< rsdk_path >/DSP/DSP_Dispatcher$ make all PLATFORM=S32R41 OSENV=sa
```

Note

In order to build with Makefile you have to add the XTENSA_CORE and XTENSA_SYSTEM environment variables. In Windows, go to Start -> type "Edit environment variables for your account" -> below the "↩ System variables" view press "New.." and add both variables.

Set XTENSA_CORE to **spt_bbe32_vfpu_gold_r1p3** Set XTENSA_SYSTEM to < **s32ds_path** >**/S32DS/build**↩ **_tools/BBE32/builds/RI-2021.7-win32/spt_bbe32_vfpu_gold_r1p3/config**

After the build process is finished, the libraries will be located in the /bin folder.

6.5.1.3 Building the DSP Host Driver

The DSP Host Driver is not built into a separate library, but integrated directly as source code in the top-level App-M7 application When building an application, make sure all source files are added and built along with the application

Example code is provided in RSDK Offline Example

6.5.1.4 Building the DSL LAL library

The tools and dependencies used for building the RSDK modules are listed in RSDK Environment Dependencies section.

Building with Design Studio

The BBE32 uses a proprietary build toolchain, which is integrated in S32 Design Studio.

Each of these projects provides 2 build targets:

- "Debug" with no compiler optimizations (-O0) and full debug (-g)
- "Release" having full optimization (-O3) and no debug symbols

To build the DSP LAL library, open DesignStudio, import the .project file located at < rsdk_path >/DSP/↩ DSP_lal/project/< hw_arch > , then press 'Project -> Build All'. Alternately, each target can be selected and built individually (Project -> Build Configurations -> Set Active, then Project -> Build Project). After the build process is finished, the libraries will be located in < rsdk_path >/DSP/DSP_lal/project/< hw_arch >/< target > , where < target > can be Debug or Release.

Building with make

You may also use the make command to build the library. The corresponding makefile is located in < rsdk_path >/DSP/DSP_lal/build .

The make command line is:

```
< rsdk_path >/DSP/DSP_lal/build$ make all PLATFORM=S32R41
```

Note

In order to build with Makefile you have to add the XTENSA_CORE and XTENSA_SYSTEM environment variables. In Windows, go to Start -> type "Edit environment variables for your account" -> below the "↩ System variables" view press "New.." and add both variables.

```
Set XTENSA_CORE to spt_bbe32_vfpu_gold_r1p3
Set XTENSA_SYSTEM to  < s32ds_path >/S32DS/build_tools/BBE32/builds/RI-2021.7-win32/spt
_bbe32_vfpu_gold_r1p3/config
```

After the build process is finished, the libraries will be located in the /bin folder.

6.5.2 API Function Call Sequence and Constraints

This section lists the DSP Host Driver API functions to be called during EcuM start-up, wake-up, run and shutdown:

EcuM State	Function calls
Startup	Dsphd_Init()
Wakeup	none
Run	Dsphd_CreateJobList(), Dsphd_SendMsg(), Dsphd_GetVersionInfo()
Shutdown	none

This section lists the DSP Host Driver API functions to be called during system initialization and runtime:

- Dsphd_Init(), RsdkDspDispatcherInit() and RsdkDspDispatcherRun() should be called during system initialization phase, on the M7 and BBE32 cores respectively.
- Dsphd_Init() initializes the DSP Host Driver. Calling the sequence of RsdkDspDispatcherInit() and RsdkDsp↩ DispatcherRun() is mandatory after booting the BBE32. DSP Host Driver and Dispatcher can be initialized in any order (BBE32 first or M7 first).
- Dsphd_CreateJobList(), Dsphd_SendMsg() and all the functions from DSP Linear Algebra Library and from DSP Radar Algo Library can be called only after the initialization phase has been completed.

Warning

RsdkDspDispatcherInit() initializes the IPCF memory for the BBE32. XRDC/MMU/MSCM access needs to be configured before booting the BBE32 otherwise the BBE32 will get stuck in a double exception. This is a NON-blocking call that does not check if the M7 memory is initialized. The check that needs to be performed to ensure everything is initialized correctly is below.

Dsphd_Init() initializes IPCF communication and is a mandatory step for the usage of the DSP Dispatcher via core interrupts. This function is NON-blocking. After this function is called it is MANDATORY to call Dsphd_↩ SendMsg() with command #DSPHD_MSG_CHECK_DISPATCHER_ALIVE and check for ACK to ensure IPCF is properly initialize on both the Host Driver and Dispatcher.

Calling Constraints

- The DSP Host Driver API functions are not reentrant.
- Dsphd_SendMsg() triggers core-to-core interrupts between the M7 and BBE32 core. It is not intended to be called more than a few times per radar cycle, since it can affect system performance.

6.5.3 Runtime Errors

The DSP Host Driver can generate the following runtime errors, reported through the Autosar DET module:

- #DSPHD_E_INVALID_PARAMETER
- #DSPHD_E_IRQ_REG
- #DSPHD_E_INVALID_MSG_BUFF
- #DSPHD_E_RELEASE_MSG_BUFF
- #DSPHD_E_TX_FAILED
- #DSPHD_E_COMM_INIT_FAIL
- #DSPHD_E_ACK_OUT_OF_ORDER
- #DSPHD_E_ACK_TIMEOUT

The DSP Dispatcher can generate the following runtime errors, reported through SPT Driver's DSP Error interrupt handler:

- #RSDK_DSP_RET_ERR_CMD_INVALID
- #RSDK_DSP_RET_ERR_CRC_INVALID
- #RSDK_DSP_RET_ERR_CMD_NO_DATA
- #RSDK_DSP_RET_WARN_CMD_CRC_DISABLED
- #RSDK_DSP_RET_ERR_EXCEPTION
- #RSDK_DSP_RET_ERR_AXI_WRITE
- #RSDK_DSP_RET_ERR_MPU_CONFIG
- #RSDK_DSP_RET_ERR_INT_CONFIG
- #RSDK_DSP_RET_ERR_INT_DISABLE
- #RSDK_DSP_RET_ERR_INT_ENABLE
- #RSDK_DSP_RET_ERR_THR_RESUME
- #RSDK_DSP_RET_ERR_TIMER_START
- #RSDK_DSP_RET_ERR_DISP_CONFIG
- #RSDK_DSP_RET_ERR_INVALID_PARAM
- #RSDK_DSP_RET_ERR_INVALID_MSG_BUFF
- #RSDK_DSP_RET_ERR_TX_FAILED
- #RSDK_DSP_RET_ERR_CE_JOBS_NOT_UPDATED
- #RSDK_DSP_RET_ERR_COMM_INIT_FAIL
- #RSDK_DSP_RET_ERR_RC_JOBS_NOT_UPDATED
- #RSDK_DSP_RET_ERR_LONG_JOBS_NOT_UPDATED
- #RSDK_DSP_RET_ERR_TRACE_DISABLED
- #RSDK_DSP_RET_ERR_TRACE_START
- #RSDK_DSP_RET_ERR_FP
- #RSDK_DSP_ALGO_WRONG_PARAM
- #RSDK_DSP_ALGO_WRONG_ALIGN
- #RSDK_DSP_ALGO_CANNOT_CONVERT_TO_FIXED_POINT

6.5.4 Used Hardware Resources

The DSP Dispatcher uses the following hardware resources:

- BBE32EP DSP
- SPT DSP Command Queue (aka DSP message queue)
- SPT DSP_ERR_INFO and DSP_DEBUGn_REG registers)
- MSCM registers through IPCF

Note

Simultaneous writes to SPT WRs by DSP via Control Lookup Unit and by CPU core via peripheral interface results in the CPU core write transaction being ignored (ERR050762). Note that RSDK BBE32 code only writes to WRs using the CRLU interface in RsdkBbe32Func1() and RsdkBbe32Func2() which are demo code.

The DSP Host Driver uses the following hardware resources

- M7 core
- MSCM registers through IPCF

The DSP Linear Algebra Library and DSP Radar Algo Library components use the following hardware resources:

- BBE32EP DSP

Please consult the corresponding hardware Reference Manual for details.

Chapter 7

RFE Abstract 2.0 - User Guide

7.1 Radar front-end (RFE) abstract 2.0 - User Guide

7.1.1 1. Introduction

RFE Abstract 2.0 API is an API for radar application development. The characteristics of this API are:

- Provides mechanisms to communicate between customer application (S32R41-M7-0) and RFE-Fw (S32↩ R41-M7-1).
- Flexible implementation, with no OS dependency, using no OS services for accessing a message buffer (SRAM) or very low OS involvement for IPCF.
- Developed and validated together with RFE-Fw to ensure compatibility with the TEF82XX front end.
- Reduces complexity of application software development by taking care of all functional configurability of the Front-End for the customer to realize radar module.
- Minimum set of abstract APIs for full Front-End control by application.
- Supports complex use cases, for example two chirp sequences in one radar cycle, reconfiguration of chirp profile, phase coding, timing adjustment of frame start.

7.1.2 2. Context

The customer radar application run on the ARM M7-0 core of the S32R41 processor. This customer radar application uses the RFE Abstract 2.0 API to configure and to execute radar cycles on TEF82XX Front-End. The configuration is defined at software design time and is uploaded by the customer application. On ARM M7-1 core is running the application which is managing the TEF82XX Front-End, executes the radar cycle without intervention of the customer application. For more flexibility, Front-End parameters can be updated at run-time between radar cycles. The figure below shows the Context of RFE Abstract 2.0 for TEF82XX, running on S32R41.

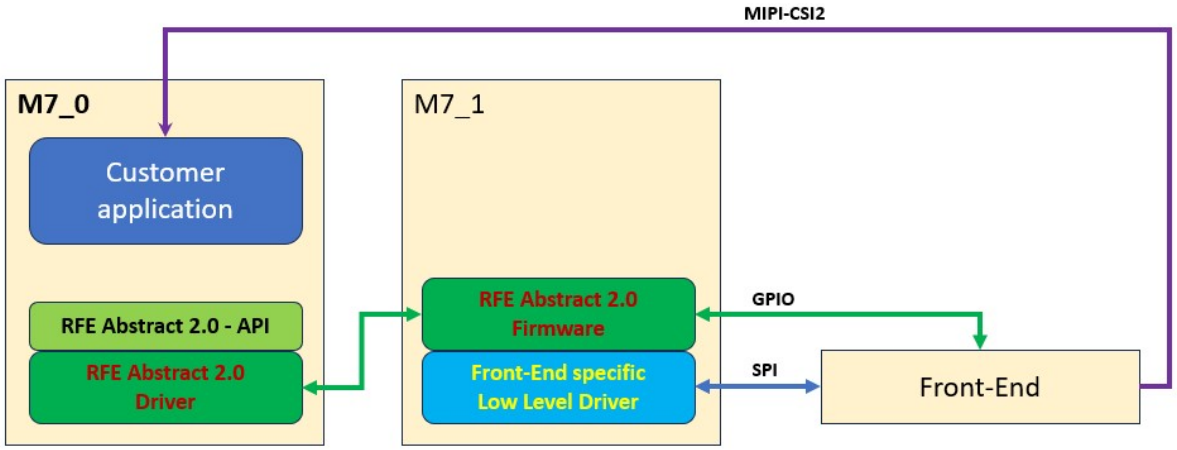


Figure 7.1 Figure 1: RFE Abstract 2.0 high level architecture

The RFE Abstract 2.0 consist of three parts:

- The generic front end API(rsdk_rfe_api_interface) and driver which abstracts the Front End functionality
- The RFE-Fw application which adapt the received commands to TEF82XX Front End
- The Low Level TEF82XX driver, which provide the real communication channel to TEF82XX Front End

7.1.3 3. RFE Abstract 2.0 Software Architecture

The RFE Abstraction 2.0 software architecture can be split in two main parts and some small parts:

- RFE Abstract 2.0 Driver - It provides the following functionalities:
  - Provides RFE Abstract 2.0 API
  - RFE Command Client translates a function call into RFE message and send the message to the M7-1 core, using the SRAM context or IPCF
- RFE-Fw - It provides the following functionalities:
  - Command Server receive and analyze the messages from M7-0 core
  - Command Dispatcher start the appropriate procedure to process the command
  - RFE-Fw executes the necessary Low Level function(s) call to realize the required functionality and provide the necessary answers/results for received calls
  - RFE-Fw starts the necessary internal management loops

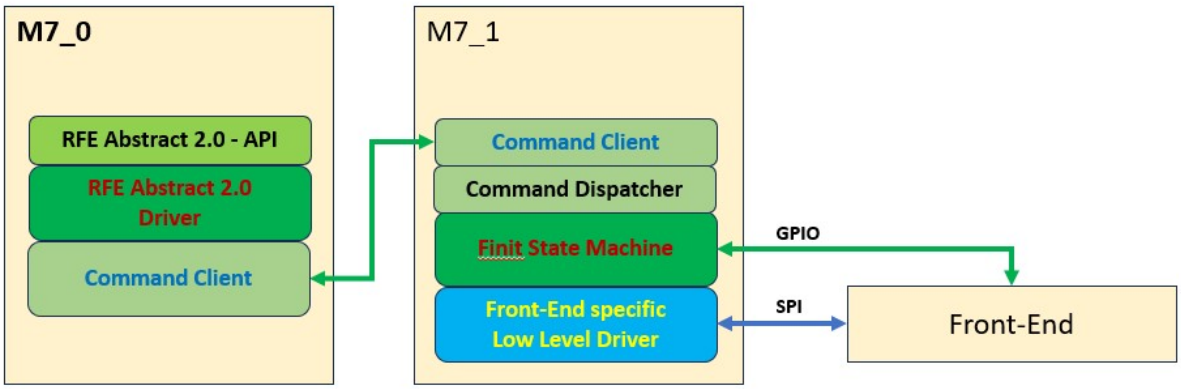


Figure 7.2 Figure 2: RFE Abstract 2.0 architecture

In the context of architecture, it should be noted that the CSI2 TX (on the front-end side) has a dependency between its *Number of lanes* + *Data Rate* and the Front-End's *Sampling Frequency*, as follows:

Front-End Sampling Frequency	CSI2 Lanes & DataRate
5MHz	Number of Lanes = 1 Data Rate 240 MHz
10MHz	Number of Lanes = 2 Data Rate 240 MHz
20MHz	Number of Lanes = 3 Data Rate 240 MHz
40MHz	Number of Lanes = 4 Data Rate 480 MHz

Attention

The above table shows how the CSI2 RX (on S32R41 side) needs to be configured.
It is important that these settings are done in the **CSI2 module**, in order for the data to be received correctly.

7.1.4 4. RFE Abstract 2.0 Driver Components

The RFE Abstract 2.0 Driver can be used at two different level :

- the high level, AUTOSAR compliant, the recommended level: rsdk_rfe_api_interface
- the low level access, directly to the Command Client level: rsdk_rfe_interface For any level, it is important to understand how the Driver and Firmware sides communicates using the Communication Memory Map and how this is implemented into the Tresos plugin.

7.1.5 4.1 RF_Abstract 2.0 Communication

RF_Abstract 2.0 Communication it is an external software commponent and must be must provide by the application. Communication Transport Layer protocol should provide support for 3 channels:

- Command channel: from client(app) to server(FSM)
- Response channel: from server(FSM) to client(app)
- Status channel: from server(FSM) to client(app). Contains FSM state, counters and events

Only one command can be send at a time. The application/driver is waiting untill the firmware sent an acknowledge response. To avoid/detect system firmware getting stuck a timeout merchanism can be implemented at the transport layer.A timeout example implementations is provided in the example app. Status channel carries information that is updated asynchronous by the firmware. These informations is transmited by the firmware enever a state change is happen and are polled on a need basis by the driver. This operation mode impose following requirements for the implementation:

- transmite status function need to be able to send updats without ending in a transmit fail situation(eg. queue full)
- poll function need to provide the latest status information that was tranmited

RSDK Example Application provides two implementation examples for this communication layer.

Messages exchanged between the command client and command server have the following structure:

Command***(256 Bytes)***	
Offset 0	**CNT/Trigger** - CNT is used as sequence counter and to be able to communication protocol errors
Offset 4	**Command ID** - This identifies the command that should be remotely executed
Offset 8	**Param Len** - This is the length of the parameters following this value in unit of Bytes
Offset 12	**Parameters** - Parameters belonging to the commands are placed here
Offset 252	**CRC** - Overall package CRC

Command Response***(256 Bytes)***	
Offset 0	**CNT/Trigger** - CNT is used as sequence counter and to be able to detect communication protocol errors
Offset 4	**Response ID** - This identifies the command that should be remotely executed
Offset 8	**ReturnLen** - This is the length of the return values following this value in unit of Bytes
Offset 12	**Return vals** - return values belonging to the commands are placed here
Offset 252	**CRC** - Overall package CRC

SharedData***(32 Bytes)***	
Offset 0	rfe state
Offset 4	radarCycleCount
Offset 8	chirpSequenceCount
Offset 12	firmwareReady
Offset 16	reserved
Offset 20	reserved
Offset 24	reserved
Offset 28	reserved

Driver - Firmware communication external API:

Function name
rfeIpc_srvRspBuffAcquire()
rfeIpc_srvStsBuffAcquire()
rfeIpc_srvStsTx()
rfeIpc_srvRspTx()
rfeIpc_srvCmdPoll()
rfeIpc_clientCmdBuffAquire()
rfeIpc_clientCmdTx()
rfeIpc_clientStsPoll()
rfeIpc_clientStsWait()
rfeIpc_clientRspWait()

Interaction between driver and firmware through the communication layer is presented in the below figure:

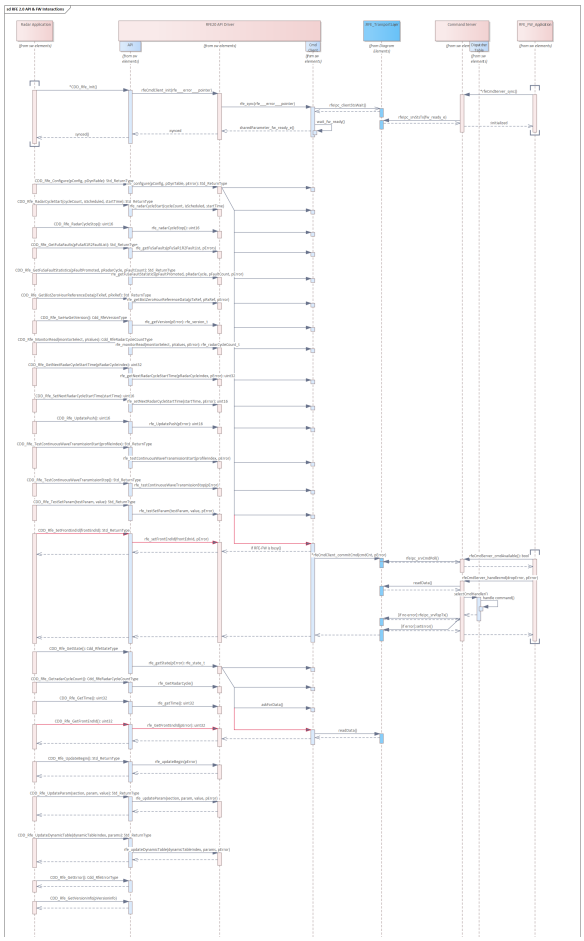


Figure 7.3 Figure 3: RFE 2.0 comunication message flow

7.1.6 4.2 RF_Abstract 2.0 Communication implementation examples

Two different example are provided:

- a communitation layer example build over a share memory area
- a communitation layer example using Inter-Platform Communication Framework(IPCF) driver

The implementation examples are provided as part Example Application.

7.1.7 4.2.1 RF_Abstract 2.0 Communication implementation shared memory example

This is a simple example of usage of a shared memory region to implement the required communication protocol. The physical organization of the shared memory is explained below.

Data Transfer Buffer Address (**BASEADDRESS**) (556 bytes length) The address is user definable.	This is the base address of the command/response server-client communication. Defined by application and hand through transport protocol initialization, witch is done by the application. The uint32_t pointer rfe↩ CmdClient_cmdArea will point to this region. Its location is defined by the RfeDriver. During initialization the RFE M7
	RFE FW is informed about the location using the fixed locations BASE_ADDRESS_PTR (see at the end of the memory map).

Command channel***BASEADDRESS+0***(260 bytes)		Response cha
Offset 0	**Command Payload** - command message payload	Offset 0
Offset 256	**CNT/Trigger** - CNT is used as sequence counter and to be able to detect a new command (trigger)	Offset 256

Data Buffer Pointer Location - Shared Memory Address ((4 bytes/32 bits value)	Location which hold the Data Transfer Buffer Address, transmitted from Driver to Firmware. It must be only read on Firmware side. This location is used as synchronization signal from Driver to Firmware.

The example provided as part Example Application is using the following memory start addresses which are defined using Tresos Rfe20 plugin. The lengths are fixed, according to the Driver - Firmware communication needs. The default values for S32R41 implementation are :

Plugin Label	Recommended value
Data Buffer Pointer Location - Shared Memory Address	0x343FFFFC
Data Transfer Buffer Address	0x343f0000
BIST Buffer Address	0x34400000

Warning

It is highly advised that the above memory locations are protected with a Memory Protection Unit (MPU), to ensure that only the intended M7-0 and M7-1 cores are writing to them. Otherwise, it might be possible due to a misconfiguration that other cores would attempt to R/W data into the shared memory causing undesired behavior.

7.1.8 4.2.2 RF_Abstract 2.0 Communication IPCF example

In Example Application is provided an exmaple for communication transport implementation based on IPCF driver. The IPCF example demostate a implementation using ipcf non-managed channels, with a queue depth of one. The channels used in direction firmware to application are enabled with interrupt(response and status channels). Channel in direction application to firmware(command channel) is without interrupt and it is handled in polling mode.

Communication with dsp cores also is implemented based on IPCF driver support.

Note

IPCF Configuration configuration needs to take in account the request for all components that are using IPCF. As part of application example IPCF is used and configured to communication with the dsp and with the firmware

Configuration of the communication channels can be performed using IPCF Design Studio plugin as shown in the bellow figure.

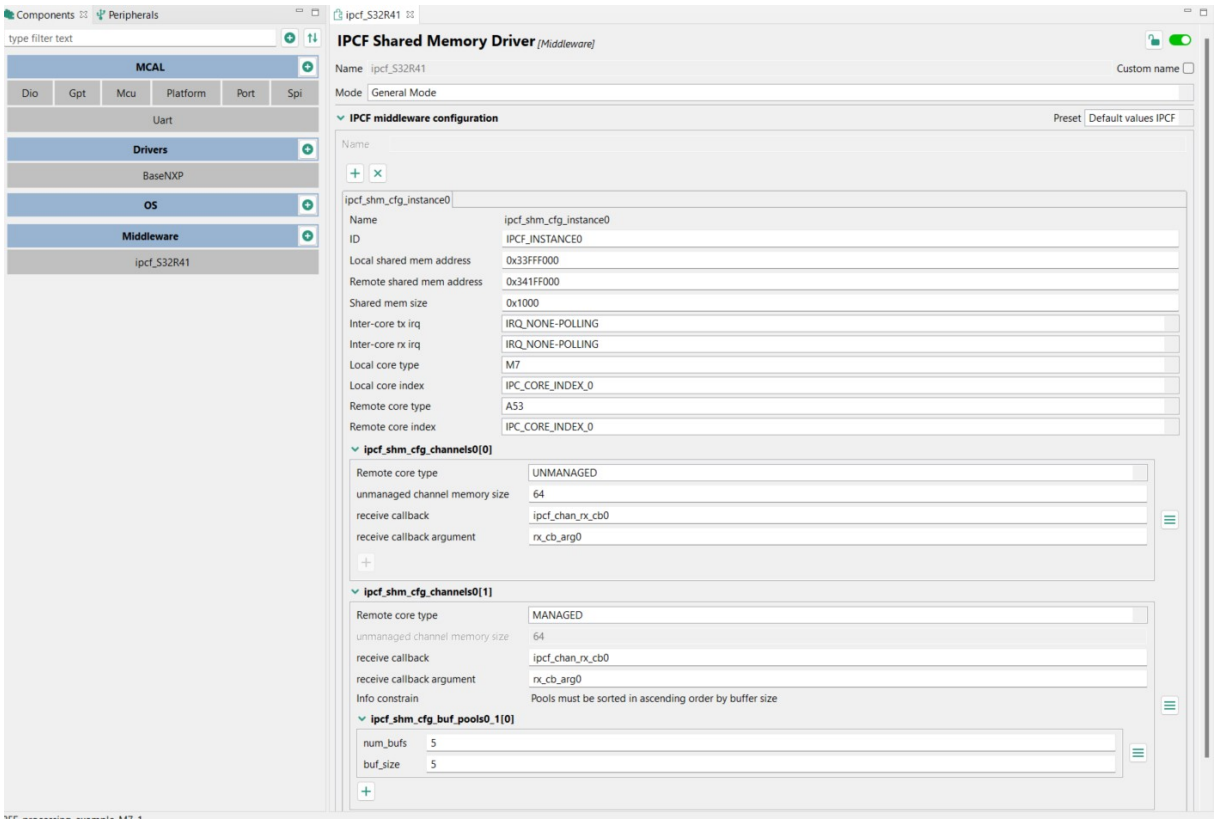


Figure 7.4 Figure 4: RFE 2.0 IPCF Design Studio plugin

Note

Parameters from the above figure are default and are not related to the one used by the application example.

7.1.9 4.3. RFE 2.0 Driver - Tresos configuration

Configuring the Driver using the Tresos plugin is simple and intuitive. The full configuration parameters are shown in the following image.

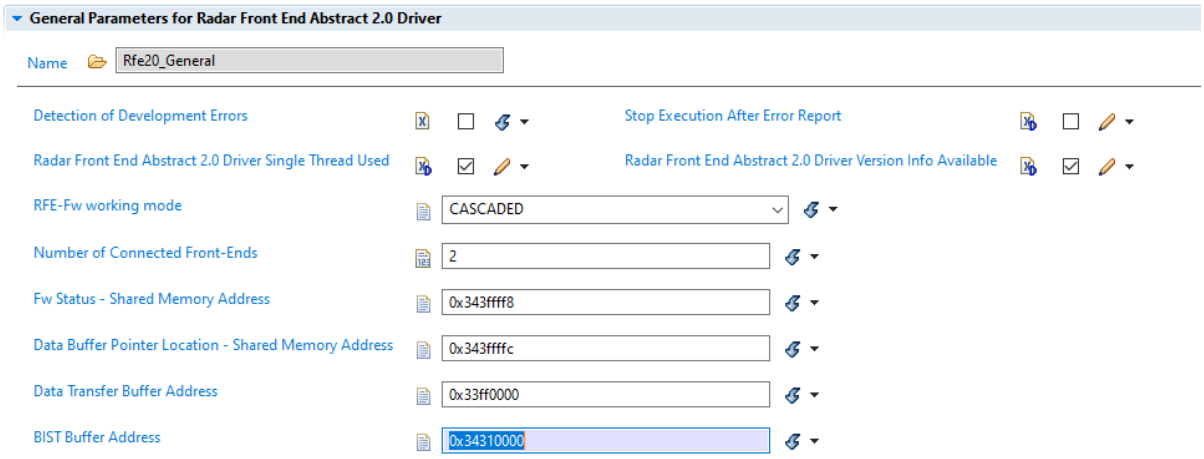


Figure 7.5 Figure 5: RFE 2.0 Driver Tresos plugin

The complete significance of each parameter is described at the plugin usage. But it could be important to add some details at this point, for each parameter :

Detection of Development Errors - Include or not to the compiled code the development checks, like calling parameters checks
FALSE - don't include the code
TRUE - include the code

Stop Execution After Error Report - Introduce a HALT_HERE execution stop loop after an error was reported; this is related to development errors too<
FALSE - don't stop execution
TRUE - stop execution

Radar Front End Abstract 2.0 Driver Single Thread Used - Specify which running context is used : single or multiple thread(s); it is necessary for defining/using the exclusive areas for the code
FALSE - multiple threads used
TRUE - single thread used

Radar Front End Abstract 2.0 Driver Version Info Available - Specify if the Driver version reporting function will be available in the compiled code
FALSE - version not reported
TRUE - version reported

RFE-Fw working mode - Specify the used working mode for the Firmware, according to the connected number of Front-End(s)
STANDALONE - single Front-End used
CASCADED - multiple Front-Ends used

Number of Connected Front-Ends - Specify the number of Front-Ends used/connected; it is not active for STANDALONE, for CASCADED/S32R41 only 2 (two) is accepted , this value is mainly limited by the number of MIPI-CSI2 interfaces of the SoC.

Fw Status - Shared Memory Address - Specify physical memory location of the Firmware status; only valid hexadecimal values as 0x.... are accepted

Data Buffer Pointer Location - Shared Memory Address - Specify physical memory location where the Transfer Buffer Address is written by Driver for Firmware; only valid hexadecimal values as 0x.... are accepted

Data Transfer Buffer Address - Specify physical memory of the beginning of the data transfer area, the minimum reserved length must be 544 bytes; only valid hexadecimal values as 0x.... are accepted

BIST Buffer Address - Specify physical memory of the beginning of the BIST data transfer area, the necessary reserved length must be 1024 bytes for each Front-End; only valid hexadecimal values as 0x.... are accepted

7.1.10 4.4. RFE Firmware FSM

The main schedule and states of the RFE SW are shown in the figure below. The colors are used to indicate the RFE State as returned by CDD_Rfe_GetState(): green corresponds with rfe_state_radarCylceIdle_e and red corresponds to rfe_state_busy_e.

Figure 7.6 Figure 6: RFE FW Main Schedule and States

The RFE FW Main FSM controls this behavior and is shown in the figure below.

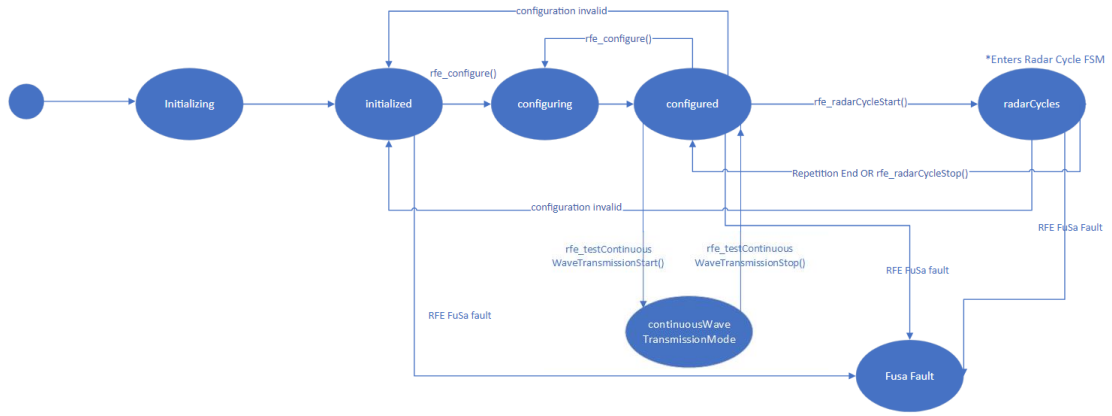


Figure 7.7 Figure 7: RFE FW Main FSM

The schedule and states for the single and dual chirp sequence examples are shown in the two figures below. The colors are used to indicate the RFE State as returned by CDD_Rfe_GetState(): green corresponds with rfe_state↩ _radarCylceIdle_e and red corresponds to rfe_state_busy_e.

Abbreviations: RC Begin: Radar Cycle Begin, CS Init: Chirp Sequence init, CSS: Chirp Sequence Start, CSE: Chirp Sequence End, M&M: Monitor read & Metadata packet.

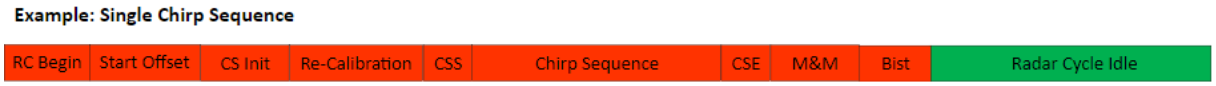


Figure 7.8 Figure 8: RFE FW Single Chirp Sequence Schedule and States Example

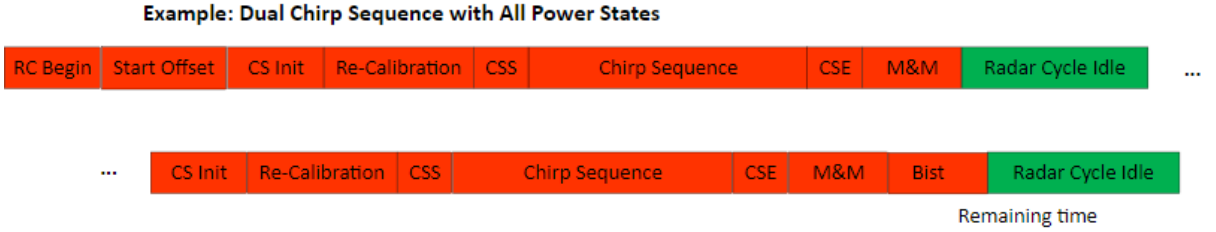


Figure 7.9 Figure 9: RFE FW Dual Chirp Sequence Schedule and States Example

The RFE FW Radar Cycle FSM controls this behavior and is shown in the figure below.

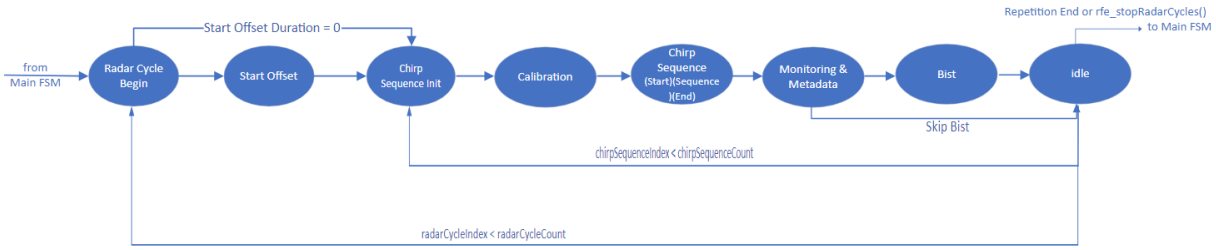


Figure 7.10 Figure 10: RFE FW Radar Cycle FSM

7.1.11 4.5. RFE Abstract API Function Acceptance per RFE State

RFE Abstract API Function \ RFE State	busy	initialized	configured	radar↩ CycleIdle	continuous← Wave↩ Transmissior	fusaFault↩ Recovery	fusaFault
CDD_Rfe↩ _Init()∗	-	-	-	-	-	-	-
CDD_Rfe↩ _GetError()	yes	yes	yes	yes	yes	yes	yes
CDD↩ _Rfe_↩ Configure()	-	yes	yes	-	-	-	-
CDD_Rfe↩ _Radar↩ CycleStart()	-	-	yes	-	-	-	-
CDD_Rfe↩ _Radar↩ CycleStop()	-	-	-	yes	-	-	-
CDD_Rfe↩ _GetNext↩ Radar↩ Cycle↩ StartTime()	-	-	-	yes	-	-	-
CDD_Rfe↩ _SetNext↩ Radar↩ Cycle↩ StartTime()	-	-	-	yes	-	-	-
CDD_Rfe↩ _GetState()	yes	yes	yes	yes	yes	yes	yes
CDD_↩ Rfe_Get↩ Radar↩ Cycle↩ Count()	yes	yes	yes	yes	yes	yes	yes
CDD_Rfe↩ _GetFu↩ SaFaults()	-	yes	yes	yes	-	-	yes

RFE Abstract API Function \ RFE State	busy	initialized	configured	radar↩ CycleIdle	continuous↩ Wave↩ Transmissior	fusaFault↩ Recovery	fusaFault
CDD_Rfe↩ _GetFu↩ SaFault↩ Statistics()	-	yes	yes	yes	-	-	yes
CDD_Rfe↩ _GetTime()	-	yes	yes	yes	yes	-	-
CDD_Rfe↩ _Monitor↩ Read()	-	yes	yes	yes	yes	-	-
CDD_↩ Rfe_Sw↩ HwGet↩ Version()	-	yes	yes	-	yes	yes	yes
CDD_Rfe↩ _GetBist↩ ZeroHour↩ Reference↩ Data()	-	-	yes	-	-	-	-
CDD_↩ Rfe_Test↩ Continuous↩ Wave↩ Transmission↩ Start()	-	-	yes	-	-	-	-
CDD_↩ Rfe_Test↩ Continuous↩ Wave↩ Transmission↩ Stop()	-	-	-	-	yes	-	-
CDD_Rfe↩ _Update↩ Begin()	yes	yes	yes	yes	yes	yes	yes
CDD_Rfe↩ _Update↩ Param()	yes	yes	yes	yes	yes	yes	yes
CDD_Rfe↩ _Update↩ Push()	-	-	yes	yes	-	-	-
CDD_Rfe↩ _TestGet↩ Internal↩ Error()	-	yes	yes	yes	-	-	yes
CDD_Rfe↩ _GetBlob↩ Value()	yes	yes	yes	yes	yes	yes	yes
CDD_Rfe↩ _SetBlob↩ Value()	yes	yes	yes	yes	yes	yes	yes

RFE Abstract API Function \ RFE State	busy	initialized	configured	radar↩ CycleIdle	continuous← Wave↩ Transmissior	fusaFault↩ Recovery	fusaFault
CDD_Rfe↩ _SetFront↩ End()	-	yes	yes	yes	-	-	yes
CDD_Rfe↩ _GetFront↩ End()	-	yes	yes	yes	-	-	yes
CDD_↩ Rfe_Get↩ Register↩ Dump()	-	yes	yes	-	-	-	yes
CDD↩ _Rfe_↩ Configure↩ Interrupt()	-	yes	yes	-	-	-	-

Remarks

: ∗ CDD_Rfe_Init() is the first function to be called. RFE State cannot be read before CDD_Rfe_Init() is called. After CDD_Rfe_Init() returns with no error, the RFE state changes to initialized and CDD_Rfe_Init() should not and does not need to be called anymore.

7.1.12 4.6. RFE Abstract API details

7.1.12.1 4.6.1 RFE Abstract API Interaction with RFE FW

The table below lists the APIs and indicates whether an API interacts with RFE FW or not.

RFE Abstract API Function	Interacts with RFE FW (yes or no "-")
CDD_Rfe_Init()	-
CDD_Rfe_Configure()	yes
CDD_Rfe_RadarCycleStart()	yes
CDD_Rfe_RadarCycleStop()	yes
CDD_Rfe_GetNextRadarCycleStartTime()	yes
CDD_Rfe_SetNextRadarCycleStartTime()	yes
CDD_Rfe_GetState()	-
CDD_Rfe_GetRadarCycleCount()	-
CDD_Rfe_GetFuSaFaults()	yes
CDD_Rfe_GetFuSaFaultStatistics()	yes
CDD_Rfe_GetTime()	yes
CDD_Rfe_MonitorRead()	yes
CDD_Rfe_SwHwGetVersion()	yes
CDD_Rfe_GetBistZeroHourReferenceData()	yes
CDD_Rfe_TestContinuousWaveTransmissionStart()	yes
CDD_Rfe_TestContinuousWaveTransmissionStop()	yes
CDD_Rfe_UpdateBegin()	-
CDD_Rfe_UpdateParam()	-

RFE Abstract API Function	Interacts with RFE FW (yes or no "-")
CDD_Rfe_UpdatePush()	yes
CDD_Rfe_TestGetInternalError()	yes
CDD_Rfe_GetBlobValue()	-
CDD_Rfe_SetBlobValue()	-
CDD_Rfe_SetFrontEnd()	yes
CDD_Rfe_GetFrontEnd()	yes
CDD_Rfe_GetRegisterDump()	yes
CDD_Rfe_ConfigureInterrupt()	yes

7.1.12.2 4.6.1 RFE Abstract API TEF82XX cascading support

The RFE API interface provides three kinds of functions, with different Front-End usage:

- Functions which apply globally to the RFE-Firmware software.
  - CDD_Rfe_Init()
  - CDD_Rfe_RadarCycleStart()
  - CDD_Rfe_RadarCycleStop()
  - CDD_Rfe_GetNextRadarCycleStartTime()
  - CDD_Rfe_SetNextRadarCycleStartTime()
  - CDD_Rfe_GetTime()
  - CDD_Rfe_SwHwGetVersion()
  - CDD_Rfe_TestContinuousWaveTransmissionStart()
  - CDD_Rfe_TestContinuousWaveTransmissionStop()
  - CDD_Rfe_TestGetInternalError()
  - CDD_Rfe_ConfigureInterrupt()
  - CDD_Rfe_SetFrontEnd()
  - CDD_Rfe_GetFrontEnd()
- Functions which are applied to multiple Front-Ends simultaneously.
  - CDD_Rfe_GetBistZeroHourReferenceData()
  - CDD_Rfe_MonitorRead()
- Functions which apply to a single, selected Radar Front-End only. For a Cascaded solution, these functions need to be preceded by a call to apply to the Radar Front-End previously selected by the call to CDD_Rfe↩ _SetFrontEnd.
  - CDD_Rfe_Configure()
  - CDD_Rfe_GetFuSaFaults()
  - CDD_Rfe_GetFuSaFaultStatistics()
  - CDD_Rfe_UpdatePush()
  - CDD_Rfe_GetRegisterDump(). This function have a first argument CDD_RfeFrontEndIdMaskType $<$ frontendSelect $>$, which defines a mask to select one or more Radar Front-Ends.

7.1.13 4.7. RFE Abstract 2.0 Configuration

The S32R41 RFE SW is configured through the two data structure shown in the figure below:

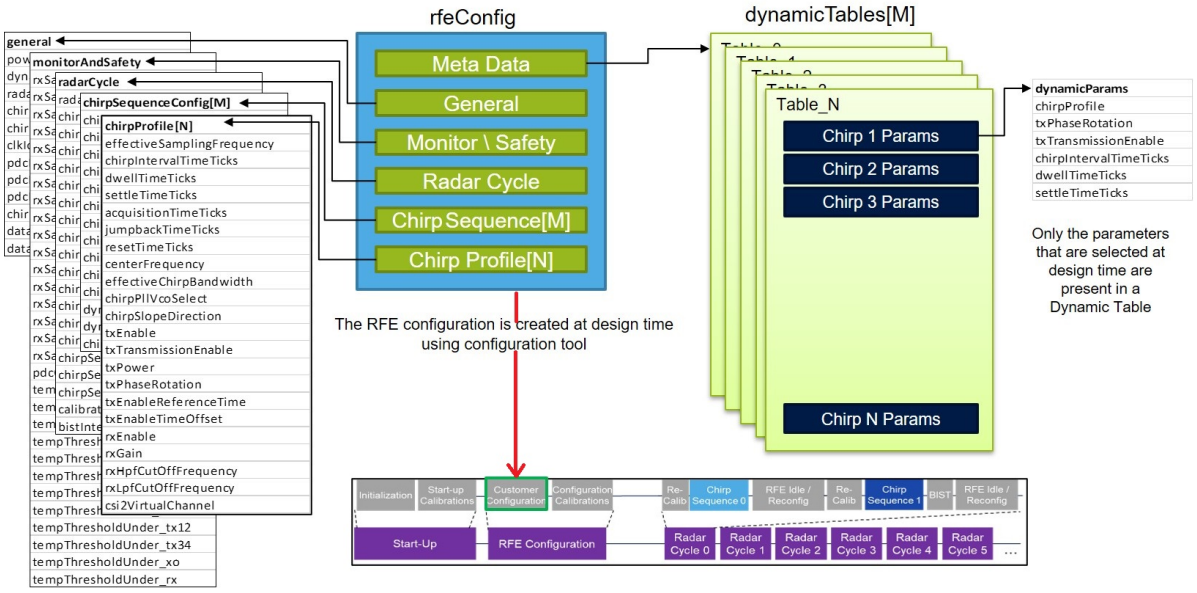


Figure 7.11 Figure 11: RFE Abstract 2.0 Configuration structure

The rfeConfig data structure contains a number of item values that are grouped together in sub-tables and this defines the static configuration for the S32R41 RFE SW. For example, a chirpProfile sub-table contains an item called effectiveSamplingFrequency that specifies that the effective sampling frequency should be either 10, 20, or 40 MHz. The dynamicTable is an additional data structure that can be used to instruct the S32R41 RFE SW to update a limited set of parameters in between chirps. For instance, it is possible to use this mechanism to change the phase rotation for a TX channel for each chirp that make up a radar cycle. These data structures can be created with the provided rfeConfigGenerator software tool.

7.1.14 4.8 RX BIST

The RX Built in Self-Test is a test performed on the Radar Front-End during the radarCycle in order to ensure the integrity of the receiver chain. RX BIST is performed using an internally generated signal with pre-defined frequency and amplitude, fed into the Rx paths. The received signals' frequency, phase and magnitude is compared with "zero-hour data" (reference measurement performed at a pre-determined time by the user, e.g. end-of-line radar device tests, rework done in service). Figure 12 shows the flowchart of zero-hour data measurement and using that data in regular radarCycle.

Note

Ensure the CSI2 driver (initialized on M7_0 side) has a correct setup for BIST, for each CSI2 unit connected to a Front-End :

```
SetupParam_0_Params_VC_3.expectedNumSamples = 128u;
SetupParam_0_Params_VC_3.channelsNum        = 4u;
SetupParam_0_Params_VC_3.expectedNumLines   = 1u;
SetupParam_0_Params_VC_3.bufNumLines        = 1u;
SetupParam_0_Params_VC_3.bufLineLen         = 1104u;    // (128 samples * 4 channels * 2 bytes/sample)
+ 80 bytes Statistics buffer (to be added even the Statistics are not used)
SetupParam_0_Params_VC_3.vcEventsReq        = 0u;
SetupParam_0_Params_VC_3.outputDataMode     = CSI2_VC_BUF_OUT_TILE8 | CSI2_VC_BUF_REAL_DATA |
CSI2_VC_BUF_NO_FLIP_SIGN;
SetupParam_0_Params_VC_3.bufDataPtr         = buffer_head for first Front-End, (buffer_head +
bufLineLen) for the second Front-End, etc., each Front-End has a different buffer.
```

This setup values must be used for any CSI2 usage, with or without Statistics reported by the CSI2 interface. buffer_head is defined at 4.1 RF_Abstract 2.0 Communication as **BIST Buffer Address** and must be used as defined.

7.1.14.1 4.8.1 Steps to get correct zero hour reference data for BIST

step-1. Set the following fields in the blob:

- Un-mask all FuSa faults for BIST (the first one is rfeCfg_param_monitorAndSafety_fuSaFaultMask_0_e)
- Set bist-interval to "every radar cycle"

step-2. Perform rfe-configure

step-3. Once rfe-configure is successful, call the API CDD_Rfe_GetBistZeroHourReferenceData()

step-4. The data obtained in step 3 shall be saved in a persistent memory and used to populate the zero hour reference data in the rsdk_rfe_blob for every subsequent rfe-configure call.

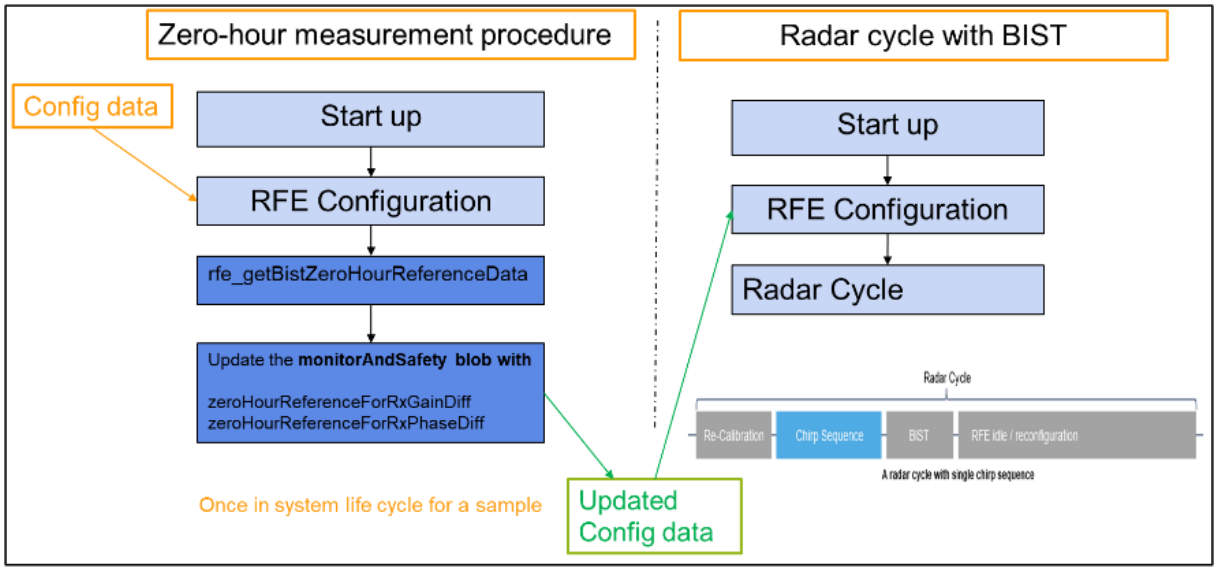


Figure 7.12 Figure 12: Flowchart showing zero-hour data measurement and normal radar-cycle

7.1.14.2 4.8.2 How to use the RX BIST feature

RX BIST is used verify the integrity of the receiver chain. A RF signal is fed to the receiver to perform RX measurements including amplitude and phase variation between combinations of receiver channels. Figure 13 shows a block diagram of RX-BIST. Below parameters are extracted and checked if their deviation from zero-hour Data is within pre-defined thresholds, for both LNA and Mixer input -

- Phase difference between RX1 to RX2/3/4
- Gain difference between RX1 to RX 2/3/4

Note: If receiver channel 1 (RX1) is disabled, the RX BIST will not be performed even if the faults are un-masked or other receiver channels are enabled.

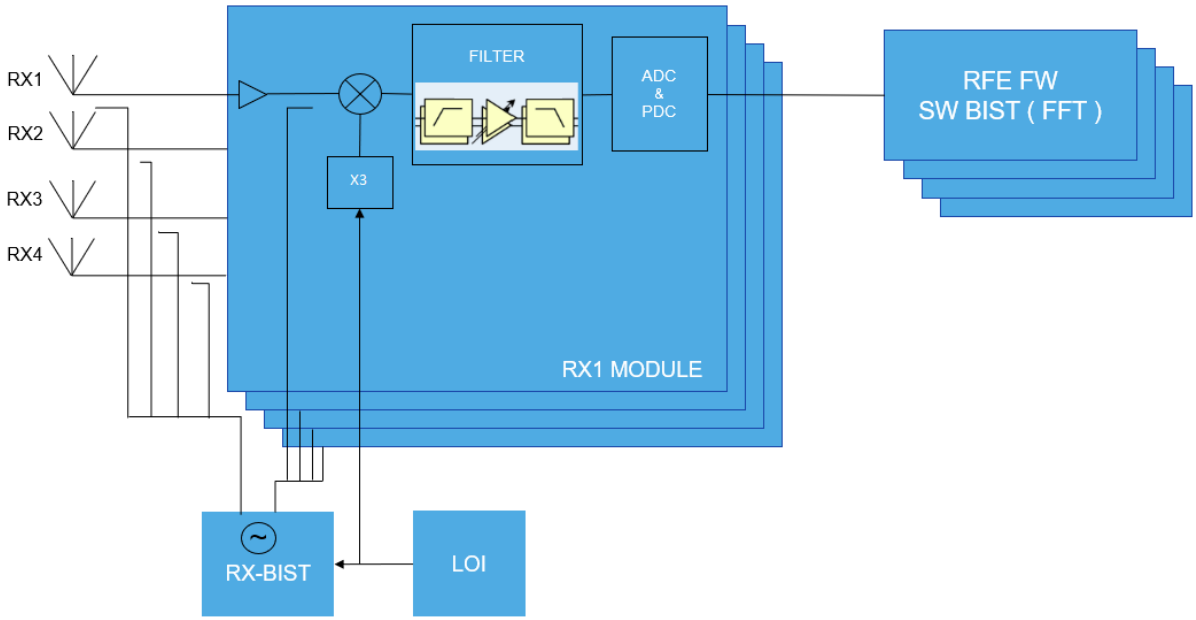


Figure 7.13 Figure 13: Block-diagram showing RX BIST

Using RX BIST feature from application:

Step 1: Set the parameters rfeCfg_param_monitorAndSafety_injectTestTone_e and monitorAndSafety in the rsdk↩_rfe_blob

Step 2: Only for the first power up of the device (e.g. during end-of-line tests, IC replacement, etc) . Application shall fetch the zero-hour Data using the Abstract API CDD_Rfe_GetBistZeroHourReferenceData() and store the values of following parameters (for both LNA and Mixer input) for future use –

- signal magnitudes difference
- signal phase difference

Note

For getting correct values of Zero-hour-data, please refer to section 4.8.1 Steps to get correct zero hour reference data for BIST.

Step 3: The data received in step-2 must be stored by the Application and be used as zero-hour reference data in the rsdk_rfe_blob for next radarCycle.

Step 4 : ENABLE / DISABLE RX BIST There are four types of faults which can be triggered by RX BIST. Below mentioned list shows all the four faults and their corresponding enumeration names. Masking of below Faults can be done in the rsdk_rfe_blob. RX BIST will be enabled if any of below faults are un-masked and at-least RX1 must be enabled.

S. No.	Fault detail	Fault name
1	Mixer phase diff error	rfe_error_rxbist_mixer_phase_fail↩ _e
2	Mixer gain diff error	rfe_error_rxbist_mixer_gain_fail_e
3	LNA phase diff error	rfe_error_rxbist_lna_phase_fail_e
4	LNA phase diff error	rfe_error_rxbist_lna_gain_fail_e

Note

Embedded in the RX bist mechanism, there is a check for absolute amplitude of the SSB, generated tone, to be greater than -30dBFS. Also, this tone level is calibrated every bist cycle to be -15dBFS.

7.1.15 4.9 Functional Safety (FUSA) faults

FuSa faults consists of faults reported by Inner Safety Monitor (ISM) that will assert Error_N pin and faults reported from BIST and integrity check procedures.

MainFSM will go to FUSA fault state(rfeSwMainFsm_mainState_fuSaFault_e) if one of the following condition is met:

- Error_N pin is low(Category 1 FUSA fault is triggered)
- Fault is reported from BIST and integrity check process(Category 2 FUSA fault is triggered)
- Any system error is active(Category 3 rfe-generic-sw is triggered)

List of FUSA faults are exposed via the configuration xml file in section monitorAndSafetyConfig.

The list of FUSA faults is defined in enumeration rfe_fuSaFault_t in rfe_error.h

There are 3 types of faults in the list

Category 1: FUSA faults reported from ISM that leads to Error_N pin to be low

Category 2: FUSA faults reported from BIST and integrity check process.(fault name ending with -sw)

Category 3: Generic software error (Non FUSA, rfe-generic-sw). Any system(internal) error reporting from TEF82XX front end will trigger this bit to be 1.

User can mask/unmask FUSA faults via the list in the configuration xml file and rsdk_rfe_blob updates.

Note

The rfe-generic-sw can NOT be masked! It is present in XML configuration mask section, fuSaFaultMask11, only for reference, masking will have no efect.

If MainFSM is in FUSA fault state (rfeSwMainFsm_mainState_fuSaFault_e), the following 2 functions should be called for reporting FUSA faults and internal system errors:

- CDD_Rfe_GetFuSaFaults() Call this function to check which FUSA fault is triggered.
  The output of this function is an array of 12 8-bit numbers. Each bit can be parsed using the fuSaFault↩ Mask entries from the rfeConfig.xml file. For example, if index 3 of the output array has the value b0010000, it means bit position 5 of fuSaFaultMask3 is set, i.e rfe_fuSaFault_sr47_lo_level_max_rx1_e fault is set. If index 11 of the output array has the value b0001000, it means bit position 4 of fuSaFaultMask11 is set, i.e. rfe-generic-sw is set.
- CDD_Rfe_TestGetInternalError() Call this function if rfe-generic-sw is reported as FuSa error after calling CDD_Rfe_GetFuSaFaults() (as described above when index 11 of the CDD_Rfe_GetFuSaFaults() output array has the following bit position set: b0001000) This function will return which system error from rfe_↩ error_t occurs.

7.1.16 4.10 Dynamic tables

The dynamic tables are used to command the QPSK I/O control lines for each of the 3 TX of TEF82XX. The control code can be specified per chirp and per sequence. The XML Dynamic table information is translated into a binary rsdk_rfe_blob by the RFE Config generator tool. The rsdk_rfe_blob must be loaded or copied to a system memory location. The system memory address is then provided as an argument to rfe_configure().

Please see TEF82XX Reference manual RM00227 chapter 4.4.8 for detailed description.

The I/Q line control is done on the rising edge of the MCU INT pin of TEF82XX and the change is applied during current chirp. The lines are sampled in a syncronous mode before tSettle of the next chirp. Please see the example app provided in the package for the call of rfeSwDynamicTables_IQUpdate() function from the interrupt.

Example of dynamic table entry:

```
<dynamicTables>
    <dynamicTable>
        <dynamicTableEntry phase-rotation-tx-1="0" phase-rotation-tx-2="0" phase-rotation-tx-3="0" />
        <dynamicTableEntry phase-rotation-tx-1="90" phase-rotation-tx-2="180" phase-rotation-tx-3="270" />
        <dynamicTableEntry phase-rotation-tx-1="180" phase-rotation-tx-2="0" phase-rotation-tx-3="180" />
        <dynamicTableEntry phase-rotation-tx-1="270" phase-rotation-tx-2="180" phase-rotation-tx-3="90" />
    </dynamicTable>
</dynamicTables>
```

The values will be applied to I and Q pins as following:

```
0   => I = 0, Q = 0
90  => I = 0, Q = 1
180 => I = 1, Q = 0
270 => I = 1, Q = 1
```

If there are more chirps in sequence then entries in the table the IQ updater will continue in a circular buffer manner.

7.1.17 5. Reference driver usage

The following figure shows the reference usage of the S32R41 RFE Abstract API with single TEF82XX frontend.

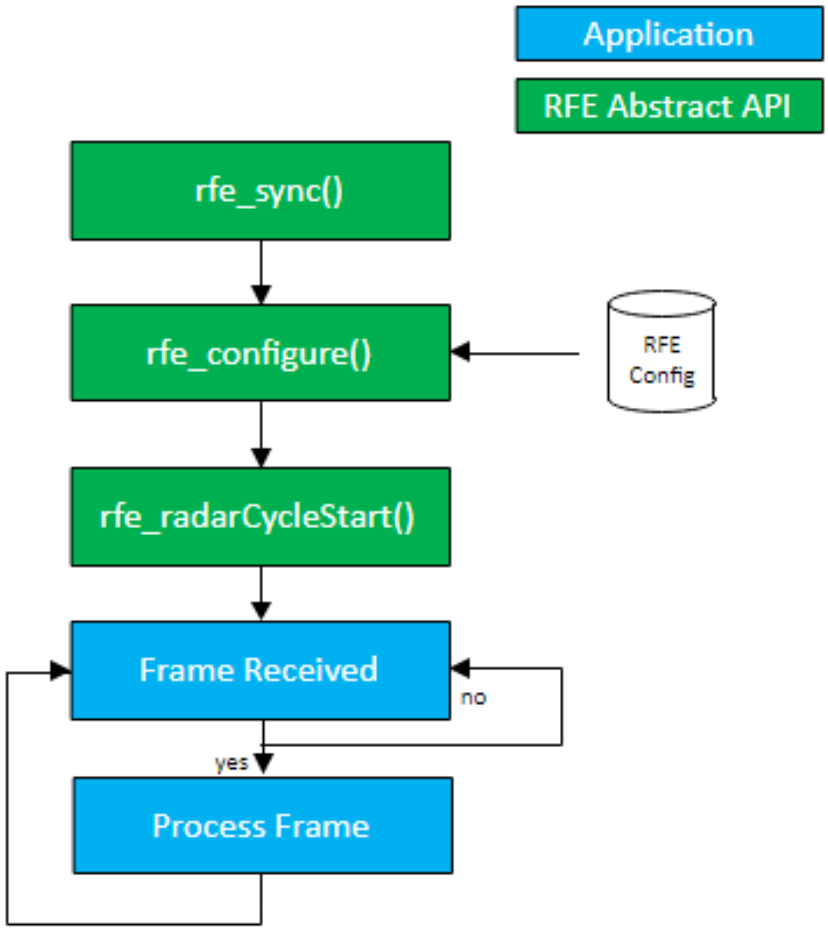


Figure 7.14 Figure 14: RFE Abstract typical application flow single frontend

The following figure shows the reference usage of the S32R41 RFE Abstract API with two cascaded TEF82XX frontends.

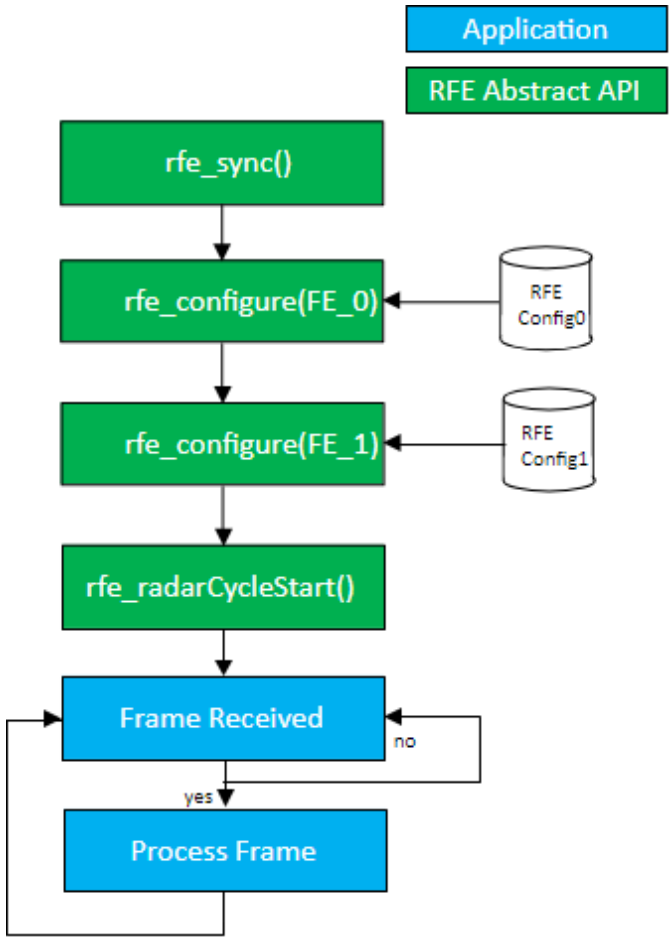


Figure 7.15 Figure 15: RFE Abstract typical application flow with two cascaded TEF82XX frontends

The following figure shows the reference usage of the S32R41 RFE Abstract API in case radar cycle involves time synchronization. The example is for single TEF82XX, for cascaded TEF82XX use the configuration part from above. Also, the timing synchronization mechanism works only for release M7_1 version, debug version do not implement idle waits and timeouts.

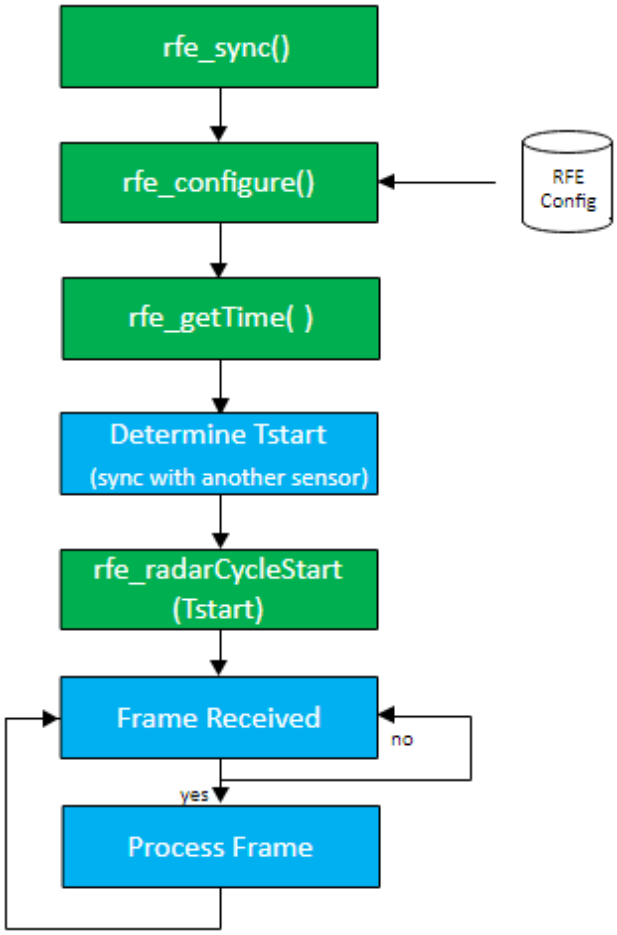


Figure 7.16 Figure 16: RFE Abstract typical application flow, with Time Synchronization

7.1.18 6. RFE Abstract 2.0 Build

All driver components have Design Studio projects for build. The component that needs to be built to have a functional driver is **RFE_abstraction_2_FW_M7**. This is the library that needs to be linked in the final application on M7_1 core that controls the TEF82XX frontends. The Design Studio project for it is located *[rsdk_package]/RFE_abstract/RFE_driver2/RF_FW/project/S32R41/RFE_abstraction_2_FW_M7*. The driver has two target configurations: debug and release.

Note

For an example how to use the RFE Abstraction 2 library on M7_1 core and the RFE driver API on M7_0 please refer to the example project RSDK RFE processing Example located at *[rsdk_package]/Apps/RFE_processing_example*

Preprocessor defines parameters MIPI_SETUP_SEQUENCE

- If this is NOT defined, driver will activate all virtual channels used in configuration, on all profiles. This is done at configuration stage only one time. TEF82XX will send the events on all MIPI CSI2 virtual channels configured, not on the specific channel programed in the current profile from current sequence.
- If this is DEFINED, driver will activate only virtual channels used in profiles from current sequence. This is done at every chirp sequence.

7.1.19 7. RFE Abstract 2.0 Config Generator

The Front-End configuration in RFE Abstract 2.0 is done through a Config Generator that takes an XML as input and outputs a rsdk_rfe_blob. The rsdk_rfe_blob is then loaded into memory and used to configure the Radar Front-End.

The tool can be found in *[rsdk_package]/Tools/RFE2_Config/*. It is comprised of the following contents:

- **rfeConfigGenerator (folder)**: Contains the core files of the configurator. The user would usually not use this folder.
- **readme.txt**: This file gives a brief overview on how to use the tool: what are the available CLI commands, some examples on their usage and what are the output files.
- **rfeConfig.xml**: This is the input file for the tool. The FrontEnd configuration is done here. All the details related to this file are found in the PDF manual (last bullet point).
- **rfeConfig_tef82xx_0.8.10.xsd**: This is the schema file needed for building the XML. It provides information about what values can be used for each parameter.
- **rfeConfigGenerator.bat**: This runs the configurator which would generate the output files.
- **TEF82XX RFE Config Generator User Manual.pdf**: This is the central document for this tool. All the information related to: usage, parameter values and their description, tools needed for building the XML, how to build the CSV generator executable and so on, can be found in this manual. Always consult this file first, for aspects related to the Config Generator.

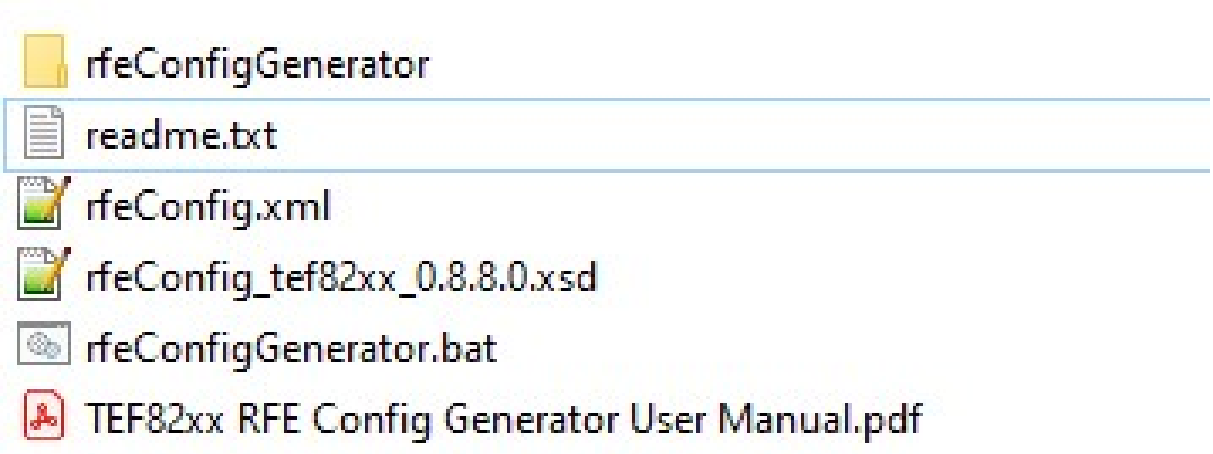


Figure 7.17 Figure 17: RFE Config Generator contents

Attention

The RSDK package does not contain the executable required to generate the CSV timing files, and it is required to be built if the CSV files are needed.
Details can be found in Chapter 6 from **TEF82XX RFE Config Generator User Manual.pdf**.

Chapter 8

Module Index

8.1 Software Modules

Here is a list of all modules:

Chapter 9

Module Documentation

9.1 SPT Driver Autosar Configuration Parameters

9.1.1 SPT Driver Autosar Configuration Parameters

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module Spt
  - Container SptGeneral
    - Parameter POST_BUILD_VARIANT_USED
    - Parameter SptDevErrorDetect
    - Parameter SptHaltOnError
    - Parameter SptSingleThread
    - Parameter SptVersionInfoApi
    - Parameter SptDspEnable
    - Parameter SptRunPoll
  - Container SptSetup
    - Parameter SptSetupStructureName
    - Parameter SptDspEnable

Module Spt

Included containers:

- SptGeneral
- SptSetup

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	False
supportedConfigVariants	VARIANT-PRE-COMPILE

Container SptGeneral

<html> Configuration of the SPT module. </html>

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Parameter POST_BUILD_VARIANT_USED

Indicates whether a module implementation has or plans to have (i.e., introduced at link or post-build time) new post-build variation points.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	EB
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | false

Parameter SptDevErrorDetect

Pre-processor switch to enable/disable development error detection for Spt API

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | true

Parameter SptHaltOnError

Pre-processor switch to enable/disable halting on error for development errors for Spt API

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | false

Parameter SptSingleThread

Pre-processor switch to enable/disable support for exclusive areas in the SPT driver (If all SPT driver calls are made from the same thread, there is no need for exclusive ares).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | true

Parameter SptVersionInfoApi

Pre-processor switch to enable/disable version info report for Spt API

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | true

Parameter SptDspEnable

Pre-processor switch to enable/disable DSP support from the SPT driver

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | true

Parameter SptRunPoll

Pre-processor switch to enable/disable polling support from the SPT driver (If this option is true, it allows the SPT context to be configured in polling mode at runtime, but does not mean that IRQ configuration is not an option).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | true

Container SptSetup

Contains initialization parameters for the SPT Driver.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Parameter SptSetupStructureName

The name of the structure to be used by Spt_Setup().

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

valueConfigClasses | defaultValue | sptInitInfo

Parameter SptDspEnable

Enable the BBE32 DSP to boot-up and start running the DSP Dispatcher.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | false

9.2 Dsp Host Driver Autosar Configuration Parameters

9.2.1 Dsp Host Driver Autosar Configuration Parameters

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module Dsphd
  - Container DsphdGeneral
    - Parameter DsphdDevErrorDetect
    - Parameter DsphdDevHaltOnError
    - Parameter DsphdVersionInfoApi

Module Dsphd

Included containers:

- DsphdGeneral

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	False
supportedConfigVariants	VARIANT-PRE-COMPILE

Container DsphdGeneral

Configuration of the DSP Host Driver module.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Parameter DsphdDevErrorDetect

Pre-processor switch to enable/disable development error detection for DSP Host Driver API

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | true

Parameter DsphdDevHaltOnError

Pre-processor switch to halt the DSP Host Driver in an infinite loop upon error detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | false

Parameter DsphdVersionInfoApi

`Pre-processor switch to enable/disable version info report for DSP Host Driver API`

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

valueConfigClasses | defaultValue | true

Index

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support